

Chapter 1: Course modalities and motivation

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

About myself

Sebastian Engelke

sebastian.engelke@unige.ch

- ▶ Born in Göttingen, Germany, where I did my PhD in mathematics/statistics.
- ▶ Worked at EPFL and HEC Lausanne since 2013.
- ▶ Joined the Research Center for Statistics in 2018.
- ▶ Research interests: extreme value theory; risk of meteorological rare events; spatial statistics; graphical models; machine learning; etc.
⇒ www.sengelke.com



Course details

Course Website

- ▶ On Moodle: <https://moodle.unige.ch/>
- ▶ Contains announcements, exercises, slides, data, code, etc.
- ▶ Course: “Machine Learning”; Password (later): ML2024

Lectures

- ▶ Lectures as videos on moodle (**watch at home before Tuesdays**)
- ▶ Tuesdays 16:15–18:00 (M R030): discussion about the lecture (questions, additional explanations, etc.)

Exercises

- ▶ Thursday 16:15–18:00; SCIII 1S081 - Edouard Kellenberger
- ▶ Weekly assignments on moodle (not graded, but important material for exam)
- ▶ Assistants: Manuel Hentschel (manuel.hentschel@unige.ch)
Christophe Valvason (christophe.valvason@unige.ch)

Recordings

- ▶ The classes and seminar are recorded and available on Mediaserver
- ▶ Presence in person is strongly recommended to have the optimal learning experience and interactions with classmates

Course assessment

Final exam (60% of grade)

- ▶ 2-hour written exam during the official examination period
- ▶ Closed book
 - ▶ Allowed: a non-programmable calculator
- ▶ Prohibited material:
 - ▶ Everything else (e.g., notes, books, cell phones, laptops, etc.)

Data challenge (40% of grade)

- ▶ Best prediction on supervised learning task
- ▶ Deliverables: Jupyter notebook and written report
- ▶ Teams of 3 students allowed
- ▶ Start: week of October 28, 2024; End: December 22, 2024

Retake exam (100% of grade)

- ▶ During official retake exam period
- ▶ Same format and rules as for final exam
- ▶ In case of an unexcused absence or failed attempt of the final exam, the student can choose (before the retake) whether to keep the grade of the semester project or have the retake exam count for 100%.

Main references

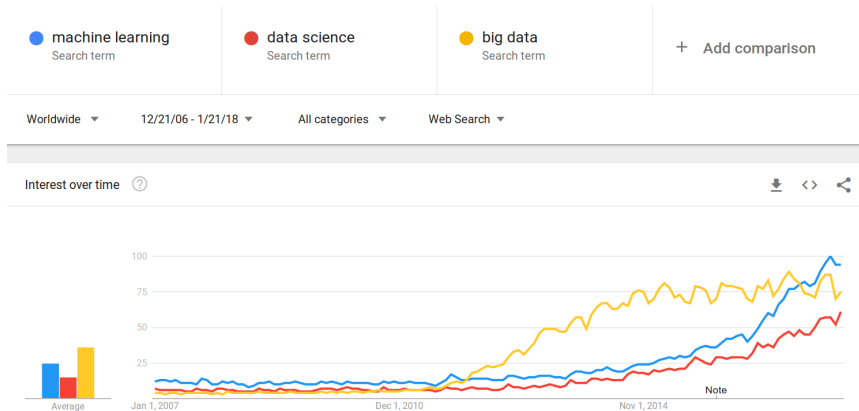
- [ISL] James, Witten, Hastie & Tibshirani. *An Introduction to Statistical Learning*, Springer.
Free pdf on <http://www-bcf.usc.edu/~gareth/ISL/>
 - [ESL] Hastie, Tibshirani & Friedman. *The Elements of Statistical Learning*, Springer.
Free pdf on <http://web.stanford.edu/~hastie/ElemStatLearn/>
 - [HOM] Géron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media, Inc.
- ▶ Several figures and data sets are taken from these books.

Some related websites

- ▶ Machine learning competition: <https://www.data-mining-cup.com/>
- ▶ More competitions: <https://www.kaggle.com/>
- ▶ News on machine learning related topics: <https://www.wired.com/>

Machine learning is trendy

Google Trends: interest in buzz words 2007–2017.



Best job in 2017

Glassdoor, best jobs in America in 2017.

1 Data Scientist



4.8 / 5
Job Score

4.4 / 5
Job Satisfaction

\$110,000
Median Base Salary

4,184
Job Openings

[View Jobs](#)

Culture: AlphaGo

After humanity spent thousands of years improving our tactics, computers tell us that humans are completely wrong... I would go as far as to say not a single human has touched the edge of the truth of Go. [Ke Jie; Go Champion]



Figure: In 2015, the first computer program won against a professional Go player, based on a combination of deep neural nets and reinforcement learning; see

<https://www.deepmind.com/research/highlighted-research/alphago>

Sociology: artificial intelligence



Technology: smart cars

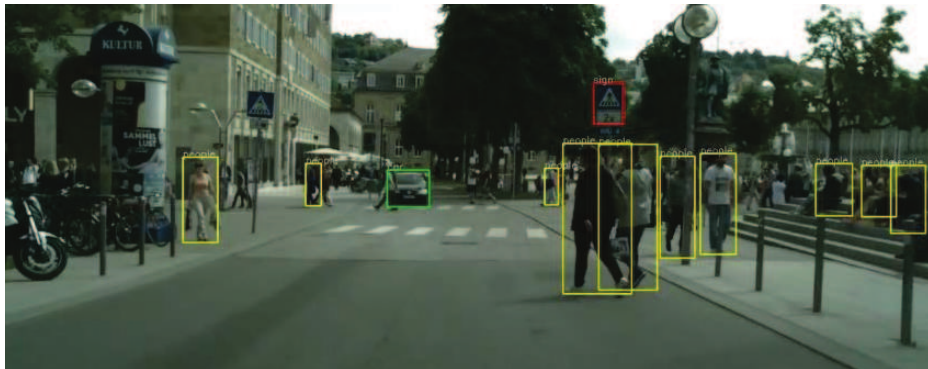
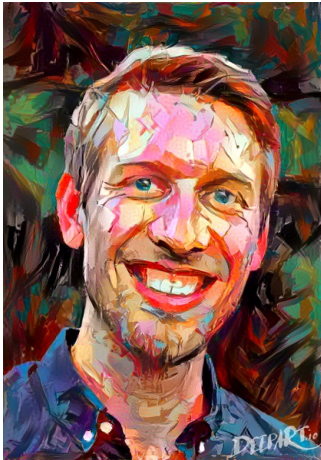


Figure: CES 2016: NVIDIA DRIVENet Demo - Visualizing a Self-Driving Future; see www.youtube.com/watch?v=HJ58dbd5g8g

Art: <https://deepart.io/>



Art: <https://deepart.io/>



Applications of machine learning

Business

- ▶ Work flows in businesses rely heavily on computer programs based on machine learning, e.g., market forecasting, customer segmentation and retention, fraud detection, etc.
- ▶ Most start-ups today develop products/technology based on machine learning.

Data mining and new technologies

- ▶ Self-customizing programs: e.g., Amazon, Netflix, recommender systems
- ▶ Computer vision, autonomous driving
- ▶ Language processing: ChatGPT

Science

- ▶ Applications in medicine such as disease diagnostics based on medical records
- ▶ Environmental applications such as weather forecasting

Artificial intelligence

- ▶ Robot navigation
- ▶ Game AI: chess, Go, computer games, etc.

Chapter 2: What is machine learning?

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

What is machine learning?

Machine learning aims at programming a computer to learn from information (**Data**) to perform a task (**Prediction, etc.**) in an optimal way in terms of a performance measure (**Loss Function**).

Recent success of **machine learning** due to advances in its main ingredients:

- (1) **Data availability**: huge growth due to automation, digitalization, web applications.
- (2) **Computing power**: significant increase thanks to GPU computing, parallelization, etc.
- (3) **The learning algorithms**: theoretical developments of new statistical models and training methods.

The statistical framework

Types of variables:

- ▶ **quantitative** variables take values in an ordered set, typically the real numbers $\mathbb{R}$;
- ▶ **qualitative** variables (also called **categorical** or **factor**) take values in a finite set $\mathcal{G} = \{\mathcal{G}_1, \dots, \mathcal{G}_q\}$ without ordering, e.g., {Yes, No}, {blue, red, green, yellow}, etc.

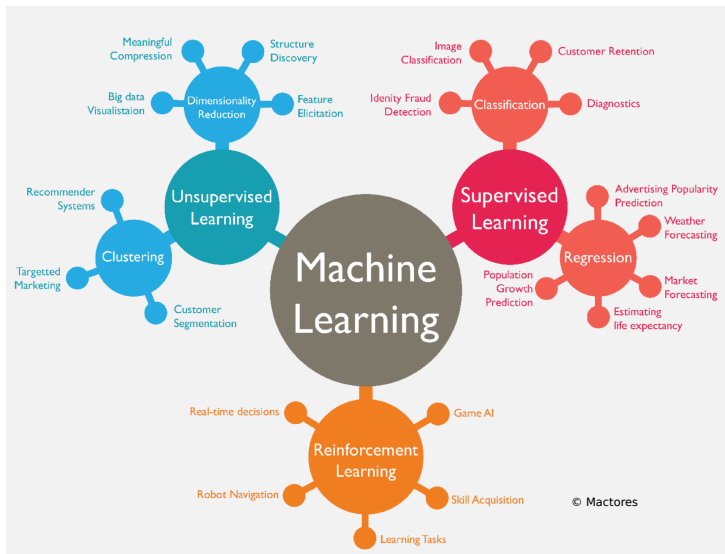
Supervised learning

- ▶ We observe **training data** (x_i, y_i) , $i = 1, \dots, n$ where
 - ▶ the inputs $x_i \in \mathbb{R}^p$ are called **predictors**, **covariates** or **features**;
 - ▶ the outputs y_i are called **responses** (in this course we assume they are univariate).
- ▶ The goal is to **predict** the (unknown) response y_0 for a new (known) predictor x_0 .
- ▶ **Regression**: the responses $y_i \in \mathbb{R}$ are quantitative.
- ▶ **Classification**: the responses $y_i \in \mathcal{G}$ are qualitative.

Unsupervised learning

- ▶ We only observe multivariate data $x_i \in \mathbb{R}^p$, $i = 1, \dots, n$ but without a particular response.
- ▶ The goal is to **summarize**, **understand**, **interpret** and **visualize** the data.

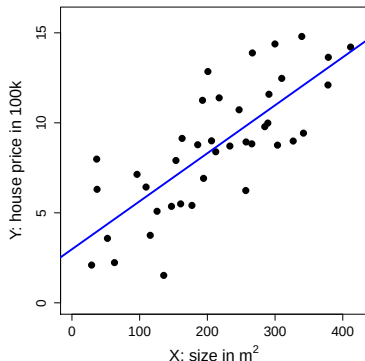
Machine Learning: an overview



Regression: simple linear model

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$

Goal: Predict unknown outcome y_0 for new predictor x_0 .



► Linear regression

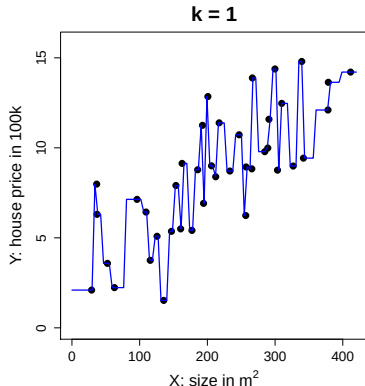
$$\hat{y}_0 = \hat{\beta}_0 + \hat{\beta}_1 x_0$$

► The numbers $\hat{\beta}_0, \hat{\beta}_1 \in \mathbb{R}$ are estimated model parameters.

Regression: the kNN method

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$

Goal: Predict unknown outcome y_0 for new predictor x_0 .



► The **k-Nearest-Neighbor (kNN)** method

$$\begin{aligned}\hat{y}_0 &= \frac{1}{k} \sum_{i: x_i \in N_k(x_0)} y_i \\ &= \text{ave}\{y_i : x_i \in N_k(x_0)\},\end{aligned}$$

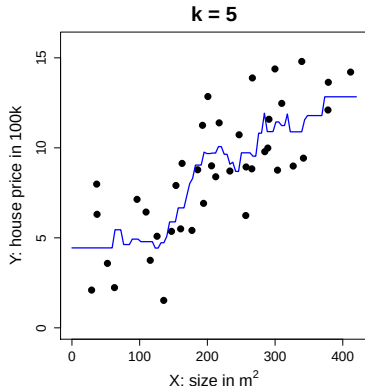
$N_k(x_0)$ are the k closest points x_i to x_0 .

► The number k is a tuning parameter.

Regression: the kNN method

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$

Goal: Predict unknown outcome y_0 for new predictor x_0 .



► The **k-Nearest-Neighbor (kNN)** method

$$\begin{aligned}\hat{y}_0 &= \frac{1}{k} \sum_{i: x_i \in N_k(x_0)} y_i \\ &= \text{ave}\{y_i : x_i \in N_k(x_0)\},\end{aligned}$$

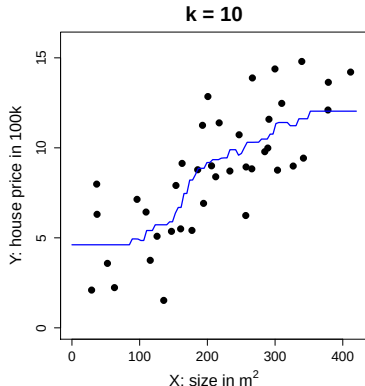
$N_k(x_0)$ are the k closest points x_i to x_0 .

► The number k is a tuning parameter.

Regression: the kNN method

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$

Goal: Predict unknown outcome y_0 for new predictor x_0 .



► The k-Nearest-Neighbor (kNN) method

$$\begin{aligned}\hat{y}_0 &= \frac{1}{k} \sum_{i: x_i \in N_k(x_0)} y_i \\ &= \text{ave}\{y_i : x_i \in N_k(x_0)\},\end{aligned}$$

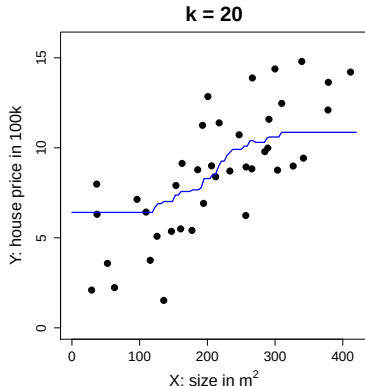
$N_k(x_0)$ are the k closest points x_i to x_0 .

► The number k is a tuning parameter.

Regression: the kNN method

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$

Goal: Predict unknown outcome y_0 for new predictor x_0 .



► The k-Nearest-Neighbor (kNN) method

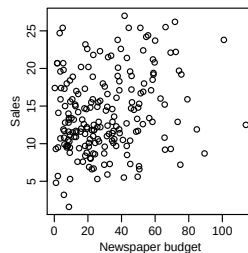
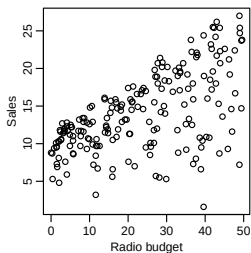
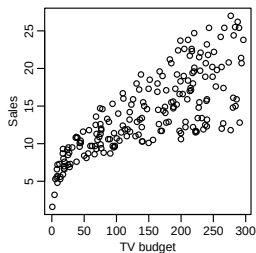
$$\begin{aligned}\hat{y}_0 &= \frac{1}{k} \sum_{i: x_i \in N_k(x_0)} y_i \\ &= \text{ave}\{y_i : x_i \in N_k(x_0)\},\end{aligned}$$

$N_k(x_0)$ are the k closest points x_i to x_0 .

► The number k is a tuning parameter.

Business application: advertising

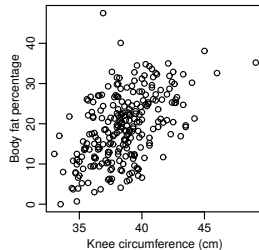
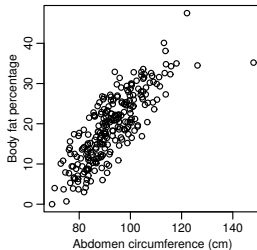
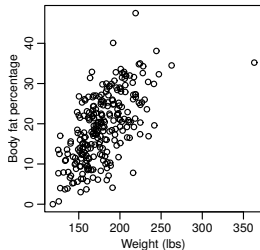
The **Advertising** data set contains **sales**, in thousands of units, as a function of **TV**, **radio**, and **newspaper** budgets, in thousands of dollars, for 200 different markets.



Medical application: body fat

Data set **bodyfat** (R library **mfp**). The data set contains body fat estimates (**siri**) for 252 men with measurements of different body attributes. The first 10 measurements and a plot of the responses versus some of the predictors:

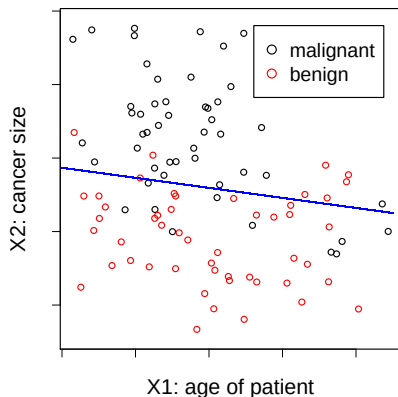
siri	age	weight	height	neck	chest	abdo	hip	thigh	knee	ankle	biceps	forearm	wrist
12.3	23	154.25	67.75	36.2	93.1	85.2	94.5	59.0	37.3	21.9	32.0	27.4	17.1
6.1	22	173.25	72.25	38.5	93.6	83.0	98.7	58.7	37.3	23.4	30.5	28.9	18.2
25.3	22	154.00	66.25	34.0	95.8	87.9	99.2	59.6	38.9	24.0	28.8	25.2	16.6
10.4	26	184.75	72.25	37.4	101.8	86.4	101.2	60.1	37.3	22.8	32.4	29.4	18.2
28.7	24	184.25	71.25	34.4	97.3	100.0	101.9	63.2	42.2	24.0	32.2	27.7	17.7
20.9	24	210.25	74.75	39.0	104.5	94.4	107.8	66.0	42.0	25.6	35.7	30.6	18.8
19.2	26	181.00	69.75	36.4	105.1	90.7	100.3	58.4	38.3	22.9	31.9	27.8	17.7
12.4	25	176.00	72.50	37.8	99.6	88.5	97.1	60.0	39.4	23.2	30.5	29.0	18.8
4.1	25	191.00	74.00	38.1	100.9	82.5	99.9	62.9	38.3	23.8	35.9	31.1	18.2
11.7	23	198.25	73.50	42.1	99.6	88.6	104.1	63.1	41.7	25.0	35.6	30.0	19.2



Classification: simple linear regression

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$, where $y_i \in \mathcal{G} = \{0, 1\}$

Goal: Predict unknown class y_0 for new predictor x_0 .



- Directly apply **linear regression** treating the classes as quantitative values 0 and 1. The prediction is

$$\hat{y}_0 = \begin{cases} 1, & \text{if } \hat{\beta}_0 + \hat{\beta}^T x_0 \geq 1/2, \\ 0, & \text{otherwise.} \end{cases}$$

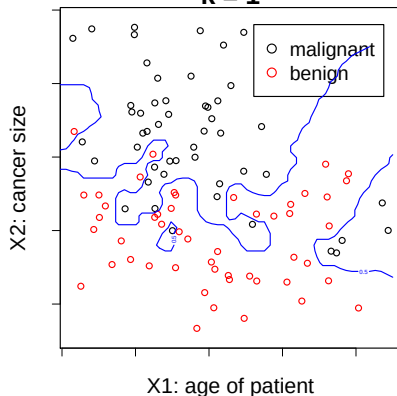
- The numbers $\hat{\beta}_0, \hat{\beta}_1 \in \mathbb{R}$ are estimated model parameters.

Classification: the kNN method

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$, where $y_i \in \mathcal{G} = \{0, 1\}$

Goal: Predict unknown class y_0 for new predictor x_0 .

k = 1



- ▶ The **k-Nearest-Neighbor (kNN)** method predicts the majority of classes in the neighborhood

$$\hat{y}_0 = \begin{cases} 1, & \text{if } \frac{1}{k} \sum_{i: x_i \in N_k(x_0)} \mathbf{1}\{y_i = 1\} > 1/2, \\ 0, & \text{otherwise.} \end{cases}$$

$N_k(x_0)$ are the k closest points x_i to x_0 .

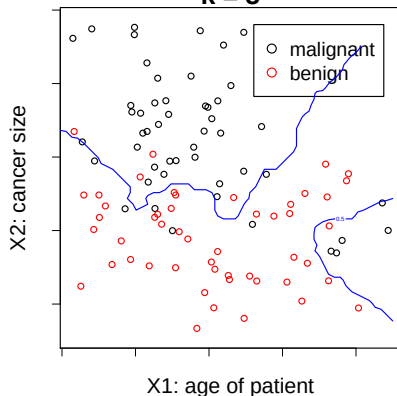
- ▶ The number k is a tuning parameter.

Classification: the kNN method

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$, where $y_i \in \mathcal{G} = \{0, 1\}$

Goal: Predict unknown class y_0 for new predictor x_0 .

k = 5



- ▶ The **k-Nearest-Neighbor (kNN)** method predicts the majority of classes in the neighborhood

$$\hat{y}_0 = \begin{cases} 1, & \text{if } \frac{1}{k} \sum_{i: x_i \in N_k(x_0)} \mathbf{1}\{y_i = 1\} > 1/2, \\ 0, & \text{otherwise.} \end{cases}$$

$N_k(x_0)$ are the k closest points x_i to x_0 .

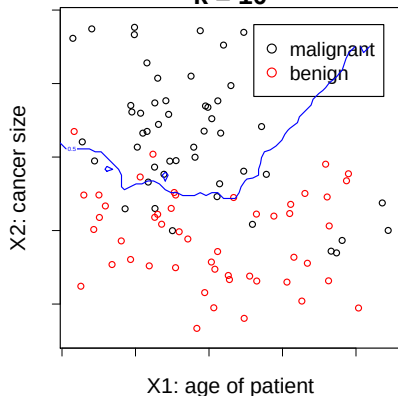
- ▶ The number k is a tuning parameter.

Classification: the kNN method

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$, where $y_i \in \mathcal{G} = \{0, 1\}$

Goal: Predict unknown class y_0 for new predictor x_0 .

k = 10



- The **k-Nearest-Neighbor (kNN) method** predicts the majority of classes in the neighborhood

$$\hat{y}_0 = \begin{cases} 1, & \text{if } \frac{1}{k} \sum_{i: x_i \in N_k(x_0)} \mathbf{1}\{y_i = 1\} > 1/2, \\ 0, & \text{otherwise.} \end{cases}$$

$N_k(x_0)$ are the k closest points x_i to x_0 .

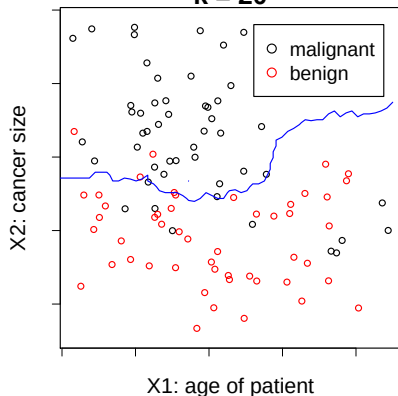
- The number k is a tuning parameter.

Classification: the kNN method

Data: Observations $(x_1, y_1), \dots, (x_n, y_n)$, where $y_i \in \mathcal{G} = \{0, 1\}$

Goal: Predict unknown class y_0 for new predictor x_0 .

k = 20



- ▶ The **k-Nearest-Neighbor (kNN)** method predicts the majority of classes in the neighborhood

$$\hat{y}_0 = \begin{cases} 1, & \text{if } \frac{1}{k} \sum_{i: x_i \in N_k(x_0)} \mathbf{1}\{y_i = 1\} > 1/2, \\ 0, & \text{otherwise.} \end{cases}$$

$N_k(x_0)$ are the k closest points x_i to x_0 .

- ▶ The number k is a tuning parameter.

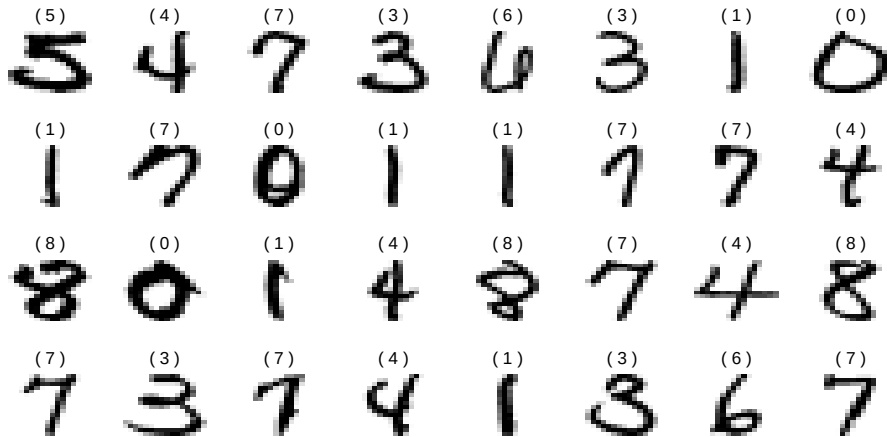
Example: spam classification

The **spam** data set (R library **kernlab**). 4601 emails classified as one of two classes, i.e., $G \in \{\text{spam}, \text{nonspam}\}$, based on and 57 predictors indicating the frequency of certain words and characters in the e-mail. A subset of the data:

	type	george	free	credit	money	hp	business	your	!	capitalTotal
1222	spam	0.00	0.00	5.19	0.64	0.00	1.29	1.29	0.09	135.00
1712	spam	0.00	0.44	0.00	0.00	0.00	0.00	0.00	0.00	186.00
2635	nonspam	0.66	1.33	0.00	0.22	3.34	0.00	0.44	0.37	411.00
4176	nonspam	0.00	0.00	0.00	0.00	0.00	0.00	0.22	0.04	97.00
928	spam	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.40	495.00
4129	nonspam	0.00	0.00	0.00	1.16	0.00	0.00	1.16	0.49	34.00
4341	nonspam	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.77	18.00
3036	nonspam	0.00	0.00	0.00	0.00	0.00	0.00	2.32	0.00	37.00
2890	nonspam	0.00	0.00	0.00	0.00	1.72	0.00	2.58	0.11	58.00
284	spam	0.00	0.40	1.81	0.60	0.00	1.61	2.62	1.45	513.00
946	spam	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.03	339.00
811	spam	0.00	0.00	1.02	0.00	0.34	0.00	0.68	0.90	1330.00
3153	nonspam	0.38	0.00	0.00	0.00	0.90	0.00	0.00	0.00	1232.00
1763	spam	0.00	0.00	0.00	0.00	0.00	0.00	11.11	0.00	4.00
3532	nonspam	2.00	0.00	0.00	0.00	4.00	0.00	0.00	0.00	46.00
2283	nonspam	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	14.00
3291	nonspam	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00
4547	nonspam	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	5.00
1742	spam	0.00	0.00	0.00	0.00	0.00	0.00	0.47	0.69	239.00
3563	nonspam	4.16	0.00	0.00	0.00	8.33	0.00	0.00	0.00	30.00

Example: handwritten characters/digits recognition

Data set from [\[ESL\]](#) homepage: handwritten digits (16×16 grey scale images). There are ten classes, namely, $G = \{0, 1, \dots, 9\}$.



Example: CIFAR-10 data set

The **CIFAR-10 data set** contains 60'000 image of size 32×32 . There are ten classes, namely, $G = \{\text{airplane}, \text{automobile}, \dots, \text{truck}\}$.

airplane



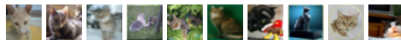
automobile



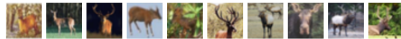
bird



cat



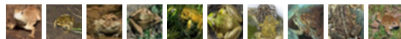
deer



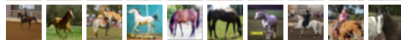
dog



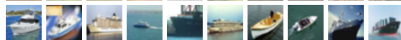
frog



horse



ship



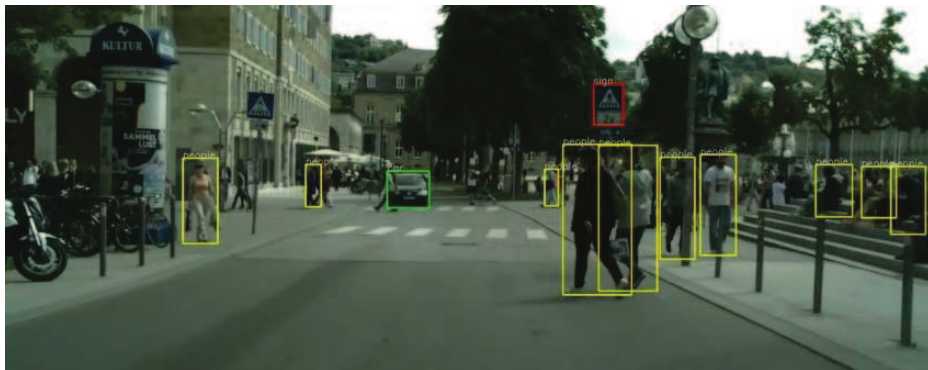
truck



Example: smart cars

Smart cars classify objects according to

$G \in \{\text{cars, pedestrian, buildings, road signs, traffic lights,...}\}$ and predict how these objects will move, and take decisions based on these predictions.



CES 2016: NVIDIA DRIVENet Demo - Visualizing a Self-Driving Future (part 5)

(www.youtube.com/watch?v=HJ58dbd5g8g)

What is the right model?

All models are wrong, some are useful.

[George Box; Statistician]

Everything should be made as simple as possible. But not simpler.

[Albert Einstein; Physicist]

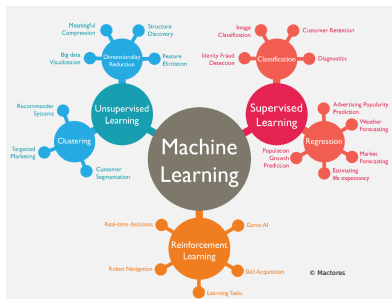
In this course

(1) Statistical concepts

- ▶ Modeling, inference, prediction.
- ▶ Model assessment and selection: training versus test error, cross-validation, etc.

(2) Machine learning methods

- ▶ Regression: linear models, basis functions, regularization for high-dimensional data.
- ▶ Classification: linear methods, trees and random forest, support vector machines.
- ▶ Recent developments: convolutional (deep) neural networks, reinforcement learning, etc.



(3) Applications

- ▶ Analysis of simulated and real data sets.
- ▶ Image recognition

(4) Practical skills

- ▶ Programming with python.
- ▶ Working with data and interpretation of the results.

Chapter 3: Regression framework

1. Course modalities and motivation
2. What is machine learning?
3. **Regression framework**
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Regression: the stochastic framework

The **stochastic/population** model

- ▶ We have a quantitative response Y and p predictors $X_1, \dots, X_p$.
- ▶ We assume there is a relationship between Y and $X = (X_1, \dots, X_p)$

$$Y = f(X) + \epsilon,$$

where ϵ is a random **error term** with $E(\epsilon) = 0$ and $\text{Var}(\epsilon) = \sigma^2$.

- ▶ The function f is fixed but **unknown** and represents the **systematic** information that X provides about Y .

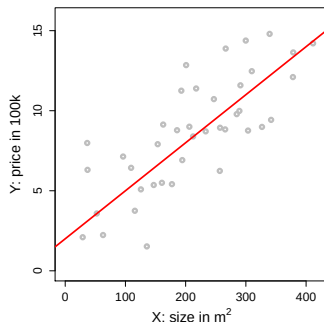
Example: [$Y = \text{house price (100k)}$; $X = \text{size (m}^2\text{)}$]

$$Y = 2 + 0.03X + \epsilon$$

$$\text{house price} = 2 + 0.03 \times \text{size} + \text{error}$$

- ▶ This true relationship $f(X) = 2 + 0.03X$ is called the **population regression line**.
- ▶ Note: (X, Y) is a $(p + 1)$ -dimensional random vector. In this simulated example:

$$X = \mathcal{N}(0, \sigma_X^2), \text{ and } Y = 2 + 0.03X + \mathcal{N}(0, \sigma_\epsilon^2).$$



Regression: the statistical framework

The training data

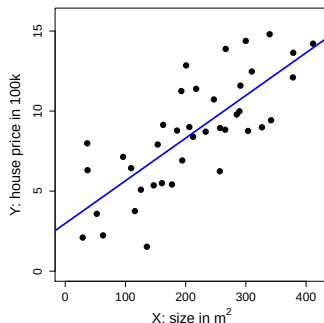
- ▶ Assume we are given n measurements $(x_1, y_1), \dots, (x_n, y_n)$ of (X, Y) , where $x_i = (x_{i1}, \dots, x_{ip})^\top$, and
 - ▶ the inputs $x_i \in \mathbb{R}^p$ are called **predictors**;
 - ▶ the outputs y_i are called **responses**.
- ▶ We will use this data to **learn** the unknown relationship f between Y and X , i.e., to **train** (or **estimate**) a predictive model $\hat{f}$.

Example: [$Y = \text{house price (100k)}$; $X = \text{size (m}^2\text{)}$]

$$\text{house price} \approx \hat{f}(\text{size}) = 2.977 + 0.027 \times \text{size}$$

- ▶ The training data are realizations of the random vector (X, Y) , e.g.,

	X	Y
(x_1, y_1)	327	9.0
(x_2, y_2)	154	7.9
(x_3, y_3)	233	8.7
(x_4, y_4)	206	9.0
(x_5, y_5)	96	7.1
(x_6, y_6)	257	6.2
...	...	...



Why do we estimate f ?

1. Prediction (Machine learning)

- ▶ A set of inputs X are readily available, but Y cannot be easily obtained.
- ▶ We can predict Y by

$$\hat{Y} = \hat{f}(X),$$

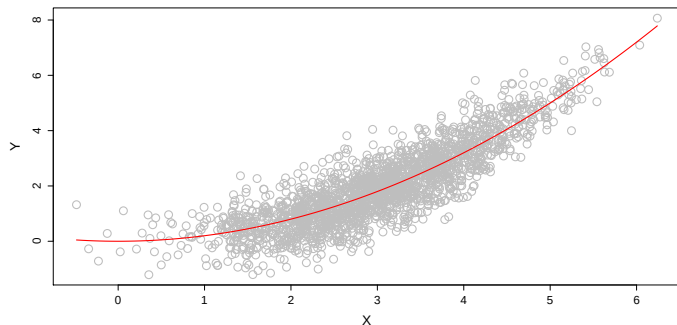
where $\hat{f}$ is our predictive model for f .

- ▶ The function $\hat{f}$ is often treated as black box: we are not concerned with the exact form of $\hat{f}$ provided it yields good prediction for Y .

2. Inference (Classical multivariate statistics)

- ▶ We are often interested in understanding how Y is affected as $X_1, \dots, X_p$ change.
- ▶ Cannot treat $\hat{f}$ as black box, we need to know the exact form.
- ▶ We want to answer questions such as:
 - ▶ Which predictor is associated with the response?
 - ▶ What is the relationship of the response and each predictor (positive, negative,...)?
 - ▶ Is the relationship linear or more complicated?
 - ▶ etc.

What is a good $\hat{f}$?



- For a fixed value of X , say $X = 3$ is there an **optimal prediction** $\hat{f}(X)$?
- There might be many values at $X = 3$, and a good choice, minimizing squared error, is

$$\hat{f}(3) = E(Y \mid X = 3),$$

where $E(Y \mid X = 3)$ is the **expected value** (average) of Y given $X = 3$.

The loss function

- ▶ We have to specify what we mean with a **good prediction**. This will depend on the application.
- ▶ Introduce a **loss function** L that measures the discrepancy between a response y and its prediction $\hat{y}$.
- ▶ In the regression setup, we typically use **squared error loss**, i.e.,

$$L(y, \hat{y}) = (y - \hat{y})^2.$$

- ▶ For our predictive model $\hat{f}$, the **expected squared prediction error** is

$$\text{Err}_{\hat{f}} = \mathbb{E}[\{Y - \hat{f}(X)\}^2],$$

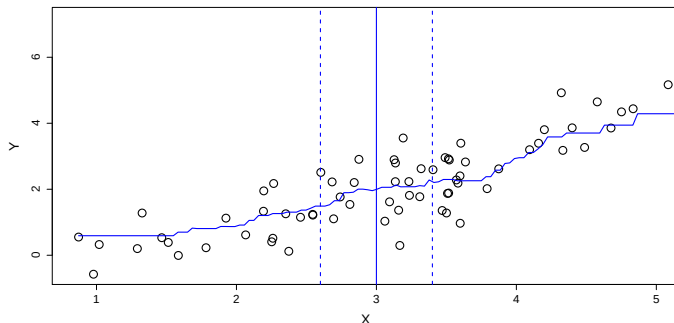
where the expectation is taken over (X, Y) .

- ▶ Ideally, we would like to find the optimal f^* that **minimizes** Err_{f^*} .
- ▶ In fact, f^* is the so-called **regression function**

$$f^*(x) = \mathbb{E}(Y|X = x),$$

which satisfies $f^* = f$ if $Y = f(X) + \epsilon$.

How do we estimate $\hat{f}$?



- ▶ Typically we have **few or no** data points with $X = 3$.
- ▶ So we cannot compute $E(Y \mid X = 3)$!
- ▶ The **k-Nearest-Neighbors** method aims at approximating this expectation by the average

$$\hat{f}(x_0) = \text{Ave}(Y \mid X \in N_k(x_0)),$$

where $N_k(x_0)$ are the k closest points x_i to $x_0 = 3$.

- ▶ The number k is a **tuning parameter**.

How do we estimate $\hat{f}$?

- ▶ Nearest neighbor averaging works good for small dimension p and a large training data size n .
- ▶ When the number of predictors $X_1, \dots, X_p$ is larger, then kNN does not work well! The reason is the **curse of dimensionality**.
- ▶ We will study **structured methods** that
 - ▶ need **less data** to fit;
 - ▶ may be nicely **interpretable**;
 - ▶ but at the price of **lower flexibility**.
- ▶ The difficulty is to find a good **compromise** between flexibility and parsimony.

The empirical loss: model evaluation

How can we **estimate** the $\text{Err}_{\hat{f}} = \mathbb{E}[\{Y - \hat{f}(X)\}^2]$?

The **test data**

- ▶ For model evaluation we require a new, independent test data set $\mathcal{T}_m = \{(\tilde{x}_1, \tilde{y}_1), \dots, (\tilde{x}_m, \tilde{y}_m)\}$.
- ▶ For each input $\tilde{x}_i$ we **predict the response** with our model: $\hat{y}_i = \hat{f}(\tilde{x}_i)$.
- ▶ We compare this prediction with the observed output $\tilde{y}_i$.
- ▶ The **empirical prediction error** of our model $\hat{f}$ is the empirical version of $\text{Err}_{\hat{f}}$, namely

$$\text{err}_{\hat{f}} = \frac{1}{m} \sum_{i=1}^m \{\tilde{y}_i - \hat{f}(\tilde{x}_i)\}^2,$$

also called **mean squared error**.

- ▶ In practice: separate all data into training and test and do **cross-validation** (later).

Chapter 4: Linear Regression

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
- 4. Linear Regression**
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Simple linear regression

► Recall: $Y = f(X) + \epsilon$

► **Simple linear model:**

$$f(X) = \beta_0 + \beta_1 X,$$

where β_0 and β_1 are model coefficients or parameters.

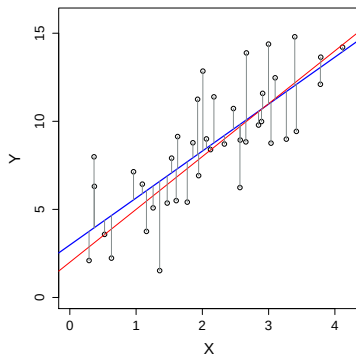
► For training data $\{(x_i, y_i)\}_{i=1}^n$, suppose we have an estimate $(\hat{\beta}_0, \hat{\beta}_1)$ of (β_0, β_1) .

► **Fitted value:** $\hat{y}_i = \hat{f}(x_i) = \hat{\beta}_0 + \hat{\beta}_1 x_i$

► **Residual:** $e_i = y_i - \hat{y}_i$

► **Residual sum of squares (RSS)**

$$\begin{aligned}\text{RSS}(\hat{\beta}_0, \hat{\beta}_1) &= e_1^2 + e_2^2 + \cdots + e_n^2 \\ &= \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 \\ &= \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2\end{aligned}$$



Estimation of parameters by least squares

- ▶ To estimate (β_0, β_1) , we minimize the RSS:

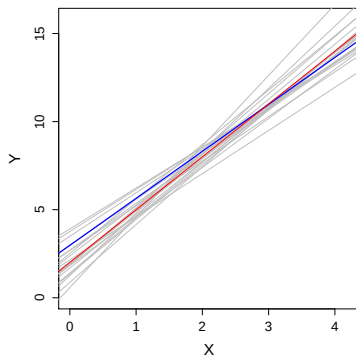
$$\begin{aligned}(\hat{\beta}_0, \hat{\beta}_1) &= \operatorname{argmin}_{(\beta_0, \beta_1) \in \mathbb{R}^2} \operatorname{RSS}(\beta_0, \beta_1) \\ &= \operatorname{argmin}_{(\beta_0, \beta_1) \in \mathbb{R}^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2\end{aligned}$$

- ▶ How do we solve this optimization problem?
- ▶ Numerically with computer algorithm (**gradient descent**, etc.)
- ▶ Analytically by differentiation (if possible):

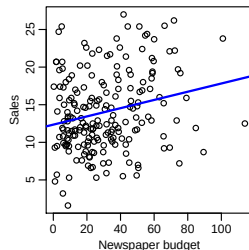
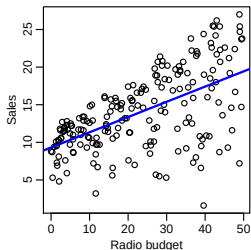
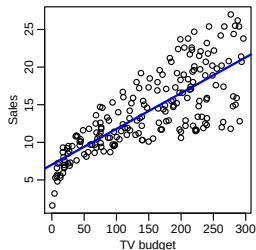
$$\begin{aligned}\hat{\beta}_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \\ \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x},\end{aligned}$$

where $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ and $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ are **sample means**.

- ▶ Note: $(\hat{\beta}_0, \hat{\beta}_1)$ are **random**, since for different training data we obtain different estimates!



Advertising data



Coefficient	Estimate	Std. Error	<i>t</i> value	<i>p</i> -value
Intercept (β_0)	7.032594	0.457843	15.36	$< 2e-16$
TV (β_1)	0.047537	0.002691	17.67	$< 2e-16$
Coefficient	Estimate	Std. Error	<i>t</i> value	<i>p</i> -value
Intercept (β_0)	9.31164	0.56290	16.542	$< 2e-16$
radio (β_1)	0.20250	0.02041	9.921	$< 2e-16$
Coefficient	Estimate	Std. Error	<i>t</i> value	<i>p</i> -value
Intercept (β_0)	12.35141	0.62142	19.88	$< 2e-16$
newspaper (β_1)	0.05469	0.01658	3.30	0.00115

Multiple linear regression

- ▶ We consider p predictors $X = (X_1, \dots, X_p)$ for the response Y . The model is

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon.$$

- ▶ For the **Advertising** data set:

$$\text{sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times \text{newspaper} + \epsilon.$$

- ▶ The training data $\{(x_i, y_i)\}_{i=1}^n$ with $x_i = (x_{i1}, \dots, x_{ip})^\top$ can be written as an $n \times p$ matrix

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

- ▶ For notational convenience, we sometimes use the **dot product** between $x, y \in \mathbb{R}^p$

$$x^\top y = \sum_{i=1}^p x_i y_i = x_1 y_1 + \dots + x_p y_p.$$

Estimation and prediction in multiple linear regression

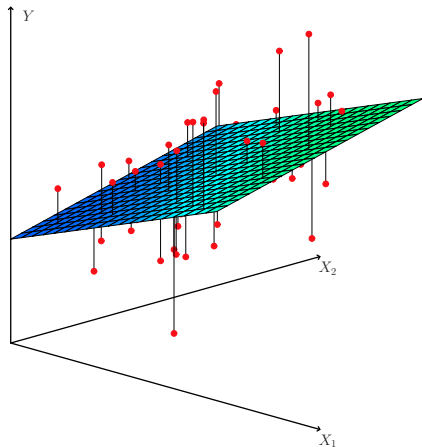
- As before, we obtain parameter estimates $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ as those parameters that minimize the **RSS**

$$\begin{aligned}\text{RSS}(\beta_0, \dots, \beta_p) &= \sum_{i=1}^n (y_i - \beta_0 - \mathbf{x}_i^\top \boldsymbol{\beta})^2 \\ &= \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip})^2\end{aligned}$$

- We make **predictions** at a new point $\mathbf{x}_0 = (x_{01}, \dots, x_{0p})$ by

$$\begin{aligned}\hat{y}_0 &= \hat{\beta}_0 + \mathbf{x}_0^\top \hat{\boldsymbol{\beta}} \\ &= \hat{\beta}_0 + \hat{\beta}_1 x_{01} + \dots + \hat{\beta}_p x_{0p},\end{aligned}$$

where $\hat{\boldsymbol{\beta}} = (\hat{\beta}_1, \dots, \hat{\beta}_p)$.



Advertising data

Coefficient	Estimate	Std. Error	<i>t</i> value	<i>p</i> -value
Intercept	2.939	0.3119	9.42	$< 2e-16$
TV	0.046	0.0014	32.81	$< 2e-16$
radio	0.189	0.0086	21.89	$< 2e-16$
newspaper	-0.001	0.0059	-0.18	0.8599

Correlations:

	TV	radio	newspaper
TV	1	0.0548	0.0567
radio		1	0.354
newspaper			1

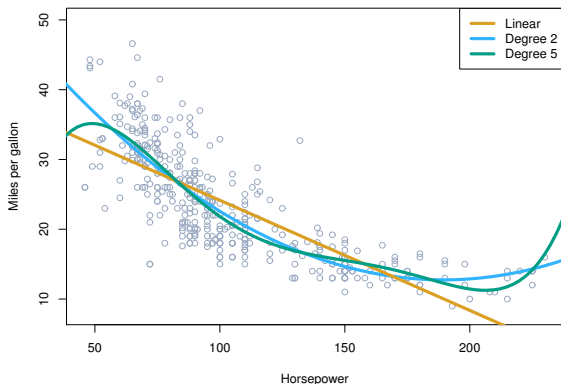
Example: the Auto data set

- ▶ The **Auto** data set contains measurements for $n = 397$ cars, such as **mpg** (miles per gallon), **horsepower**, **year**, etc.
- ▶ We can fit the model

$$\text{mpg} = \beta_0 + \beta_1 \times \text{horsepower} + \beta_2 \times \text{horsepower}^2 + \epsilon,$$

this is still a linear model (in $X_1 = \text{horsepower}$ and $X_2 = \text{horsepower}^2$).

- ▶ In this case, both β_1 and β_2 are significant.
- ▶ Even higher polynomials can be fitted.



Beyond linearity: linear basis functions

- ▶ Most often the true function $f(X)$ is not linear in the predictors $X_1, \dots, X_p$! But we can adjust the linear model.
- ▶ Instead of the $X = (X_1, \dots, X_p)$ we consider transformation of them. Let $h_m(X) : \mathbb{R}^p \rightarrow \mathbb{R}$ be the m th transformation, $m = 1, \dots, M$. The model is

$$f(X) = \sum_{m=1}^M \beta_m h_m(X).$$

- ▶ This model is **linear** in the new predictors, the **basis functions** $h_1(X), \dots, h_M(X)$; estimation and prediction can be applied as before!

Possible choices for h_m are:

- ▶ $h_m(X) = X_m$, $m = 1, \dots, p$, essentially recovers the original linear model.
- ▶ $h_m(X) = X_j^2$ or $h_m(X) = X_j X_k$ allows to achieve higher order polynomials.
- ▶ $h_m(X) = \log(X_j)$, $\sqrt{X_j}$ covers other non-linear transformation.
- ▶ etc.

Chapter 5: Gradient Descent

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. **Gradient Descent**
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Numerical optimization: gradient descent

$$(\hat{\beta}_0, \hat{\beta}_1) = \operatorname{argmin}_{(\beta_0, \beta_1) \in \mathbb{R}^2} \operatorname{RSS}(\beta_0, \beta_1) = \operatorname{argmin}_{(\beta_0, \beta_1) \in \mathbb{R}^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

For simplicity: fix β_1 to the true value β_1^* and only estimate β_0 by least squares.

Gradient descent

- ▶ Start with an arbitrary initial value $\beta_0^{(0)} \in \mathbb{R}$.
- ▶ In the i th step:
- ▶ Compute the **gradient/derivative** of J at the position $\beta_0^{(i-1)}$:

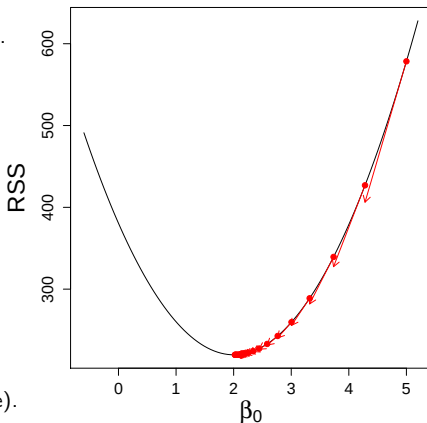
$$\nabla J(\beta_0^{(i-1)}) = \left. \frac{\partial J(\beta_0)}{\partial \beta_0} \right|_{\beta_0 = \beta_0^{(i-1)}}$$

- ▶ Update your estimate to

$$\beta_0^{(i)} = \beta_0^{(i-1)} - \alpha \nabla J(\beta_0^{(i-1)}),$$

where α is a **tuning parameter**.

- ▶ Stop when converged ($\beta_0^{(i)}$ changes no more).



Numerical optimization: gradient descent

$$\hat{\beta}_0 = \operatorname{argmin}_{\beta_0 \in \mathbb{R}} \operatorname{RSS}(\beta_0) = \operatorname{argmin}_{\beta_0 \in \mathbb{R}} J(\beta_0)$$

For simplicity: fix β_1 to the true value β_1^* and only estimate β_0 by least squares.

Gradient descent

- ▶ Start with an arbitrary initial value $\beta_0^{(0)} \in \mathbb{R}$.
- ▶ In the i th step:
- ▶ Compute the **gradient/derivative** of J at the position $\beta_0^{(i-1)}$:

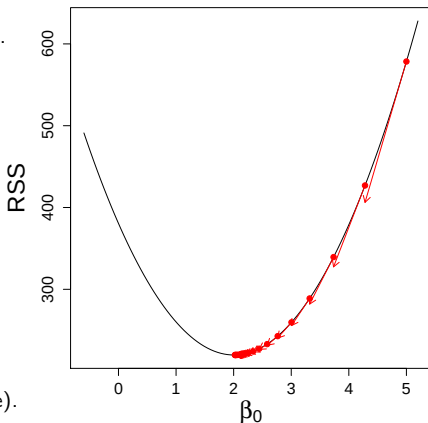
$$\nabla J(\beta_0^{(i-1)}) = \left. \frac{\partial J(\beta_0)}{\partial \beta_0} \right|_{\beta_0 = \beta_0^{(i-1)}}$$

- ▶ Update your estimate to

$$\beta_0^{(i)} = \beta_0^{(i-1)} - \alpha \nabla J(\beta_0^{(i-1)}),$$

where α is a **tuning parameter**.

- ▶ Stop when converged ($\beta_0^{(i)}$ changes no more).



Gradient descent: higher dimensions

$$(\hat{\beta}_0, \hat{\beta}_1) = \operatorname{argmin}_{(\beta_0, \beta_1) \in \mathbb{R}^2} \operatorname{RSS}(\beta_0, \beta_1) = \operatorname{argmin}_{\theta \in \mathbb{R}^d} J(\theta)$$

Gradient descent

- ▶ Start with an arbitrary initial value $\theta^{(0)} \in \mathbb{R}^d$.
- ▶ In the i th step:
- ▶ Compute the **gradient/derivative** of J at the position $\theta^{(i-1)}$:

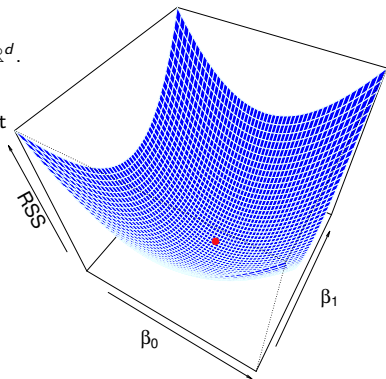
$$\nabla J(\theta^{(i-1)}) = \left. \frac{\partial J(\theta)}{\partial \theta} \right|_{\theta=\theta^{(i-1)}} \in \mathbb{R}^d$$

- ▶ Update your estimate to

$$\theta^{(i)} = \theta^{(i-1)} - \alpha \nabla J(\theta^{(i-1)}) \in \mathbb{R}^d,$$

where α is a **tuning parameter**.

- ▶ Stop when converged ($\theta^{(i)}$ changes no more).



Gradient descent: higher dimensions

$$(\hat{\beta}_0, \hat{\beta}_1) = \operatorname{argmin}_{(\beta_0, \beta_1) \in \mathbb{R}^2} \text{RSS}(\beta_0, \beta_1) = \operatorname{argmin}_{\theta \in \mathbb{R}^d} J(\theta)$$

Gradient descent

- ▶ Start with an arbitrary initial value $\theta^{(0)} \in \mathbb{R}^d$.
- ▶ In the i th step:
- ▶ Compute the **gradient/derivative** of J at the position $\theta^{(i-1)}$:

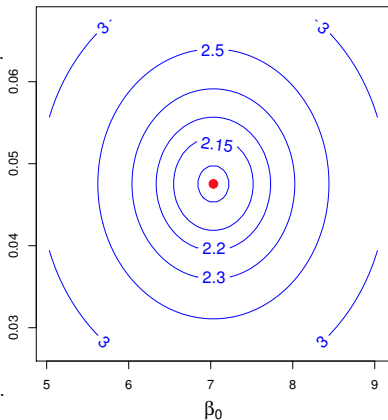
$$\nabla J(\theta^{(i-1)}) = \left. \frac{\partial J(\theta)}{\partial \theta} \right|_{\theta=\theta^{(i-1)}} \in \mathbb{R}^d$$

- ▶ Update your estimate to

$$\theta^{(i)} = \theta^{(i-1)} - \alpha \nabla J(\theta^{(i-1)}) \in \mathbb{R}^d,$$

where α is a **tuning parameter**.

- ▶ Stop when converged ($\theta^{(i)}$ changes no more).



Gradient descent: remarks

- ▶ Gradient descent can be used to solve numerically a minimization problem

$$\operatorname{argmin}_{\theta \in \mathbb{R}^d} J(\theta).$$

- ▶ The ingredients are the **objective function** J and its derivatives in all directions, namely the **gradient** ∇J .
- ▶ Recall that the gradient $\nabla J(\theta)$ points in the direction of the **steepest increase** of J at the point θ .
- ▶ Important restriction: J needs to be differentiable!
- ▶ Gradient descent is a generic method that typically finds a **local minimum**.
- ▶ Only if the problem is convex, convergence to the **global minimum** can be guaranteed.
- ▶ The tuning parameter, also called **learning rate**, has to be adapted to the problem (many techniques exist).
- ▶ In complex models, gradient descent is the only way to find a (local) minimum.
- ▶ Depending on the model, different implementations/approximations exist, e.g., **stochastic gradient descent**, **backpropagation**.

Chapter 6: Training versus test error

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
- 6. Training versus test error**
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Training versus test error

Training Data

- ▶ Fit our model $\hat{f}$ on the training data $\{(x_1, y_1), \dots, (x_n, y_n)\}$, the n measurements of (X, Y) .
- ▶ Compute the **training error** as

$$\begin{aligned}\text{MSE}_{\text{Tr}} &= \text{RSS}/n \\ &= \frac{1}{n} \sum_{i=1}^n \{y_i - \hat{f}(x_i)\}^2.\end{aligned}$$

- ▶ Since higher model complexity always reduces this error, training error is overly **optimistic**!

Test Data

- ▶ The **test data** are m new samples $\{(\tilde{x}_1, \tilde{y}_1), \dots, (\tilde{x}_m, \tilde{y}_m)\}$ of (X, Y) .
- ▶ We test our model $\hat{f}$ fitted to the training data by predicting $\hat{y}_i = \hat{f}(\tilde{x}_i)$ at all $\tilde{x}_i$.
- ▶ The **test/generalization error** is

$$\text{MSE}_{\text{Te}} = \frac{1}{m} \sum_{i=1}^m \{\tilde{y}_i - \hat{f}(\tilde{x}_i)\}^2.$$

- ▶ The test error is an estimate of the **expected prediction error**

$$\text{Err}_{\hat{f}} = \mathbb{E}[\{Y - \hat{f}(X)\}^2],$$

Training versus test error

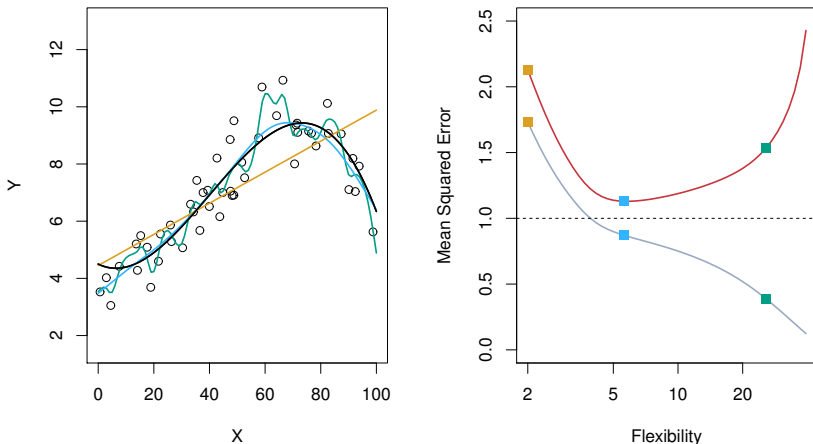


Figure: Left: Data simulated from f , the black line, and three estimates, the linear fit (orange) and higher order polynomials (blue and green). Right: Training error MSE_{Tr} (grey) and test error MSE_{Te} (red) for the three fits.

The Bias–Variance decomposition

- ▶ As usually, we assume $Y = f(X) + \epsilon$, with $E(\epsilon) = 0$ and $\text{Var}(\epsilon) = \sigma_\epsilon^2$.
- ▶ For a regression fit $\hat{f}$, we compute the expected prediction error at an input point $X = x_0$:

$$\begin{aligned}\text{Err}_{\hat{f}}(x_0) &= E[(Y - \hat{f}(X))^2 \mid X = x_0] \\ &\stackrel{\text{Exercise}}{=} \sigma_\epsilon^2 + [E\hat{f}(x_0) - f(x_0)]^2 + E[\hat{f}(x_0) - E\hat{f}(x_0)]^2 \\ &= \sigma_\epsilon^2 + \text{Bias}^2(\hat{f}(x_0)) + \text{Var}(\hat{f}(x_0)) \\ &= \text{Irreducible Error} + \text{Bias}^2 + \text{Variance}.\end{aligned}$$

- ▶ The **Irreducible Error** is the error due to the noise variable ϵ and cannot be improved.
- ▶ The **Bias** is the difference between the average prediction $E\hat{f}(x_0)$ and the truth at x_0 .
- ▶ The **Variance** is the variability of the prediction $\hat{f}(x_0)$ when $\hat{f}$ is fitted for different data sets.



The Bias–Variance decomposition

Example: The **k-Nearest-Neighbors** with tuning parameter $k \in \{1, 2, \dots\}$ gives

$$\hat{f}_k(x_0) = \frac{1}{k} \sum_{x_i \in N_k(x_0)} y_i.$$

We obtain from the above:

$$\begin{aligned} \text{Err}_{\hat{f}}(x_0) &= \mathbb{E}[(Y - \hat{f}_k(x_0))^2 \mid X_0 = x_0] \\ &= \sigma_\epsilon^2 + \left[f(x_0) - \frac{1}{k} \sum_{x_i \in N_k(x_0)} f(x_i) \right]^2 + \frac{\sigma_\epsilon^2}{k}. \end{aligned}$$

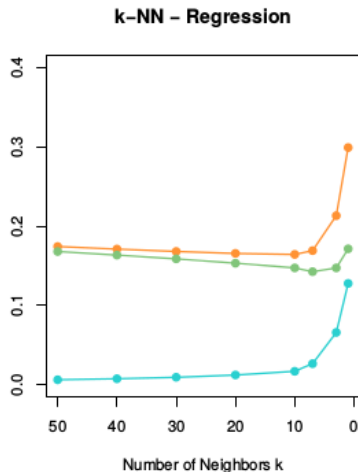


Figure: Expected prediction error (orange), squared bias (green) and variance (blue).

Bias–Variance trade off

Simple parametric methods such as **linear models**

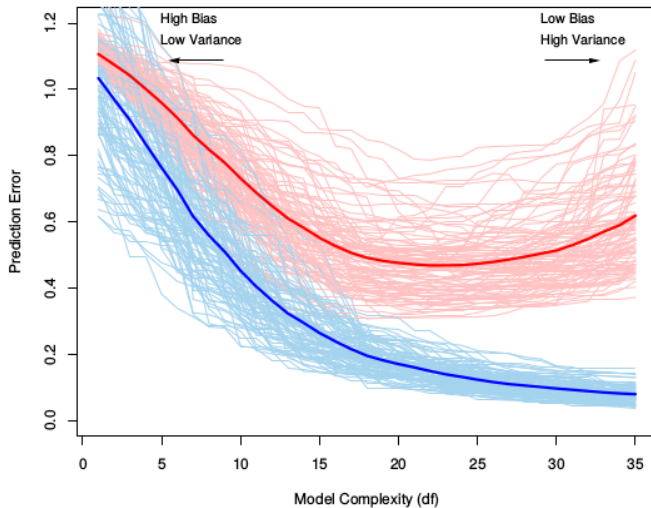
- ▶ have low flexibility (**high bias**) but need less data to fit;
- ▶ are stable for different data sets (**low variance**);
- ▶ fit well if the data satisfies the model assumptions;
- ▶ are good for inference and **interpretation**.

Complex (possibly non-parametric) methods such as **k-Nearest-Neighbors**

- ▶ can be very flexible (**low bias**) but need a lot of data;
- ▶ are prone to **overfitting** and vary strongly depending on the data set (**high variance**);
- ▶ are problematic in high dimensions (large p), so-called **curse of dimensionality**;
- ▶ are potentially better for **prediction**.



Training versus test error



Chapter 7: Cross-validation

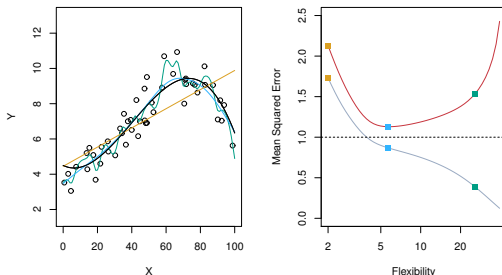
1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
- 7. Cross-validation**
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Model selection

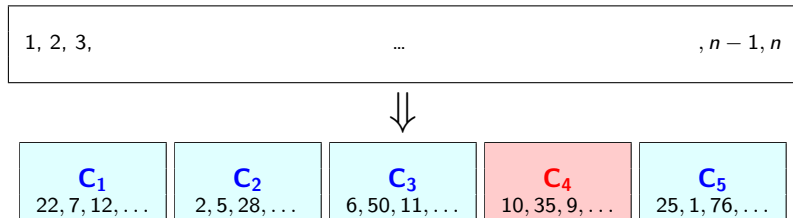
- ▶ The training error MSE_{Tr} is not suited for **model selection** since a method that overfits the data may have a small MSE_{Tr} but perform badly on new test data.
- ▶ A good model should be able to predict well on new data, thus we choose the model $\hat{f}$ that minimizes the **expected prediction error** $\text{Err}_{\hat{f}}$.
- ▶ We only have one data set, how can we obtain an estimate of $\text{Err}_{\hat{f}}$?

Two classes of approaches

- ▶ Theoretical approximations of $\text{Err}_{\hat{f}}$: **Akaike Information Criterion** (AIC), **Bayesian Information Criterion** (BIC), etc. (only works for certain model classes)
- ▶ Estimation of $\text{Err}_{\hat{f}}$ based on the **test error** MSE_{Te} : **Bootstrap**, **Cross-Validation**, etc.



K-Fold Cross-Validation



- ▶ We only have **one data set** $\{(x_1, y_1), \dots, (x_n, y_n)\}$, how do we obtain a new test sample?
- ▶ For **K-Fold Cross-Validation** we randomly divide the data into K groups $C_1, \dots, C_K$, also called folds, of approximately equal size n/K .
- ▶ We treat the k th group C_k as **validation/test** set and fit a model $\hat{f}_{-k}$ on the remaining $K - 1$ groups. We then compute the $\text{MSE}_k = (K/n) \sum_{i \in C_k} \{y_i - \hat{f}_{-k}(x_i)\}^2$ on the held-out group. This is done successively for all groups C_k , $k = 1, \dots, K$.
- ▶ The **K-fold CV estimate** is the average of these values

$$\text{CV}_{(K)} = \frac{1}{K} \sum_{k=1}^K \text{MSE}_k.$$

K-Fold Cross-Validation

C_1 22, 7, 12, ...	C_2 2, 5, 28, ...	C_3 6, 50, 11, ...	C_4 10, 35, 9, ...	$C_5 = C_K$ 25, 1, 76, ...
-------------------------	------------------------	-------------------------	-------------------------	-------------------------------

- ▶ The **K-fold CV estimate** is widely used and generic approach that can be applied in most cases.
- ▶ This estimate can be used to **select the best model**, and to give an idea of the test error $\text{Err}_{\hat{f}}$ of the final model.
- ▶ A good choice for K depends on the size n of the data set, but typically $K = 5$ or $K = 10$ is used.

Important:

- ▶ We automatically get an idea of the **uncertainty** of the K -fold CV estimate $CV_{(K)}$, by

$$\widehat{\text{SE}}(CV_{(K)}) = \frac{1}{\sqrt{K}} \sqrt{\sum_{k=1}^K (\text{MSE}_k - CV_{(K)})^2 / (K - 1)}.$$

Leave-One-Out Cross-Validation

- ▶ One drawback of K -fold CV is that it requires to fit K **different models**, which can be computationally costly.
- ▶ A nice special case of K -fold CV is **Leave-One-Out CV (LOOCV)**, namely each of the observations is treated as a separate fold, that is, $K = n$ and $C_k = \{k\}$, $k = 1, \dots, n$.
- ▶ The **Leave-One-Out CV (LOOCV) estimate** is then

$$CV_{(n)} = \frac{1}{n} \sum_{k=1}^n \{y_k - \hat{f}_{-\{k\}}(x_k)\}^2.$$

- ▶ In general, this requires even more models to be fit, but in **linear models** there is a convenient approximation

$$CV_{(n)} \approx \frac{1}{n} \sum_{k=1}^n \left[\frac{y_k - \hat{f}(x_k)}{1 - S_{kk}} \right]^2,$$

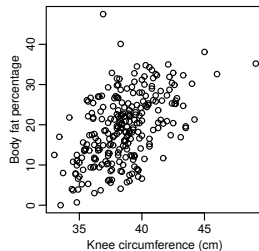
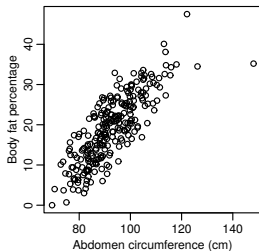
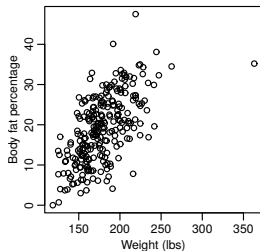
where S_{kk} are constants automatically available from the linear model fit.

- ▶ To summarize: $CV_{(K)}$ is a good estimate of the **expected prediction error** $\text{Err}_{\hat{f}}$, and the choice of K is a **bias-variance trade off** (large K : small bias, large variance; small K : large bias, small variance).

Medical application: body fat

Data set **bodyfat** (library **mfp**). The data set contains body fat estimates (**siri**) for 252 men with measurements of different body attributes, in total $p = 13$ predictors. The first 10 measurements and a plot of the responses versus some of the predictors:

siri	age	weight	height	neck	chest	abdo	hip	thigh	knee	ankle	biceps	forearm	wrist
12.3	23	154.25	67.75	36.2	93.1	85.2	94.5	59.0	37.3	21.9	32.0	27.4	17.1
6.1	22	173.25	72.25	38.5	93.6	83.0	98.7	58.7	37.3	23.4	30.5	28.9	18.2
25.3	22	154.00	66.25	34.0	95.8	87.9	99.2	59.6	38.9	24.0	28.8	25.2	16.6
10.4	26	184.75	72.25	37.4	101.8	86.4	101.2	60.1	37.3	22.8	32.4	29.4	18.2
28.7	24	184.25	71.25	34.4	97.3	100.0	101.9	63.2	42.2	24.0	32.2	27.7	17.7
20.9	24	210.25	74.75	39.0	104.5	94.4	107.8	66.0	42.0	25.6	35.7	30.6	18.8
19.2	26	181.00	69.75	36.4	105.1	90.7	100.3	58.4	38.3	22.9	31.9	27.8	17.7
12.4	25	176.00	72.50	37.8	99.6	88.5	97.1	60.0	39.4	23.2	30.5	29.0	18.8
4.1	25	191.00	74.00	38.1	100.9	82.5	99.9	62.9	38.3	23.8	35.9	31.1	18.2
11.7	23	198.25	73.50	42.1	99.6	88.6	104.1	63.1	41.7	25.0	35.6	30.0	19.2



Medical application: body fat

Data set `bodyfat` (library `mfp`). The data set contains body fat estimates (`siri`) for 252 men with measurements of different body attributes, in total $p = 13$ predictors.

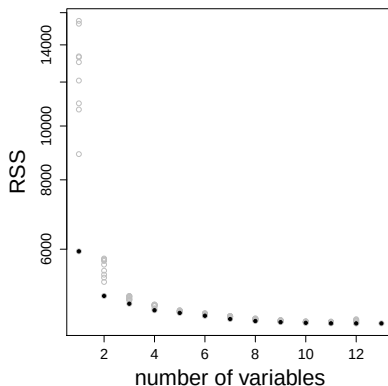


Figure: Best model fit (multiple linear regression) for a fixed number of predictors in terms of RSS; search is over all subsets with this number of predictors.

Medical application: body fat

Data set `bodyfat` (library `mfp`). The data set contains body fat estimates (`siri`) for 252 men with measurements of different body attributes, in total $p = 13$ predictors.

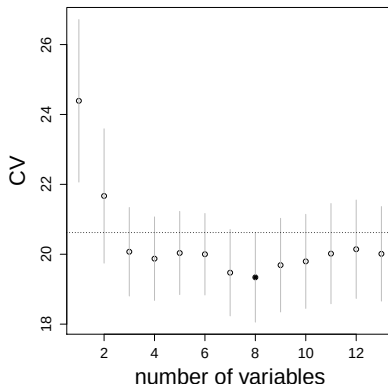


Figure: Results of 10 fold cross-validation for best-subset selection on the body fat dataset. The folds were randomly selected. The grey vertical lines are one-standard-error bars.

Medical application: body fat

Data set **bodyfat** (library **mfp**). The data set contains body fat estimates (**siri**) for 252 men with measurements of different body attributes, in total $p = 13$ predictors.

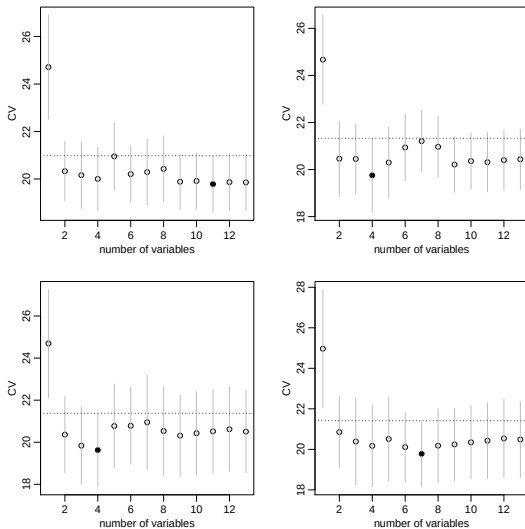


Figure: Results for 4 repetitions of 10-fold CV using different folds (randomly selected).

Medical application: body fat

- ▶ The previous slides show that the results of CV depend on the randomness of the folds.
- ▶ Model selection based on the so-called one-standard-error rule should be more stable: Choose the most parsimonious model whose error is no more than one standard error above the error of the best model.
- ▶ To investigate this, we can repeat 10-fold CV 100 times and identify the number of variables k minimizing CV and the optimal values of k chosen using the one-standard-error rule. The table below gives the number of times $\hat{k} = k$ was selected by each method.

k	1	2	3	4	5	6	7	8	9	10	11	12	13
arg min of CV	0	5	8	11	0	1	11	20	11	12	18	3	0
one-sd-err rule	0	83	15	2	0	0	0	0	0	0	0	0	0

⇒ Use one-standard-error rule!

Medical application: body fat

To conclude on the body fat example:

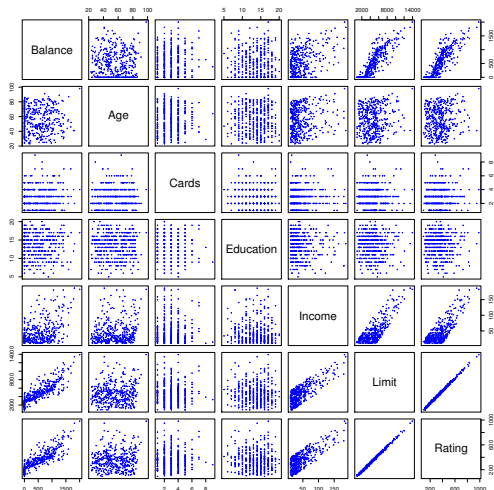
- ▶ Using cross-validation, we may choose $\hat{k} = 3$.
- ▶ We then estimate our final model $\hat{f}_{\hat{k}} = \hat{f}_3$ on all 252 observations: this means we identify the best-subset model with 3 predictors and take this as our final model.
- ▶ Our final model uses the features **weight**, **abdomen** and **wrist**.
- ▶ We easily get the parameter estimates for this model: simply fit the multiple linear model with the features **weight**, **abdomen** and **wrist**.

Chapter 8: Linear model selection and regularization

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
- 8. Linear model selection and regularization**
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Example: Credit data set

The **Credit data set** from package **ISLR** contains the response **balance** (average credit card debt) and quantitative predictors such as **age**, number of credit **cards**, etc. There are also four qualitative predictors **gender**, **student**, **status** and **ethnicity**; see **[ISL]**, Section 3.3 for treatment of qualitative predictors in linear models.



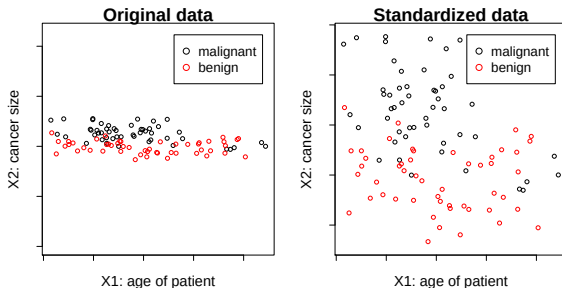
Data standardization

- ▶ The predictors $X_1, \dots, X_p$ might have very **different scales**.
- ▶ This is not problematic in methods where the parameters can account for these different scales; e.g., linear regression with least squares, LDA, logistic regression.
- ▶ In methods that treat all **predictors equally**, this might cause a problem; e.g., k -Nearest-Neighbor since it uses the distances between predictors x_i and x_j .
- ▶ In general, it is good to **standardize the predictors** in the training data x_i , $i = 1, \dots, n$, that is, for each margin $j = 1, \dots, p$:

$$x_{ij}^{\text{scaled}} = \frac{x_{ij} - \bar{x}_j}{\sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}} = \frac{x_{ij} - \bar{x}_j}{\hat{\sigma}_j},$$

which can be done with the transformer `StandardScaler()` in python.

- ▶ For a new input $x_0 \in \mathbb{R}^p$, **standardize margins** first to $x_{0j}^{\text{scaled}} = (x_{0j} - \bar{x}_j)/\hat{\sigma}_j$ and then use the predictive model.



Back to linear models

- In the **regression setting** a multiple linear model

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p,$$

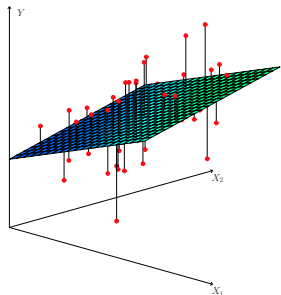
is fitted to the data (x_i, y_i) , $i = 1, \dots, n$.

- The estimated vector $\hat{\beta} = (\hat{\beta}_0, \dots, \hat{\beta}_p)$ minimizes the residual sum of squares (RSS)

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2.$$

- Why might we want to use a **different estimation method** than least squares?

- (1) **Prediction accuracy**: If n is not much larger than the number of predictors p , or even $p \gg n$ (**high-dimensional setting**), then the least squares estimate $\hat{\beta}$ may have high variance, resulting in overfitting. By **shrinking/regularizing** the estimate we can reduce the variance and improve prediction accuracy on new test data.
- (2) **Model interpretability**: Some or many of the predictors might not be associated with the response, but least squares will almost never estimate their coefficients to zero. Excluding these **irrelevant predictors** reduces the complexity of the model and allows interpretation.



Alternatives to least squares

- (1) **Subset selection**: These approaches identify a subset of the p predictors and then fit a linear model by least squares:
 - ▶ best subset selection;
 - ▶ stepwise selection (forward/backward)
 - ▶ Details in [ISL], Chapter 6.
- (2) **Dimension reduction**: Projecting the p predictors into a M -dimensional subspace (PCA, etc.), $M < p$, and fit a linear regression to the projected variables by least squares.
 - ▶ Details in [ISL], Chapter 6.
- (3) **Shrinkage**: A linear regression is fitted to all p predictors, but the estimated coefficients are **shrunk/regularized** towards zero relative to the least squares fit. Depending on the type of regularization (Lasso, Ridge, etc.), some coefficients may be estimated to be exactly zero, and thus shrinkage can perform **automatic variable selection**.

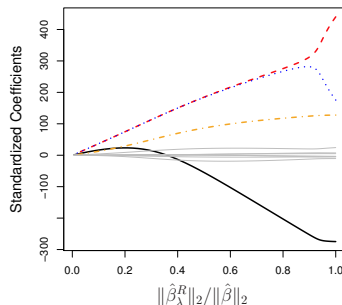
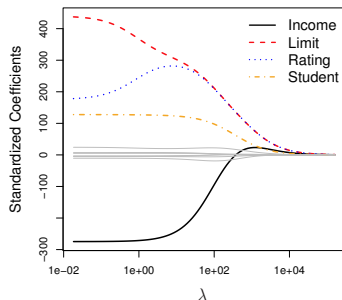
Ridge regression

- ▶ Instead of the RSS the **ridge regression coefficient estimates** $\hat{\beta}_\lambda^R$ minimize the penalized version

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 = \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2,$$

where $\lambda \geq 0$ is called the **tuning parameter**.

- ▶ The penalty term $\lambda \sum_j \beta_j^2$ is called a **shrinkage penalty** and it is small when $\beta_1, \dots, \beta_p$ are close to 0.
- ▶ For each $\lambda \geq 0$, ridge regression produces a different set of coefficient estimates $\hat{\beta}_\lambda^R$: if $\lambda = 0$ they approach the least squares estimates, and if $\lambda \rightarrow \infty$ they shrink to 0.
- ▶ A good value of λ is critical and is chosen by **cross-validation**.



Ridge regression: closed-form solution

- ▶ Ridge regression has a **closed-form solution**:

$$\hat{\beta}_{\lambda}^R = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T y,$$

where $\mathbf{X} \in \mathbb{R}^{n \times p}$ is the data matrix, $\mathbf{I} \in \mathbb{R}^{p \times p}$ is the identity matrix and $y = (y_1, \dots, y_n)$ is the vector of responses.

- ▶ We can compare this to the **least squares solution**

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T y.$$

- ▶ This shows how ridge regression **shrinks** the least squares estimates to zero.

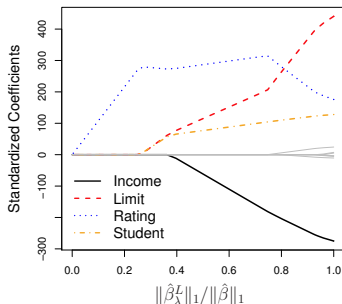
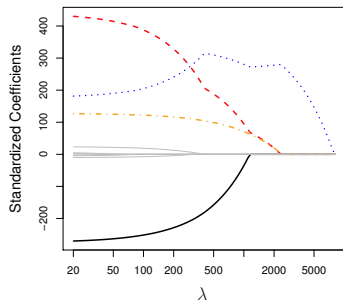
Lasso: least absolute shrinkage and selection operator

- ▶ Ridge regression has the disadvantage that it includes **all p predictors** in the final model.
- ▶ The **lasso** overcomes this problem. The lasso coefficients $\hat{\beta}_\lambda^L$ minimize the penalized RSS

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| = \text{RSS} + \lambda \sum_{j=1}^p |\beta_j|,$$

where $\lambda \geq 0$ is again a **tuning parameter**.

- ▶ Only difference to ridge regression: the **lasso penalty** uses the ℓ_1 -norm of β .
- ▶ This penalty forces some coefficient estimates to be **exactly 0**, if λ is large enough; lasso thus performs variable selection!
- ▶ Again, a good value of λ is critical and is chosen by **cross-validation**.



Regularization: geometric interpretation

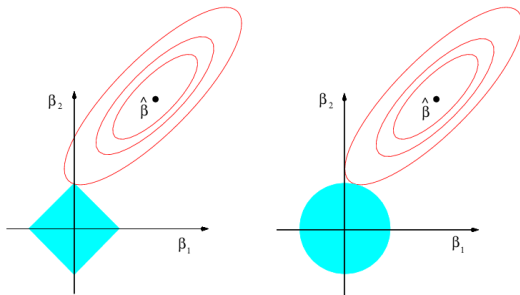
- **Equivalent formulation:** one can show that ridge regression and lasso coefficient estimates $\hat{\beta}_{\lambda}^R$ and $\hat{\beta}_{\lambda}^L$ minimize for some s , respectively,

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \quad \text{subject to} \quad \sum_{j=1}^p \beta_j^2 \leq s,$$

and

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \quad \text{subject to} \quad \sum_{j=1}^p |\beta_j| \leq s.$$

- This allows a geometric interpretation of the effect of **penalization**.



- ▶ For large p , best subset selection is computationally infeasible. In this case, lasso can be used to perform **variable selection**.
- ▶ While the ridge coefficients $\hat{\beta}_{\lambda}^R$ have a closed-form solution, the lasso coefficients $\hat{\beta}_{\lambda}^L$ do not have a closed-form solution. They are obtained by **numerical optimization**, which is not too hard since the problems are **convex**.
- ▶ Usually, ridge and lasso will have similar **predictive performances** (for good λ) but lasso results are more interpretable.
- ▶ **Elastic-net** (Zou & Hastie, 2005) combines ridge and lasso penalties into

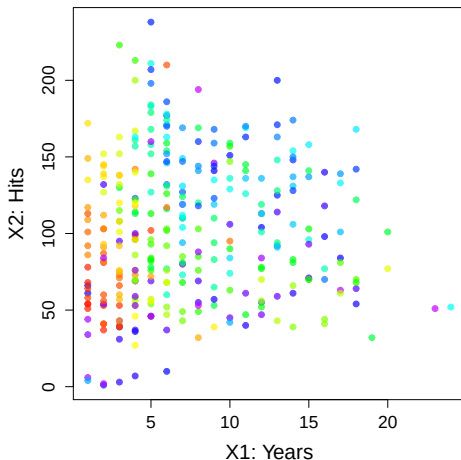
$$\lambda \sum_{j=1}^p \left((1 - \alpha) \beta_j^2 + \alpha |\beta_j| \right),$$

where $\alpha \in [0, 1]$ (note: $\alpha = 0$ results in ridge regression and $\alpha = 1$ in lasso).

- ▶ In practice, lasso allows to **pre-select** a reasonable number of variables that are then used in more flexible prediction models.
- ▶ Regularization can be used for other models such as **logistic regression**, etc.

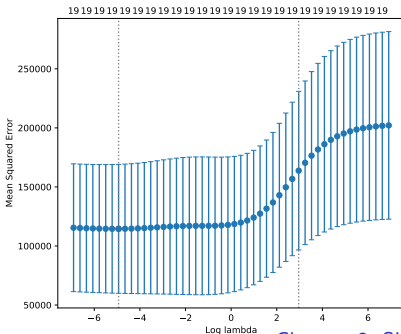
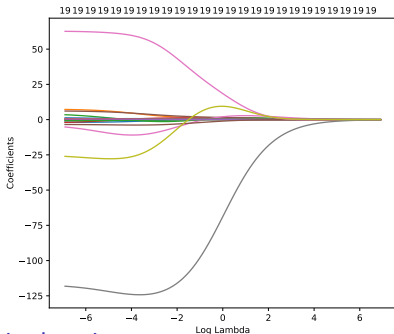
Example: Hitters data set

The **Hitters** data set from the **ISLR** package contains the annual **Salary** of 263 baseball players and $p = 19$ predictors such as numbers **Years** played in major leagues and the number of **Hits**. **Salary** color-code: low (yellow-red) to high (blue, green).



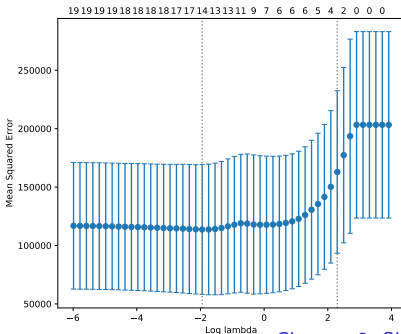
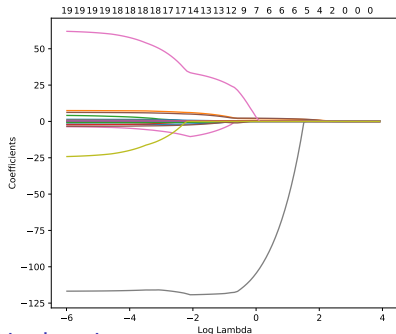
Ridge regression in python

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.pipeline import Pipeline
X, y = pd.get_dummies(data, drop_first=True).drop("Salary", axis=1), data["Salary"]
grid = 10 ** np.linspace(-3, 6, 100)
coefs = []
for a in grid:
    ridge_pipeline = Pipeline([("scaler", StandardScaler()), ("ridge", Ridge(alpha=a))]).fit(X, y)
    coefs.append(ridge_pipeline["ridge"].coef_)
ridgeCV = GridSearchCV(estimator=Pipeline([("scaler", StandardScaler()), ("ridge", Ridge())]), param_grid={
resCV = ridgeCV.cv_results_
best_ind = np.argmin(-resCV["mean_test_score"])
one_std_best = np.max(grid[-resCV["mean_test_score"] <=
                        -resCV["mean_test_score"][best_ind] + resCV["std_test_score"][best_ind]])
```



Lasso in python

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.pipeline import Pipeline
X, y = pd.get_dummies(data, drop_first=True).drop("Salary", axis=1), data["Salary"]
grid = 10 ** np.linspace(-2.6, 3, 100)
coefs = []
for a in grid:
    lasso_pipeline = Pipeline([("scaler", StandardScaler()), ("lasso", Lasso(alpha=a, max_iter=1e5))]).fit(X, y)
    coefs.append(lasso_pipeline["lasso"].coef_)
lassoCV = GridSearchCV(estimator=Pipeline([("scaler", StandardScaler()), ("lasso", Lasso(max_iter=1e5))]), param_grid={"alpha": grid}, cv=5)
resCV = lassoCV.cv_results_
best_ind = np.argmin(-resCV["mean_test_score"])
one_std_best = np.max(grid[-resCV["mean_test_score"] <= -resCV["mean_test_score"][best_ind] + resCV["std_test_score"][best_ind]])
```



Chapter 9: Classification framework

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
- 9. Classification framework**
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Classification: introduction

- ▶ Classification, also called **pattern recognition**, aims to predict **qualitative** response variables Y that take values in an unordered set $\mathcal{G} = \{\mathcal{G}_1, \dots, \mathcal{G}_q\}$, e.g.

infected $\in \{\text{Yes}, \text{No}\}$

eye color $\in \{\text{brown}, \text{blue}, \text{green}\}$

- ▶ Given the vector of predictors $X = (X_1, \dots, X_p)$, the classification task is to build a function G , called **classifier**, that takes the vector X and predicts the value of Y , i.e., $G(X) \in \mathcal{G}$.
- ▶ Often one is more interested in the **probability** that Y belongs to each class $\mathcal{G}_1, \dots, \mathcal{G}_q$, e.g.,
 - ▶ the probability that a client of a bank will default;
 - ▶ the probability that a cancer is malignant;
 - ▶ etc.
- ▶ We typically identify the q classes with the numbers 1 to q , such that $\mathcal{G} = \{1, \dots, q\}$, or if we only have two classes (**binary data**), then $\mathcal{G} = \{0, 1\}$.

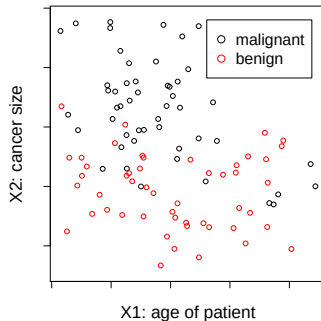
Classification: introduction

Example.: [Classification of cancer; $p = 2$, $q = 2$]

- We want to find a classifier

$$G(\text{age}, \text{size}) \in \mathcal{G} = \{\text{benign}, \text{malignant}\},$$

that predicts whether a cancer is **benign** or **malignant**, based on the predictors **age** and **size**.



The 0–1 loss function

- ▶ How do we measure the quality of prediction?
- ▶ For classification, squared error loss might not be optimal. For a prediction $\hat{y}$ we usually use the **0 – 1 loss function**

$$L(y, \hat{y}) = \mathbf{1}\{y \neq \hat{y}\} = \begin{cases} 0 & \text{if } y = \hat{y} \\ 1 & \text{if } y \neq \hat{y} \end{cases}$$

to measure the **misclassification** error.

- ▶ For a classifier $\hat{G}$, the **expected misclassification rate** is

$$\text{Err}_{\hat{G}} = \mathbb{E}[\mathbf{1}\{Y \neq \hat{G}(X)\}] = \Pr\{Y \neq \hat{G}(X)\},$$

where the expectation is taken over (X, Y) .

- ▶ Note: Also other loss functions for classification tasks are common, since
 - ▶ in medicine, for example, false negative and false positive are not equally severe, and we want to penalize one type of error more than the other;
 - ▶ the 0-1 loss is not smooth and may lead to optimization problems.

The Bayes classifier

- ▶ What is a good **classifier**? Ideally, we would like to find the optimal G^* that **minimizes** $\text{Err}_{\widehat{G}} = \Pr\{Y \neq \widehat{G}(X)\}$.
- ▶ For simplicity, consider two-class classification, i.e. $\mathcal{G} = \{0, 1\}$.
- ▶ It can be shown, that the optimal G^* is the so-called **Bayes classifier**, which assigns to a predictor x the **most likely class**, i.e.,

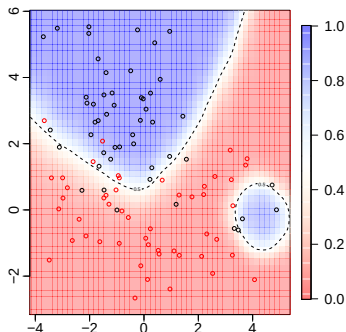
$$G^*(x) = \operatorname{argmax}_{j=0,1} \Pr(Y = j \mid X = x) = \begin{cases} 0 & \text{if } \Pr(Y = 0 \mid X = x) > 1/2 \\ 1 & \text{otherwise.} \end{cases}$$

- ▶ The **Bayes decision boundary** is the curve of all $x \in \mathbb{R}^p$ with

$$\Pr(Y = 0 \mid X = x) = 1/2.$$

- ▶ The Bayes classifier gives the smallest possible 0 – 1 error. It is the irreducible error, also called the **Bayes error rate**:

$$\text{Err}_{G^*} = \Pr\{Y \neq G^*(X)\}.$$



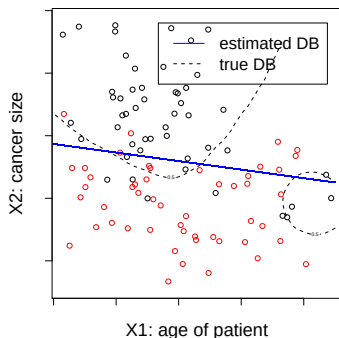
The Bayes decision rule

- ▶ The Bayes classifier motivates to estimate the **posterior class probabilities** $\widehat{\Pr}(Y = j | X = x)$, $j = 0, 1$, and then use the **plug-in decision rule**

$$\widehat{G}(x) = \begin{cases} 0 & \text{if } \widehat{\Pr}(Y = 0 | X = x) > 1/2 \\ 1 & \text{otherwise.} \end{cases}$$

- ▶ The **decision boundary** is the line where

$$\widehat{\Pr}(Y = 0 | X = x) = \widehat{\Pr}(Y = 1 | X = x) = 1/2.$$



Statistical modeling of $\hat{G}$

- (a) **Generative models:** estimate the joint distribution of (Y, X) and deduce the posterior class probabilities for $j = 0, 1$

$$\widehat{\Pr}(Y = j \mid X = x) = \frac{\widehat{\Pr}(Y = j, X = x)}{\widehat{\Pr}(X = x)}.$$

- ▶ This is often more complex and needs more data.
- ▶ Examples: **Linear/Quadratic Discriminant analysis**, **Naive Bayes**, etc.

- (b) **Discriminative models:** estimate only the conditional distribution $(Y \mid X)$, that is, directly the posterior class probabilities $\widehat{\Pr}(Y = j \mid X = x)$.

- ▶ Usually preferred, as it provides posterior class probabilities and is less complex than generative models.
- ▶ Examples: **kNN**, **Logistic regression**, **Random Forest**, **Neural Networks**, etc.

- (c) **Geometric models:** directly estimate a function that predicts the class 0, 1 of y from x .

- ▶ Simple, since no distribution is needed, but does not provide estimates for the probabilities of the class predictions.
- ▶ Examples: **Support vector machines**

The empirical misclassification rate

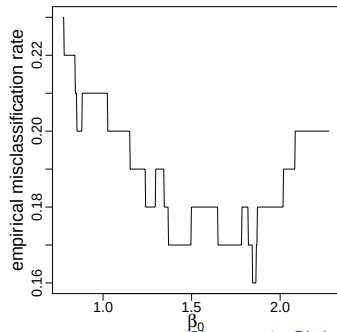
Given a **training data** set $(x_1, y_1), \dots, (x_n, y_n)$, with $x_i \in \mathbb{R}^p$ and $y_i \in \{1, \dots, q\}$, how can we estimate the model $\hat{G}$?

- ▶ In regression, we minimize the empirical mean squared error on the training set. We could do the same for the **empirical misclassification rate** by minimizing

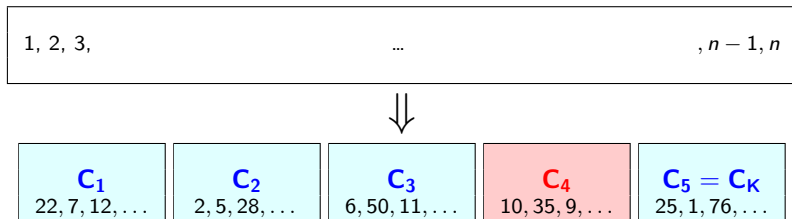
$$\text{err}_{\hat{G}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i \neq \hat{G}(x_i)\}$$

- ▶ Problem: The 0-1 loss function $\mathbf{1}\{y \neq \hat{y}\}$ is discontinuous, which makes **numerical optimization** very hard.

- ▶ Solution: Different methods such as **maximum likelihood** are used for **parameter estimation**, which are close to 0 – 1 loss but differentiable.
- ▶ The 0 – 1 loss can still be used for **model selection**, or to compare models fitted with different methods.



K-Fold Cross-Validation



- ▶ For a data set $\{(x_1, y_1), \dots, (x_n, y_n)\}$ we perform **K-Fold Cross-Validation** as in the regression framework.
- ▶ For each fold C_k , $k = 1, \dots, K$ we treat C_k as **validation/test** set and fit a classification model $\hat{G}_{-k}$ on the remaining $K - 1$ folds. We then compute the empirical misclassification rate

$$\text{err}_k = \frac{1}{|C_k|} \sum_{i \in C_k} L(y_i, \hat{G}_{-k}(x_i)) = \frac{1}{n/K} \sum_{i \in C_k} \mathbf{1}\{y_i \neq \hat{G}_{-k}(x_i)\}.$$

- ▶ The **K-fold CV estimate** is the average of these values

$$\text{CV}_{(K)} = \frac{1}{K} \sum_{k=1}^K \text{err}_k.$$

Chapter 10: Linear and quadratic discriminant analysis

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
- 10. Linear and quadratic discriminant analysis**
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Linear decision boundaries: hyperplanes

- ▶ Suppose we have only two classes, denoted by $\mathcal{G} = \{0, 1\}$.
- ▶ Linear classification methods produce **linear decision boundaries**, called **hyperplanes**.
- ▶ In a p -dimensional space, a hyperplane is a flat **affine subspace of dimension $p - 1$** , and it can be written, for parameters $b_0, b_1, \dots, b_p \in \mathbb{R}$, as the set

$$x \in \mathbb{R}^p : \quad b_0 + b_1x_1 + \dots + b_px_p = 0.$$

- ▶ Suppose a point $x \in \mathbb{R}^p$ is not on the hyperplane, then

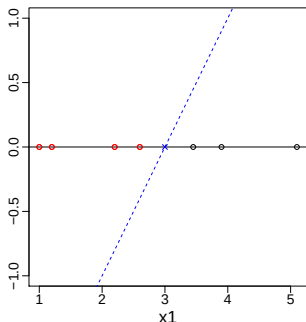
$$b_0 + b_1x_1 + \dots + b_px_p > 0,$$

tells us that it lies on the **one side** of the hyperplane, or with “<” on the other.

Example:

- ▶ For $p = 1$, the hyperplane is just a point

$$x \in \mathbb{R} : \quad b_0 + b_1x = 0 \quad \Leftrightarrow \quad x = -b_0/b_1.$$



Linear decision boundaries: hyperplanes

- ▶ Suppose we have only two classes, denoted by $\mathcal{G} = \{0, 1\}$.
- ▶ Linear classification methods produce **linear decision boundaries**, called **hyperplanes**.
- ▶ In a p -dimensional space, a hyperplane is a flat **affine subspace of dimension $p - 1$** , and it can be written, for parameters $b_0, b_1, \dots, b_p \in \mathbb{R}$, as the set

$$x \in \mathbb{R}^p : \quad b_0 + b_1x_1 + \dots + b_px_p = 0.$$

- ▶ Suppose a point $x \in \mathbb{R}^p$ is not on the hyperplane, then

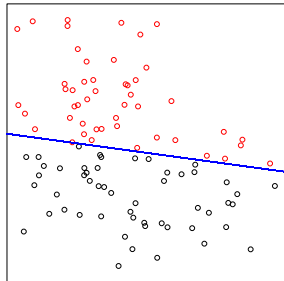
$$b_0 + b_1x_1 + \dots + b_px_p > 0,$$

tells us that it lies on the **one side** of the hyperplane, or with “<” on the other.

Example:

- ▶ For $p = 2$, the hyperplane is a line

$$\begin{aligned}(x_1, x_2) \in \mathbb{R}^2 : \quad & b_0 + b_1x_1 + b_2x_2 = 0 \\ \Leftrightarrow \quad & x_2 = -b_0/b_2 - (b_1/b_2)x_1.\end{aligned}$$



Linear decision boundaries: hyperplanes

- ▶ Suppose we have only two classes, denoted by $\mathcal{G} = \{0, 1\}$.
- ▶ Linear classification methods produce **linear decision boundaries**, called **hyperplanes**.
- ▶ In a p -dimensional space, a hyperplane is a flat **affine subspace of dimension $p - 1$** , and it can be written, for parameters $b_0, b_1, \dots, b_p \in \mathbb{R}$, as the set

$$x \in \mathbb{R}^p : \quad b_0 + b_1 x_1 + \dots + b_p x_p = 0.$$

- ▶ Suppose a point $x \in \mathbb{R}^p$ is not on the hyperplane, then

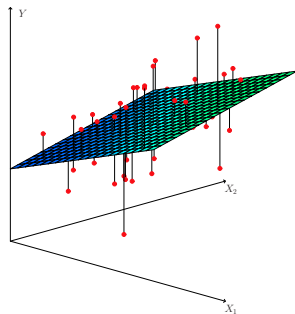
$$b_0 + b_1 x_1 + \dots + b_p x_p > 0,$$

tells us that it lies on the **one side** of the hyperplane, or with “<” on the other.

Example:

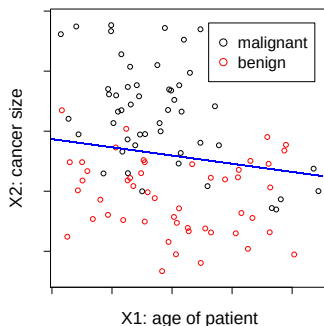
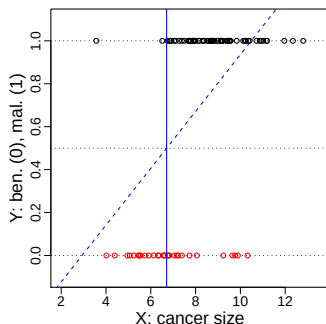
- ▶ For $p = 3$, the hyperplane is

$$(x_1, x_2, x_3) \in \mathbb{R}^3 : \quad b_0 + b_1 x_1 + b_2 x_2 + b_3 x_3 = 0.$$



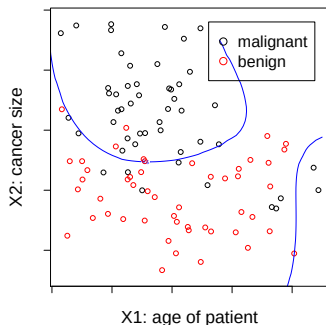
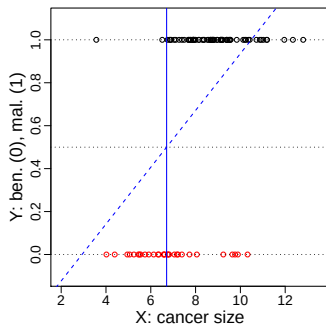
Linear methods for classification

- ▶ Example: direct linear regression on $\mathcal{G} = \{0, 1\}$; usually not a good idea!
- ▶ Many other methods will produce linear decision boundaries.
- ▶ Decision boundaries of linear methods are linear only in the **feature space**.
- ▶ **Augmenting the feature space** by basis functions might lead to non-linear decision boundaries in the original space of $(X_1, \dots, X_p)$.
- ▶ Right: the model is linear in $X_1, X_2, X_1^2, X_2^2, X_1^3, X_2^3, X_1X_2, X_1X_2^2, X_1^2X_2, X_1^2X_2^2$.
- ▶ For more than two classes, linear classification methods produce **piecewise linear decision boundaries**.



Linear methods for classification

- ▶ Example: direct linear regression on $\mathcal{G} = \{0, 1\}$; usually not a good idea!
- ▶ Many other methods will produce linear decision boundaries.
- ▶ Decision boundaries of linear methods are linear only in the **feature space**.
- ▶ **Augmenting the feature space** by basis functions might lead to non-linear decision boundaries in the original space of $(X_1, \dots, X_p)$.
- ▶ Right: the model is linear in $X_1, X_2, X_1^2, X_2^2, X_1^3, X_2^3, X_1X_2, X_1X_2^2, X_1^2X_2, X_1^2X_2^2$.
- ▶ For more than two classes, linear classification methods produce **piecewise linear decision boundaries**.



Linear discriminant analysis

- **Setup:** Classify $Y \in \mathcal{G} = \{1, \dots, q\}$ given predictors $X = (X_1, \dots, X_p)$
- **Idea:** model the **class-conditional densities** g_j of $X \mid Y = j$, and the marginals $\pi_j = \Pr(Y = j)$, $j = 1, \dots, q$. Deduce $\Pr(Y \mid X)$ from **Bayes' formula**:

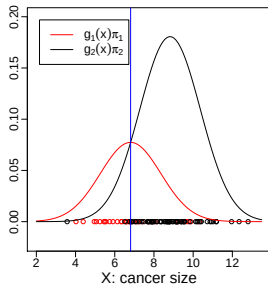
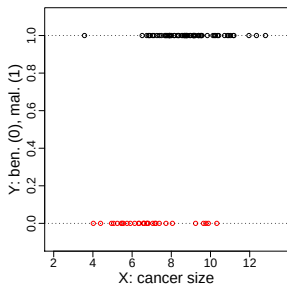
$$\Pr(Y = j \mid X = x) = \frac{\Pr(X = x \mid Y = j)\Pr(Y = j)}{\Pr(X = x)} \propto g_j(x)\pi_j.$$

- Linear discriminant analysis (LDA):

- (1) model $g_j(x)$ by a **multivariate normal distribution**,

$$g_j(x) = \frac{1}{(2\pi)^{p/2}(\det \Sigma_j)^{1/2}} \exp \left\{ -\frac{1}{2}(x - \mu_j)^T \Sigma_j^{-1}(x - \mu_j) \right\};$$

- (2) assume the g_j have the **same covariance matrix** $\Sigma_j = \Sigma$ for all j .



► **Model parameters:**

- the class/prior probabilities $\pi_1, \dots, \pi_q \in (0, 1)$;
- the mean vectors $\mu_1, \dots, \mu_q \in \mathbb{R}^p$;
- the covariance matrix $\Sigma \in \mathbb{R}^{p \times p}$.

- **Estimation:** From a training sample $(y_1, x_1), \dots, (y_n, x_n)$ we estimate the parameters of LDA by the usual **maximum likelihood estimators**

$$\begin{aligned}\hat{\pi}_j &= \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i = j\} \\ \hat{\mu}_j &= \frac{\sum_{i=1}^n x_i \mathbf{1}\{y_i = j\}}{\sum_{i=1}^n \mathbf{1}\{y_i = j\}} \\ \hat{\Sigma} &= \frac{\sum_{j=1}^q \sum_{i=1}^n (x_i - \hat{\mu}_j)(x_i - \hat{\mu}_j)^T \mathbf{1}\{y_i = j\}}{n - q}\end{aligned}$$

- This is very simple and **computationally fast**!
- Note: Different to linear regression, we don't use the numerical value of the y_i 's but only the class!

LDA: prediction

- ▶ **Goal:** for a new input value $x_0 \in \mathbb{R}^p$ predict its unobserved class y_0 .
- ▶ **Method:** compute the **posterior class probabilities** $\widehat{\Pr}(Y = j \mid X = x_0)$ and use the **Bayes classifier** resulting from these estimates (plug-in estimate), i.e.,

$$\hat{y}_0 = \arg \max_{j=1, \dots, q} \widehat{\Pr}(Y = j \mid \mathbf{X} = x_0) = \arg \max_{j=1, \dots, q} \log \widehat{\Pr}(Y = j \mid X = x_0).$$

- ▶ Note that

$$\begin{aligned} \log \widehat{\Pr}(Y = j \mid X = x_0) &= \log \widehat{g}_j(x_0) + \log \widehat{\pi}_j + \text{constant} \\ &= -\frac{1}{2}(x_0 - \widehat{\mu}_j)^T \widehat{\Sigma}^{-1}(x_0 - \widehat{\mu}_j) + \log \widehat{\pi}_j + \text{constant} \\ &= x_0^T \widehat{\Sigma}^{-1} \widehat{\mu}_j - \frac{1}{2} \widehat{\mu}_j^T \widehat{\Sigma}^{-1} \widehat{\mu}_j + \log \widehat{\pi}_j + \text{constant}, \end{aligned} \tag{1}$$

where the constant includes terms that do not depend on j .

- ▶ **Geometric interpretation:** eq. (1) implies that LDA classifies x_0 in terms of the **Mahalanobis distance**, $d_{\widehat{\Sigma}^{-1}}(x, y) = x^T \widehat{\Sigma}^{-1} y$, of x_0 from the centers $\widehat{\mu}_1, \dots, \widehat{\mu}_q$ (with a correction for the prior class probabilities).

LDA: decision boundaries

- ▶ The **decision boundary** that separates two classes $j, m \in \{1, \dots, q\}$ are all $x \in \mathbb{R}^p$ with

$$\Pr(Y = j \mid X = x) = \Pr(Y = m \mid X = x),$$

which is equivalent to

$$\log \Pr(Y = j \mid X = x) - \log \Pr(Y = m \mid X = x) = 0.$$

Replacing the probabilities by their expressions in terms of the Gaussian densities and the prior class probabilities we get the decision boundary

$$\log \frac{\pi_j}{\pi_m} - \frac{1}{2}(\mu_j + \mu_m)^T \Sigma^{-1}(\mu_j - \mu_m) + x^T \Sigma^{-1}(\mu_j - \mu_m) = 0,$$

a linear equation in x .

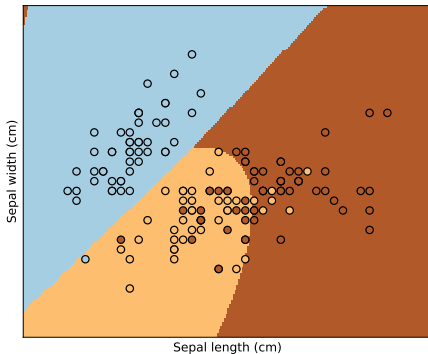
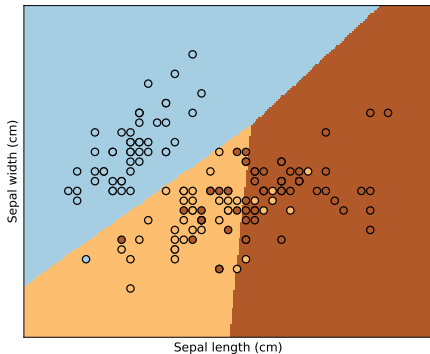
- ▶ Hence the decisions boundaries separating any two classes j and m are hyperplanes in $\mathbb{R}^p$ of dimension $p - 1$.
- ▶ Thus the decision boundary that separates the q classes is **piecewise linear**.

Iris data set

For 150 measured plants; $p = 4$ predictors as response the three Iris species

$\mathcal{G} = \{\text{setosa}, \text{versicolor}, \text{virginica}\}$ the plant belongs. We only use two predictors here.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target # we only keep the first two features: 'Sepal length', 'Sepal width'
lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
h = .02 # step size in the mesh
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
Z = lda.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
```



Extensions of LDA

- There are many possibilities to extend LDA. The Bayes' construction

$$\Pr(Y = j \mid X = x) = \frac{\Pr(X = x \mid Y = j)\Pr(Y = j)}{\Pr(X = x)} = \frac{g_j(x)\pi_j}{\sum_{j=1}^q g_j(x)\pi_j}.$$

can be used with other densities g_j .

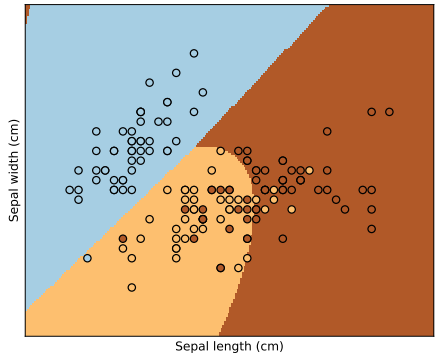
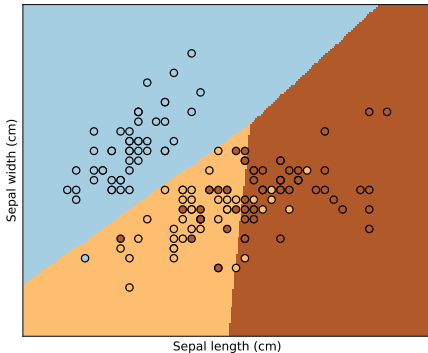
- **Quadratic discriminant analysis** (QDA): model g_j by multivariate normals with different covariance matrices Σ_j .

Iris data set: QDA

For 150 measured plants; $p = 4$ predictors as response the three Iris species

$\mathcal{G} = \{\text{setosa}, \text{versicolor}, \text{virginica}\}$ the plant belongs. We only use two predictors here.

```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target # we only keep the first two features: 'Sepal length', 'Sepal width'
qda = QuadraticDiscriminantAnalysis()
qda.fit(X, y)
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
h = .02 # step size in the mesh
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
Z = qda.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
```



Extensions of LDA

- ▶ There are many possibilities to extend LDA. The Bayes' construction

$$\Pr(Y = j \mid X = x) = \frac{\Pr(X = x \mid Y = j)\Pr(Y = j)}{\Pr(X = x)} = \frac{g_j(x)\pi_j}{\sum_{j=1}^q g_j(x)\pi_j}.$$

can be used with other densities g_j .

- ▶ **Quadratic discriminant analysis** (QDA): model g_j by multivariate normals with different covariance matrices Σ_j .
- ▶ **Mixture** models: use mixtures of Gaussians for the g_j .
- ▶ **Naive Bayes**: conditional independence assumption, $g_j(x) = \prod_{l=1}^p g_{j,l}(x_l)$.
- ▶ When some features are **categorical**, for example X_1 , we can use

$$g_j(x) = \Pr(X_1 = x_1 \mid Y = j)g_j(x_2, \dots, x_p \mid x_1),$$

where $g_j(x_2, \dots, x_p \mid x_1)$ is a density for $X_2, \dots, X_p \mid X_1 = x_1, Y = j$.

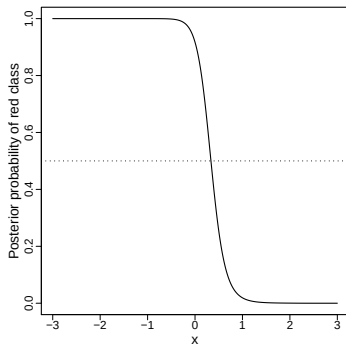
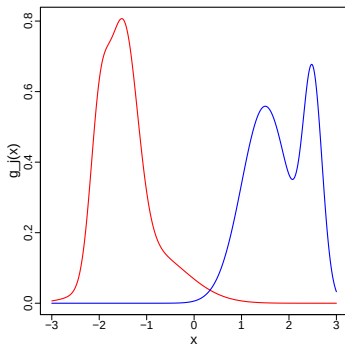
- ▶ **Non-parametric** approaches: estimate g_j non-parametrically.

Comments on LDA

- ▶ LDA is easy and **fast to compute**.
- ▶ LDA provides estimates of the **posterior class probabilities**

$$\widehat{\Pr}(Y = j \mid X = x_0) = \frac{\widehat{g}_j(x_0)\widehat{\pi}_j}{\sum_{j=1}^q \widehat{g}_j(x_0)\pi_j}.$$

- ▶ LDA usually performs well and can compete with more complex methods.
- ▶ Why? Clearly not because the normality assumption is realistic, but because a linear decision boundary is often the best the data can support and LDA is a **stable estimation method**.
- ▶ We don't need to have good models for the g_j to obtain a good model for the posterior: complex g_j may not be visible in the decision (see figure).



Chapter 11: Logistic Regression

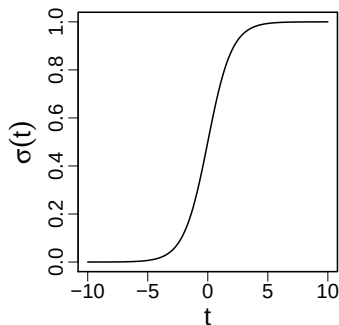
1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
- 11. Logistic Regression**
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Logistic regression

- ▶ For simplicity we first consider classification for 2 classes $Y \in \{0, 1\}$.
- ▶ We assume the feature vector X is such that $X_1 = 1$ (intercept), so that we can simply write $x^\top \beta$ instead of $\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$.
- ▶ Logistic regression links the probability of class 1 to $x^\top \beta$ by

$$\Pr(Y = 1 \mid X = x) = \sigma(x^\top \beta) \quad \text{where} \quad \sigma(t) = \frac{\exp(t)}{1 + \exp(t)}.$$

- ▶ The function σ is called the **logistic function**: it maps $\mathbb{R}$ to $(0, 1)$.



Logistic regression: interpretation

- We observe

$$\Pr(Y = 1 \mid X = x) = \sigma(x^\top \beta) \Leftrightarrow \sigma^{-1}(\Pr(Y = 1 \mid X = x)) = x^\top \beta,$$

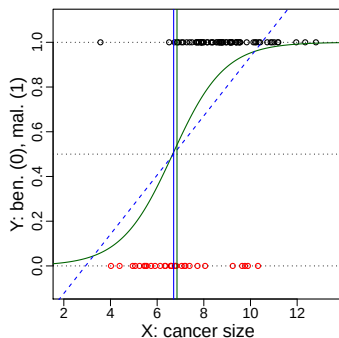
where $\sigma^{-1}(u) = \log\{u/(1-u)\}$ called the **logit function**.

- Logistic regression thus implies that **log-odds** are linear

$$\log \frac{\Pr(Y = 1 \mid X = x)}{\Pr(Y = 0 \mid X = x)} = \log \frac{\sigma(x^\top \beta)}{1 - \sigma(x^\top \beta)} = x^\top \beta,$$

which helps interpreting the coefficients β_j .

- Note that this implies that the decision boundary from logistic regression is linear.
- Figure: Logistic regression (green) versus linear regression (blue).

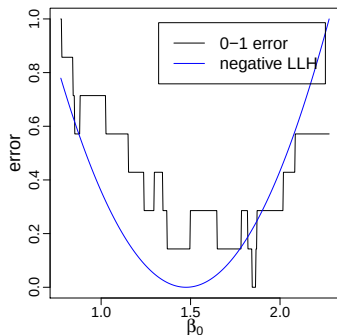


Logistic regression: estimation

- ▶ We estimate the parameter β by maximizing the **conditional log-likelihood** of the sample $(x_1, y_1), \dots, (x_n, y_n)$ where $y_i \in \{0, 1\}$.
- ▶ By the logistic model assumption $Y \mid X = x_i$ is **Bernoulli** distributed with success probability $\sigma(x_i^\top \beta)$. This gives the log-likelihood for the sample

$$\ell(\beta) = \sum_{i=1}^n \log \Pr(Y = y_i \mid X = x_i) = \sum_{i=1}^n y_i x_i^\top \beta - \log(1 + e^{x_i^\top \beta}).$$

- ▶ The **maximum likelihood estimator** (MLE) $\hat{\beta} = \operatorname{argmax}_{\beta} \ell(\beta)$ has no explicit formula; ℓ must be maximized numerically (Newton–Raphson, gradient descent, etc.).



Logistic regression: estimation

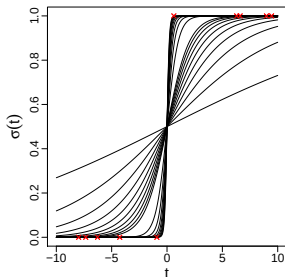
- **Prediction:** From $\hat{\beta}$ we estimate for a new input $x_0 \in \mathbb{R}^p$

$$\widehat{\Pr}(Y = 1 \mid X = x_0) = \sigma(x_0^\top \hat{\beta}).$$

We use the **Bayes classifier** based on these estimates to predict the class

$$\hat{y}_0 = \begin{cases} 1 & \text{if } \sigma(x_0^\top \hat{\beta}) > 1/2, \\ 0 & \text{otherwise.} \end{cases}$$

- Careful: The MLE $\hat{\beta}$ **does not exist** when the classes are **perfectly separable**.
- The figure illustrate the problem of **perfect separation**. The observations are in red.
- The two classes can be perfectly separated by letting $\beta_1 \rightarrow \infty$.



Multiclass logistic regression: multinomial regression

- ▶ Logistic regression extends to the **multiclass** case $Y \in \{1, \dots, q\}$ by specifying

$$\Pr(Y = j \mid X = x) = \frac{e^{x^\top \beta^{(j)}}}{1 + \sum_{l=1}^{q-1} e^{x^\top \beta^{(l)}}}, \quad j = 1, \dots, q-1,$$
$$\Pr(Y = q \mid X = x) = \frac{1}{1 + \sum_{l=1}^{q-1} e^{x^\top \beta^{(l)}}},$$

where $\beta^{(1)}, \dots, \beta^{(q-1)} \in \mathbb{R}^p$ are parameter vectors to be estimated.

- ▶ The model uses $q - 1$ equations for q classes: this is because of the constraint that the probabilities must sum to one.
- ▶ Maximum likelihood estimation: the distribution of $Y \mid X$ is **multinomial** and we can obtain the likelihood

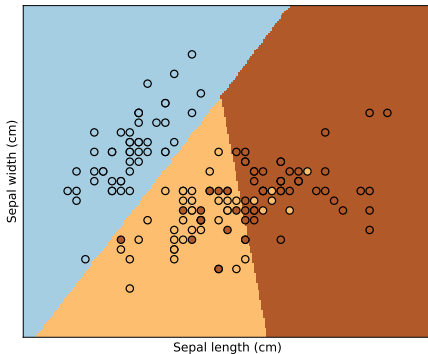
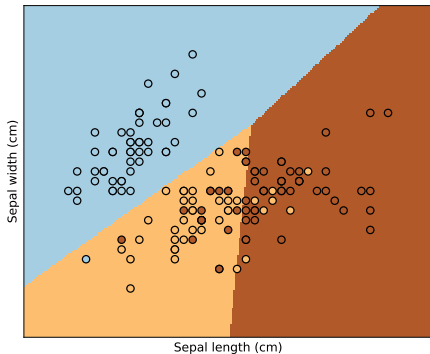
$$\ell(\beta) = \sum_{i=1}^n \log \Pr(Y = y_i \mid X = x_i).$$

Iris data set: multinomial (logistic) regression

For 150 measured plants; $p = 4$ predictors as response the three Iris species

$\mathcal{G} = \{\text{setosa}, \text{versicolor}, \text{virginica}\}$ the plant belongs. We only use two predictors here.

```
from sklearn.linear_model import LogisticRegression
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target # we only keep the first two features: 'Sepal length', 'Sepal width'
logreg = LogisticRegression(penalty="none")
logreg.fit(X, y)
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
h = .02 # step size in the mesh
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
Z = logreg.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
```



- ▶ Other link functions than σ can be used to map responses $x^\top \beta$ to probabilities, e.g., the so-called **probit regression**

$$\Pr(Y = 1 \mid X = x) = \Phi(x^\top \beta),$$

where Φ is the univariate Gaussian distribution function. Logistic is usually preferable to probit regression: it is more robust to outliers.

- ▶ Logistic regression is a special case of a **generalized linear model** (GLM).

Logistic regression vs. LDA

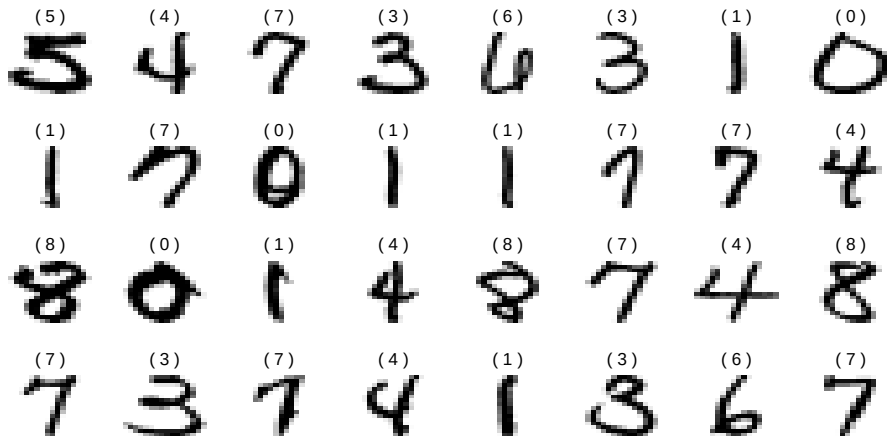
- ▶ For both models the log-odds between any two classes are linear in x .
- ▶ The difference between LDA and logistic regression comes from the **estimation** of their parameters.
- ▶ LDA models the joint distribution of (Y, X) , while logistic regression only models $Y \mid X$. If the Gaussian assumption for the conditional class densities is reasonable then LDA may perform better. If the Gaussian assumption is clearly unreasonable then logistic regression will outperform LDA.
- ▶ Often they have similar performance.

Chapter 12: Image recognition

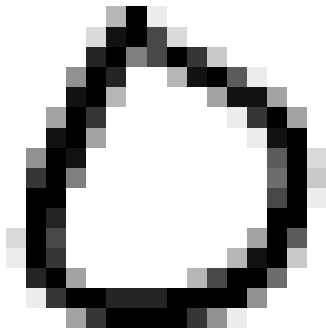
1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
- 12. Image recognition**
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Example: handwritten digit classification

Dataset from [\[ESL\]](#) webpage. Normalized handwritten digits (16×16 grayscale images), automatically scanned from envelopes by the **U.S. Postal Service**. Observations consists of the digit $y_i \in \{0, \dots, 9\}$ and the $p = 256$ grayscale values $\mathbf{x}_i = (x_{i1}, \dots, x_{i256})$. The training and test sets contain 7290 and 2006 examples, respectively.

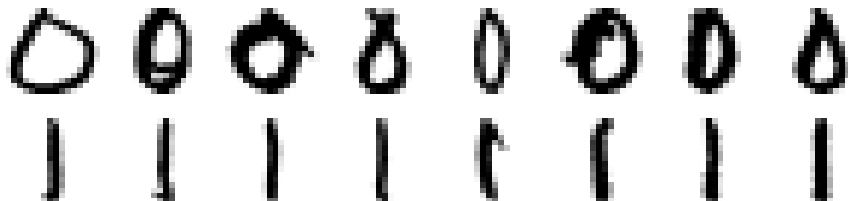


Digital black-white images

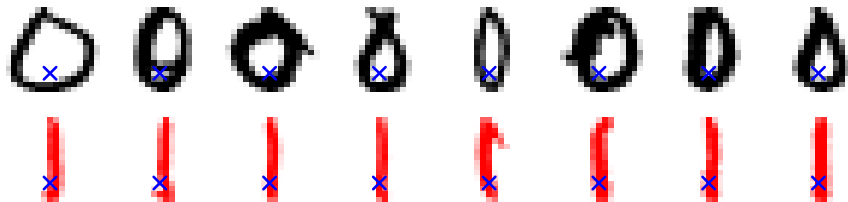


0	0	0	0	0	70	240	33	0	0	0	0	0	0	0	0	0	0
0	0	0	0	36	230	255	181	43	0	0	0	0	0	0	0	0	0
0	0	0	3	218	255	131	180	248	199	59	0	0	0	0	0	0	0
0	0	0	112	255	212	12	4	82	230	253	169	26	0	0	0	0	0
0	0	8	225	255	81	0	0	0	11	95	234	227	99	1	0	0	0
0	0	95	255	196	2	0	0	0	0	0	21	198	255	91	0	0	0
0	8	235	255	94	0	0	0	0	0	0	0	22	235	244	7	0	0
0	109	255	225	3	0	0	0	0	0	0	0	0	157	255	49	0	0
0	200	255	122	0	0	0	0	0	0	0	0	0	150	255	55	0	0
10	247	252	17	0	0	0	0	0	0	0	0	0	173	255	28	0	0
17	253	211	1	0	0	0	0	0	0	0	0	0	8	239	239	5	0
51	255	180	0	0	0	0	0	0	0	0	0	0	92	255	162	0	0
28	254	204	0	0	0	0	0	0	0	0	4	72	230	253	60	0	0
0	208	244	88	15	0	0	0	13	85	176	255	253	152	0	0	0	0
0	26	194	255	252	206	206	206	240	255	255	241	116	0	0	0	0	0
0	0	6	91	190	255	255	255	255	192	105	24	0	0	0	0	0	0

Digital black-white images

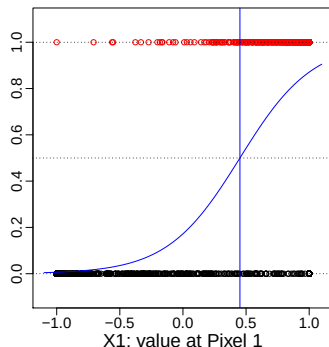


Two-class classification: one predictor

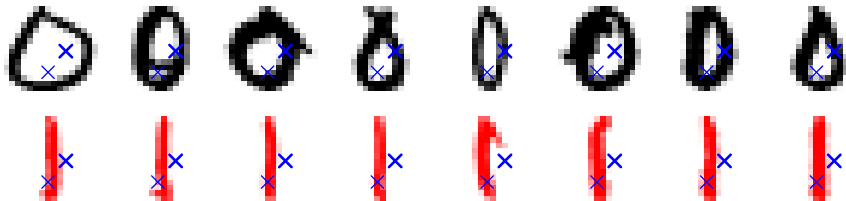


- ▶ Value at Pixel 1 (blue cross) is the predictor
 $X_1 = \text{"value at Pixel 1"} \in [-1, 1]$
- ▶ The class is $Y \in \{0, 1\} = \{\text{zero}, \text{one}\}$
- ▶ We apply logistic regression to obtain a **classifier** based only on Pixel 1:

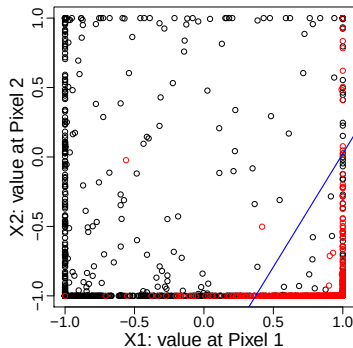
$$\hat{G}(x) = \begin{cases} \text{zero} & \text{if (Pixel 1)} < 0.45 \\ \text{one} & \text{otherwise.} \end{cases}$$



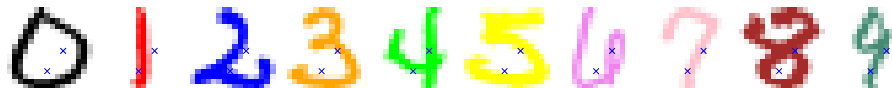
Two-class classification: two predictors



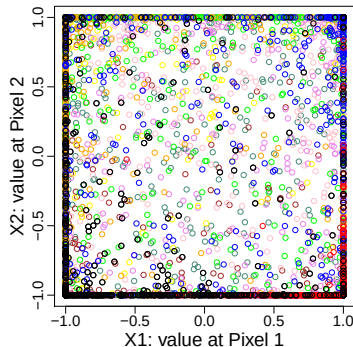
- ▶ $p = 2$: Predictors are
 X_1 = “value at Pixel 1” $\in [-1, 1]$
 X_2 = “value at Pixel 2” $\in [-1, 1]$
- ▶ The class is $Y \in \{0, 1\} = \{\text{zero}, \text{one}\}$
- ▶ We apply logistic regression to obtain a **classifier** based only on X_1 and X_2 .



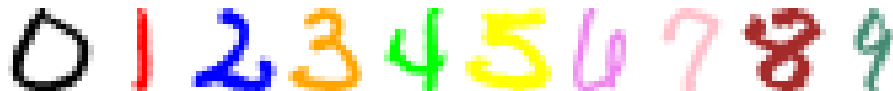
10-class classification: two predictors



- ▶ $p = 2$: Predictors are
 $X_1 = \text{"value at Pixel 1"} \in [-1, 1]$
 $X_2 = \text{"value at Pixel 2"} \in [-1, 1]$
- ▶ The class is
 $Y \in \{0, 1, \dots, 9\} = \{\text{zero}, \text{one}, \dots, \text{nine}\}$
- ▶ We can apply **multinomial regression**, based on X_1 and X_2 , but with only two pixels this a very hard task!
- ▶ Luckily we have $p = 256$ predictors (pixel values) to perform this task...



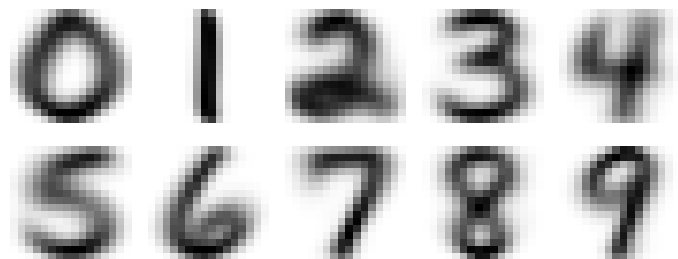
Digit classification



- ▶ **Training data:** $(x_1, y_1), \dots, (x_n, y_n)$, $n = 7290$, are used for model fitting/selection, where
 - ▶ predictors $x_i = \text{i-th image} \in \mathbb{R}^p$ are the grayscale values of the $p = 256$ pixels;
 - ▶ outcomes $y_i = \text{i-th digit} \in \{\text{zero, one, } \dots, \text{nine}\}$.
 - ▶ **Test data:** $(\tilde{x}_1, \tilde{y}_1), \dots, (\tilde{x}_m, \tilde{y}_m)$, $m = 2006$, are not used for model fitting/selection, but only to evaluate the performance afterwards.
 - ▶ linreg: direct linear regression
 - ▶ LDA1: LDA based on the values x_i
 - ▶ LDA2: LDA based on x_i and x_i^2
 - ▶ QDA: different cov. matrix for each class
 - ▶ log: logistic regression (multinomial)
- | | Training error | Test error |
|--------|----------------|------------|
| linreg | 7.6% | 13.1% |
| LDA1 | 6.2% | 11.5% |
| LDA2 | 3.9% | 10.2% |
| QDA | 1.8% | 13.5% |
| log | 0.01% | 11.1% |
- ▶ QDA is worst of the four methods since its variance is too high (overfitting).

Digit classification: visualization of LDA1

- ▶ Figure shows estimates $\hat{\mu}_1, \dots, \hat{\mu}_q$ for LDA1.
- ▶ This gives an impression of the centers of the classes, i.e., how American people write in ``average'' the 10 digits.

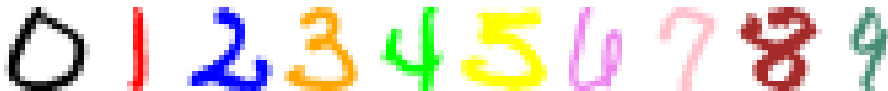


Example: digit classification

The table shows some misclassified test data from LDA1. The 10 columns on the right give the **predicted class probabilities** (in %) for $k \in \{0, \dots, 9\}$.

Obs		0	1	2	3	4	5	6	7	8	9
	(6)	59	0	0	0	0	0	40	0	0	0
	(2)	94	0	0	6	0	0	0	0	0	0
	(3)	0	0	0	11	0	0	0	0	89	0
	(3)	0	0	0	1	0	99	0	0	0	0
	(8)	1	0	0	81	1	11	0	0	5	0
	(4)	0	0	0	0	15	0	0	0	0	85
	(9)	0	0	0	0	0	0	0	100	0	0
	(2)	100	0	0	0	0	0	0	0	0	0

Digit classification with regularization



- ▶ We can update our results on the **digit classification**.
- ▶ When estimating the logistic multinomial regression, we introduce a **ridge penalty** and a **lasso penalty** in the likelihood.
- ▶ The value of $\hat{\lambda}$ for the tuning parameters were found by 5-fold CV.
- ▶ linreg: direct linear regression;
- ▶ LDA1: LDA based on the values x_i ;
- ▶ LDA2: LDA based on x_i and x_i^2 ;
- ▶ QDA: different cov. matrix for each class;
- ▶ log: logistic regression (multinomial).
- ▶ log-ridge: logistic regression (multinomial) with ridge penalty.
- ▶ log-lasso: logistic regression (multinomial) with lasso penalty.

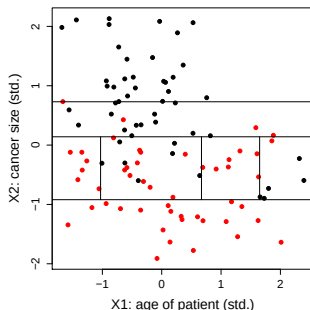
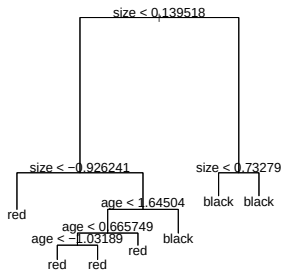
	Training error	Test error
linreg	7.6%	13.1%
LDA1	6.2%	11.5%
LDA2	3.9%	10.2%
QDA	1.8%	13.5%
log	0.01%	11.1%
log-ridge	4.1%	9.0%
log-lasso	2.7%	8.8%

Chapter 13: Decision trees: regression

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
- 13. Decision trees: regression**
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

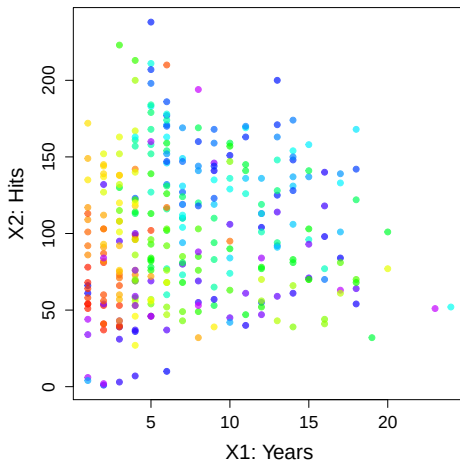
Tree-based methods

- ▶ **Tree-based methods** stratify/segment the feature space recursively into simple regions, namely rectangles $R_1, \dots, R_J$.
 - ▶ They can be applied to both **classification** and **regression**.
 - ▶ To make predictions for a given observation, we use the **mean** or the **most common class** of the training observations in the region to which it belongs.
1. **Decision trees**: Single tree with good **interpretability**.
 2. **Bagging/Boosting**: Combining multiple trees to **improve prediction**.
 3. **Random Forest**: **Decorrelation** of bagged trees.



Regression example: Hitters data set

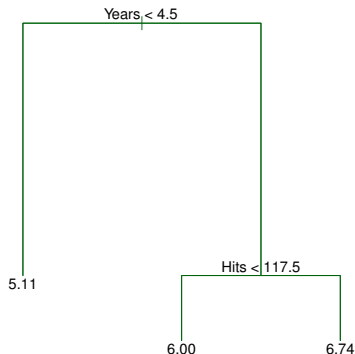
The **Hitters** data set from the **ISLR** package contains the annual **Salary** of 263 baseball players and $p = 19$ predictors such as numbers **Years** played in major leagues and the number of **Hits**. **Salary** color-code: low (yellow-red) to high (blue, green).



Regression trees: terminology

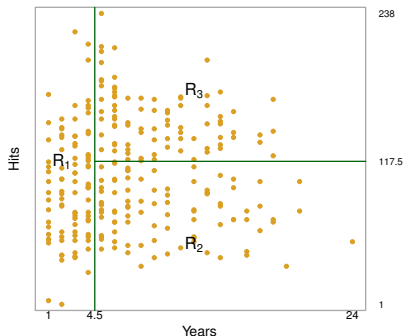
Regression tree for predicting the log **Salary** of a baseball player using only the variables **Years** and **Hits** as predictors.

- ▶ Each split is called **node**.
- ▶ A terminal node is called a **leaf**.
- ▶ The interior nodes lead to **branches**.
- ▶ The label $(X_j < t)$ indicates the **left-hand branch** from that split and the right-hand split corresponds to $(X_j \geq t)$.
- ▶ The number in each leaf is the **mean** of the responses for the observations that fall there.
- ▶ Note that an advantage of trees is the **interpretability**: the structure corresponds to the way people might think/act in a **decision problems**.



Regression trees: segmentation of feature space

- ▶ The tree segments the feature space into three regions corresponding to the **terminal nodes**.
- ▶ $R_1 = \{X \mid \text{Years} < 4.5\}$
- ▶ $R_2 = \{X \mid \text{Years} \geq 4.5, \text{Hits} < 117.5\}$
- ▶ $R_3 = \{X \mid \text{Years} \geq 4.5, \text{Hits} \geq 117.5\}$



Interpretation:

- ▶ **Years** is the most important factor in determining **Salary**, and players with less experience earn lower salaries than more experienced players.
- ▶ Given that a player is less experienced, the number of **Hits** seems to play little role in his **Salary**.
- ▶ But among players who have been in the major leagues for five or more years, the number of **Hits** does affect **Salary**.

How to grow a regression tree?

Given a **training data** set $(x_1, y_1), \dots, (x_n, y_n)$, with $x_i \in \mathbb{R}^p$ and $y_i \in \mathbb{R}$, how do we grow a regression tree?

- ▶ We want to divide the predictor space of $(X_1, \dots, X_p)$ into J distinct non-overlapping regions $R_1, \dots, R_J$.
- ▶ The regions could have any shape, but for trees we choose **high-dimensional rectangles** or **boxes** for the R_j .
- ▶ For every observation that falls in to region R_j we predict the **same value**

$$\hat{y}_{R_j} = \frac{1}{n_j} \sum_{i: x_i \in R_j} y_i,$$

namely the mean of the responses of all training observations in R_j , where n_j denotes the number of training training observations in R_j .

- ▶ The goal is to find boxes $R_1, \dots, R_J$ that **minimize the RSS** given by

$$\sum_{j=1}^J \sum_{i: x_i \in R_j} (y_i - \hat{y}_{R_j})^2.$$

Tree growing: greedy algorithm

- ▶ Unfortunately it is **computationally impossible** to search over all possible partitions of the feature space in J boxes.
- ▶ Instead, one uses a **top-down, greedy** algorithm that is also known as **recursive binary splitting**.
- ▶ It is **top-down** because it begins at the top of the tree and then **successively splits** the predictor space. Each split is indicated via two new branches further down on the tree.
- ▶ It is **greedy** because at each step of the tree-building process, the best split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.

Tree growing: greedy algorithm

The greedy algorithm:

- (1) For any predictor j and **cutpoint** t define the half-planes

$$R_1(j, t) = \{X \mid X_j < t\} \text{ and } R_2(j, t) = \{X \mid X_j \geq t\}.$$

- (2) We seek the values of j^* and t^* that minimize the RSS

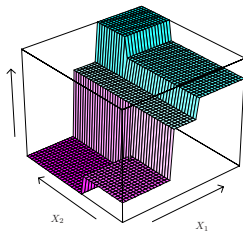
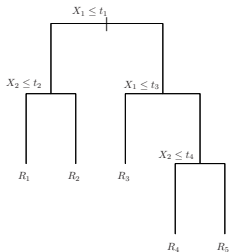
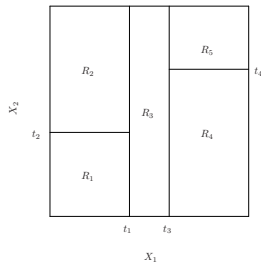
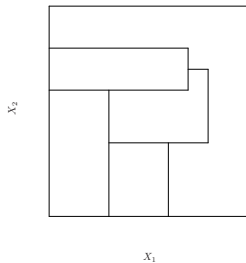
$$\sum_{i: x_i \in R_1(j, t)} (y_i - \hat{y}_{R_1(j, t)})^2 + \sum_{i: x_i \in R_2(j, t)} (y_i - \hat{y}_{R_2(j, t)})^2.$$

We split the predictor space in the two regions $R_1(j^*, t^*)$ and $R_2(j^*, t^*)$.

- (3) We continue the process by looking for the **best predictor and cutpoint**, but now, instead of splitting the entire predictor space, we split one of the two previously identified regions.
- (4) We continue the process until a **stopping criterion** is reached, e.g., until no region contains more than five observations.
- (5) Once the regions $R_1, \dots, R_J$ are created, we predict the response for a given **test observation** using the mean $\hat{y}_{R_j}$ of the training observations in the region R_j to which that test observation belongs, that is, a tree corresponds to a **piece-wise constant predictive function**

$$\hat{f}(x) = \sum_{j=1}^J \hat{y}_{R_j} \mathbf{1}\{x \in R_j\}.$$

Regression tree: two-dimensional feature space



Tree pruning

Tree properties

- ▶ Intuitively, the **complexity** of the predictive model corresponding to a tree T is the number of terminal nodes, denoted by $|T|$, e.g., the number J of boxes $R_1, \dots, R_J$.

Bias-variance trade off

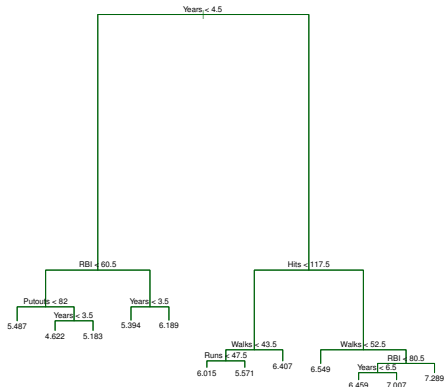
- ▶ A very complex, large tree may produce good prediction on the training set, but it is likely to **overfit** the data leading to a poor test error.
- ▶ A **less complex, smaller tree** with fewer splits/regions might lead to lower variance but to higher bias, and *vice versa* for more complex trees.
- ▶ A way of avoiding overfitting is to first grow a large tree T_0 and then **prune it back** in order to obtain a subtree.

Cost-complexity pruning

We can consider a sequence of trees indexed by a tuning parameter $\alpha \geq 0$. For each value of α there corresponds a subtree $T_\alpha \subset T_0$ that minimizes

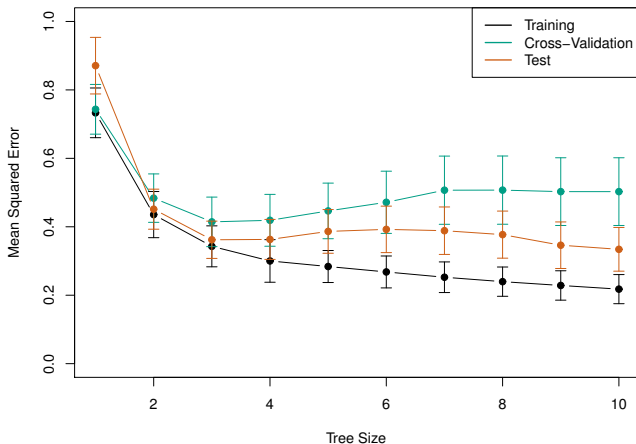
$$\sum_{j=1}^{|T_\alpha|} \sum_{i: x_i \in R_j} (y_i - \hat{y}_{R_j})^2 + \alpha |T_\alpha|.$$

- ▶ This is called **cost-complexity pruning** or **weakest-link pruning**.
- ▶ The number $|T_\alpha|$ of terminal nodes of T_α indicates the complexity of the tree.
- ▶ The **regularization parameter** α controls the tree complexity.
- ▶ It has a similar interpretation as the parameter λ in lasso or ridge regression.
- ▶ The optimal value is chosen by **cross-validation**.



Regression example: Hitters data set

Tree growing and pruning illustrated on the **Hitters** data set. The data is split into 132 training observations and 131 test observations. A large regression tree is built on the 132 training data and α is varied to obtain pruned subtrees with different numbers of terminal nodes. **Six-fold CV** is performed on the 132 training data to estimate the MSE of the trees. The test error is computed on the 131 test observations that are **not used** for model fitting or selection.



Chapter 14: Decision trees: classification

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
- 14. Decision trees: classification**
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Classification trees

- **Classification trees** are very similar to regression trees, except that they are used to predict a **qualitative response** $Y \in \mathcal{G} = \{1, \dots, q\}$ instead of a quantitative one.
- The prediction in a region R_j is the constant $\hat{y}_{R_j}$ that equals the **most commonly occurring class** of the training observations in R_j , i.e., the $g = 1, \dots, q$ that maximizes

$$\hat{p}_{jg} = \frac{1}{n_j} \sum_{i: x_i \in R_j} \mathbf{1}\{y_i = g\},$$

so that the classifier is

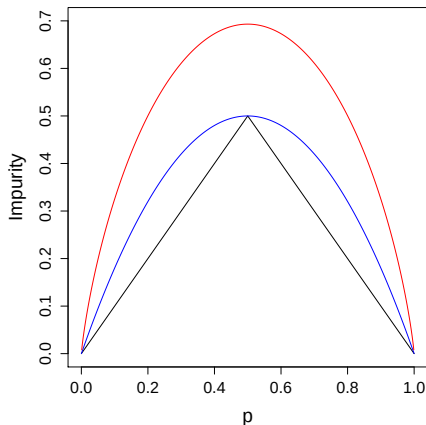
$$\hat{G}(x) = \sum_{j=1}^J \hat{y}_{R_j} \mathbf{1}\{x \in R_j\}.$$

- The principles of **pruning** and **subtrees** are the same as for regression trees.
- For classification, the RSS cannot be used as **loss function** and instead one uses **measures of node impurity** for the terminal node R_j :

- (1) misclassification error: $M = \frac{1}{n_j} \sum_{i: x_i \in R_j} \mathbf{1}\{y_i \neq \hat{y}_{R_j}\};$
- (2) **Gini index**: $G = \sum_{g=1}^q \hat{p}_{jg}(1 - \hat{p}_{jg});$
- (3) **Cross-entropy/deviance**: $D = - \sum_{g=1}^q \hat{p}_{jg} \log \hat{p}_{jg}.$

Node impurity measures

- ▶ In the case of two classes, the misclassification error is $1 - \max(p, 1 - p)$ (black), the Gini index is $2p(1 - p)$ (blue), and the cross-entropy is $-p \log p - (1 - p) \log(1 - p)$ (red).
- ▶ The misclassification impurity is non-differentiable and thus bad for optimization.
- ▶ The Gini index and the cross-entropy will **favor pure nodes** with $p_{jg} \approx 0$ or $p_{jg} \approx 1$; they are very **similar numerically**.



Example: heart disease

The **heart** data set in the **ISLR** package contains whether or not the patient has a heart disease (AHD), i.e. $G \in \{\text{Yes}, \text{No}\}$, for 303 patients who presented with chest pain, with 13 predictors (**sex**, **age**, **chol**,...). A subset of the data:

	AHD	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR
1	No	63	1	typical	145	233	1	2	150
2	Yes	67	1	asymptomatic	160	286	0	2	108
3	Yes	67	1	asymptomatic	120	229	0	2	129
4	No	37	1	nonanginal	130	250	0	0	187
5	No	41	0	nontypical	130	204	0	2	172
6	No	56	1	nontypical	120	236	0	0	178
7	Yes	62	0	asymptomatic	140	268	0	2	160
8	No	57	0	asymptomatic	120	354	0	0	163
9	Yes	63	1	asymptomatic	130	254	0	2	147
10	Yes	53	1	asymptomatic	140	203	1	2	155
11	No	57	1	asymptomatic	140	192	0	0	148
12	No	56	0	nontypical	140	294	0	2	153
13	Yes	56	1	nonanginal	130	256	1	2	142
14	No	44	1	nontypical	120	263	0	0	173
15	No	52	1	nonanginal	172	199	1	0	162
16	No	57	1	nonanginal	150	168	0	0	174
17	Yes	48	1	nontypical	110	229	0	0	168
18	No	54	1	asymptomatic	140	239	0	0	160
19	No	48	0	nonanginal	130	275	0	0	139
20	No	49	1	nontypical	130	266	0	0	171

Example: heart disease

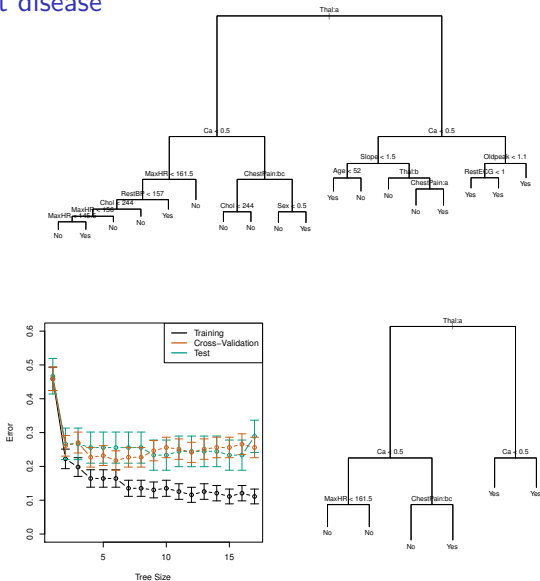
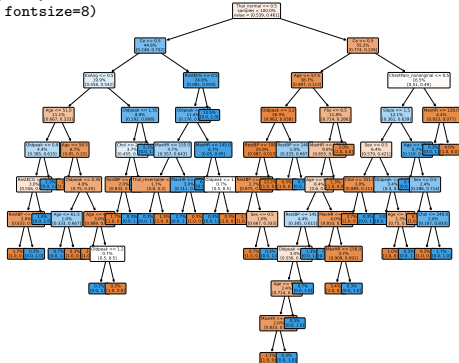
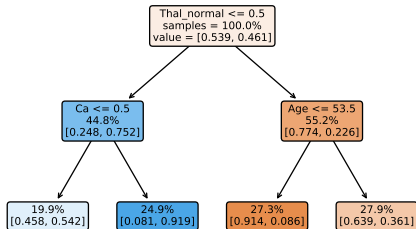


Figure: Heart data. Top: The unpruned tree. Bottom left: CV error, training, and test error, for different sizes of the pruned tree. Bottom right: The pruned tree corresponding to the minimal CV error.

- ▶ In the above tree there are some **qualitative predictors**, e.g., **Thal** and **ChestPain**.
- ▶ For a predictor with r possible unordered values, the tree considers the best split into one of the $2^{r-1} - 1$ **possible partitions** of the r values into two groups.
- ▶ For instance, the split **ChestPain:bc** splits the tree such that the left-hand branch coming out of that node consists of observations with the second and third values of the **ChestPain** variable.
- ▶ Some of the splits yield two terminal nodes that have the **same predicted value**. The split is performed because it leads to **increased node purity**, that is, in one of the terminal nodes we are **more certain** about the prediction.
- ▶ Trees can handle missing predictors values; for details see Section 9.2.4 in **[ESL]**.

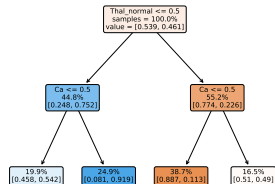
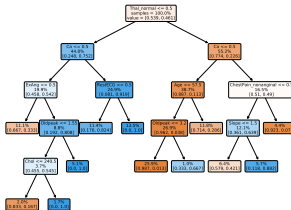
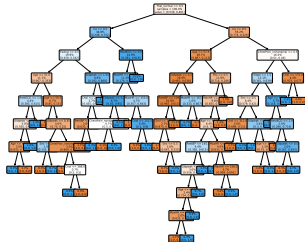
Tree growing in python

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
heart = pd.read_csv("Heart.csv")
X, y = heart.drop("AHD", axis=1), heart["AHD"]
tree1 = DecisionTreeClassifier(criterion='entropy', min_samples_leaf=50)
tree1.fit(X, y)
plot_tree(tree1, feature_names=X.columns.tolist(), impurity=False, label="root",
          filled=True, proportion=True, rounded=True, fontsize=8)
tree2 = DecisionTreeClassifier(criterion='entropy', min_samples_leaf=1)
tree2.fit(X, y)
plot_tree(tree2, feature_names=X.columns.tolist(), impurity=False, label="root",
          filled=True, proportion=True, rounded=True, fontsize=8)
```



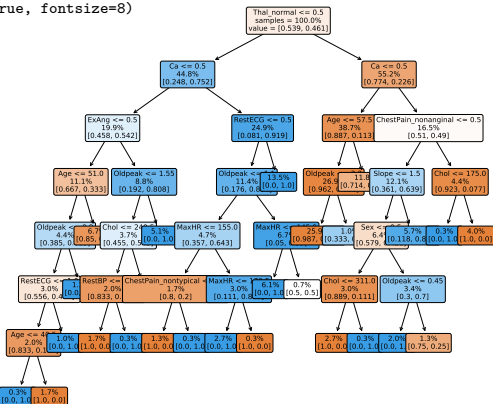
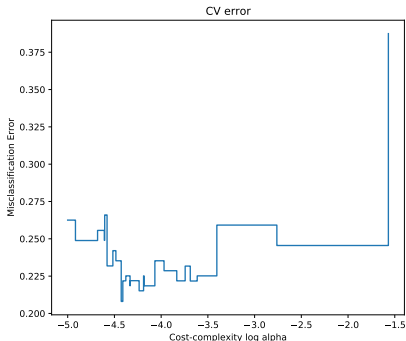
Tree pruning in python

```
tree_full = DecisionTreeClassifier(criterion='entropy', min_samples_leaf=1, ccp_alpha=0)
tree_full.fit(X, y)
plot_tree(tree_full, feature_names=X.columns.tolist(), impurity=False, label="root",
          filled=True, proportion=True, rounded=True, fontsize=8)
tree_pruned1 = DecisionTreeClassifier(criterion='entropy', min_samples_leaf=1, ccp_alpha=0.02)
tree_pruned1.fit(X, y)
plot_tree(tree_pruned1, feature_names=X.columns.tolist(), impurity=False, label="root",
          filled=True, proportion=True, rounded=True, fontsize=8)
tree_pruned2 = DecisionTreeClassifier(criterion='entropy', min_samples_leaf=1, ccp_alpha=0.05)
tree_pruned2.fit(X, y)
plot_tree(tree_pruned2, feature_names=X.columns.tolist(), impurity=False, label="root",
          filled=True, proportion=True, rounded=True, fontsize=8)
```

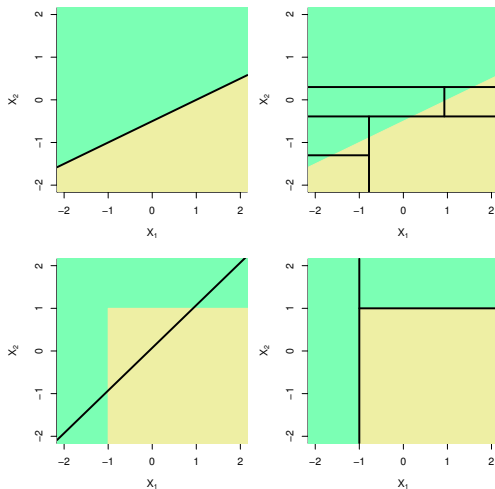


Cost-complexity pruning in python

```
tree_full = DecisionTreeClassifier(criterion='entropy', min_samples_leaf=1, ccp_alpha=0)
path = tree_full.cost_complexity_pruning_path(X, y)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
hyper_parameters = {"ccp_alpha" : grid_alphas}
treeCV = GridSearchCV(estimator=tree_full, param_grid=hyper_parameters, cv=folds)
treeCV.fit(X, y)
best_alpha = alpha_grid[np.argmax(1-treeCV.cv_results_["mean_test_score"])]
best_tree = DecisionTreeClassifier(criterion='entropy', min_samples_leaf=1, ccp_alpha=best_alpha)
best_tree.fit(X,y)
plot_tree(best_tree, feature_names=X.columns.tolist(), impurity=False, label="root",
          filled=True, proportion=True, rounded=True, fontsize=8)
```



Trees versus linear classifiers



Pros and cons of trees

Pros:

- ▶ Trees are very **easy to explain** to people. In fact, they are even easier to explain than linear regression!
- ▶ They mirror human **decision-making**.
- ▶ Trees can be displayed **graphically**, and are easily interpreted even by a non-expert.
- ▶ Can handle **qualitative predictors** without using dummy variables.

Cons:

- ▶ Trees generally have **poor predictive** performance.
- ▶ A small change in the data can completely change the tree.

Although a single tree usually gives poor predictions, very accurate models can be obtained by **combining many trees** (bagging, random forests, boosting).

Chapter 15: Bagging

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
- 15. Bagging**
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Bagging: main idea

- ▶ **Bootstrap aggregation**, or **bagging** is a general-purpose procedure for reducing the variance of a statistical learning method.
- ▶ Recall that given a set of B **independent** observations $z_1, \dots, z_B$ each with variance σ^2 , the variance of the mean

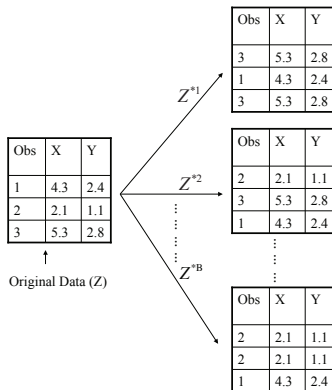
$$\bar{z} = \frac{1}{B} \sum_{i=1}^B z_i$$

of the observations is given by σ^2/B .

- ▶ We can thus **reduce the variance** by **averaging a set of predictions**. Of course, this is not practical because we generally do not have access to multiple training sets.
- ▶ Instead we use bootstrap by taking repeated samples from the training data $Z = \{(x_i, y_i), i = 1, \dots, n\}$.

The bootstrap

- ▶ Let B be the number of desired bootstrap data sets $Z^{*1}, \dots, Z^{*B}$.
- ▶ For $b = 1, \dots, B$:
- ▶ sample n observations from Z **with replacement**;
- ▶ this is the b th **bootstrap data set** denoted by Z^{*b} .
- ▶ Each bootstrap data set contains **on average** 63.2% of the original samples.



Bagging

- ▶ Let $Z^{*1}, \dots, Z^{*B}$ be B bootstrap data sets from the training data.
- ▶ For all $b = 1, \dots, B$ we can fit our predictive regression model $\hat{f}^{*b}$ on the b th bootstrap data set Z^{*b} .
- ▶ We then average all the predictions to obtain a single, bagged model

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x),$$

which is called bagging.

- ▶ Especially if the single functions $\hat{f}^{*b}$ have a **low bias but a high variance** the **combined estimator** $\hat{f}_{\text{bag}}$ has a similar bias but **lower variance**!
- ▶ In practice we use a value of B sufficiently large that the **error has settled down**. Using a value of B between 100 and 1000 is often sufficient.

Out-of-bag error estimate

- ▶ There is a very straightforward way to estimate the test error of a bagged model, called the **out-of-bag (OOB)** error estimate.
- ▶ The key idea of bagging is that the model is repeatedly fitted to bootstrapped subsets of the observations, which contain on average around **two-thirds of the observations** ($\sim 63\%$).
- ▶ The **remaining one-third** ($\sim 37\%$) of the observations not used to fit a given bagged model are referred to as the **out-of-bag** observations.
- ▶ We can predict the response for the i th observation using all models $\hat{f}^{*b}$ with $b \in I_{x_i} = \{b = 1, \dots, B : (x_i, y_i) \notin Z^{*b}\}$, in which that **observation was OOB**. This will yield around $B/3$ predictions for the i th observation, which we average:

$$\text{Err}_{\hat{f}_{\text{bag}}}(x_i) = \frac{1}{|I_{x_i}|} \sum_{b \in I_{x_i}} (y_i - \hat{f}^{*b}(x_i))^2.$$

- ▶ The average over all n observations is called **out-of-bag (OOB) error** estimate:

$$\text{Err}_{\hat{f}_{\text{bag}}} = \frac{1}{n} \sum_{i=1}^n \text{Err}_{\hat{f}_{\text{bag}}}(x_i).$$

- ▶ For large B this estimator can be shown to be **equivalent to the Leave-One-Out CV** estimator. But in bagging it comes for free!

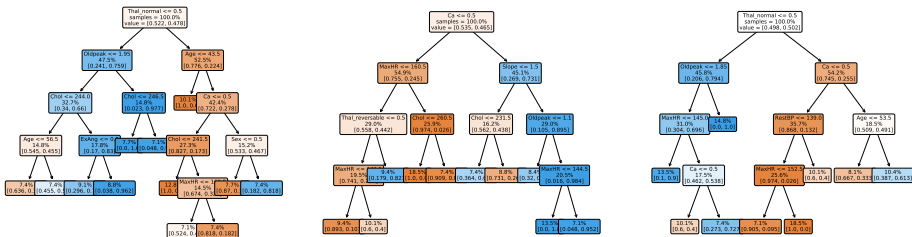
Bagging for trees

- ▶ Classification and regression trees are **perfectly suited** for bagging, since their poor predictive accuracy comes from their high variance.
- ▶ For a regression problem, for all B bootstrap training sets we grow a large tree **without pruning**, to obtain a **high-variance** predictive model.
- ▶ We then average all these (possibly overfitted) trees to obtain a **bagged** model with reduced variance.
- ▶ This has shown to give impressive improvements in **prediction accuracy**, at the price of interpretability.
- ▶ Important: here, B is **not** a classical **tuning parameter**, since large B will not overfit the data; the error will just stabilize.
- ▶ For **classification**, we fit B classification trees $\hat{G}^{*b}$ and then choose the predicted class by **majority vote**, that is

$$\hat{G}_{\text{bag}}(x) = \operatorname{argmax}_{g=1,\dots,q} \frac{1}{B} \sum_{b=1}^B \mathbf{1}\{g = \hat{G}^{*b}(x)\}.$$

Bootstrapped trees in python

```
np.random.seed(1)
fig, axs = plt.subplots(1,3,figsize=(24,7))
for i in range(1,4):
    boot_indexes = np.random.choice(X.index, size=X.shape[0], replace=True)
    tree = DecisionTreeClassifier(criterion='entropy', min_samples_leaf=21, ccp_alpha=0)
    tree.fit(X.loc[boot_indexes], y.loc[boot_indexes])
    plot_tree(tree, feature_names=X.columns.tolist(), impurity=False, label="root",
              filled=True, proportion=True, rounded=True, fontsize=8, ax=axs[i-1])
plt.show()
```



Heart data: single tree

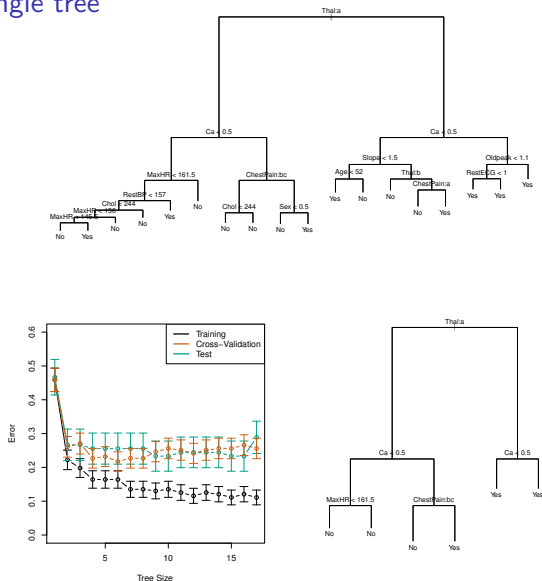
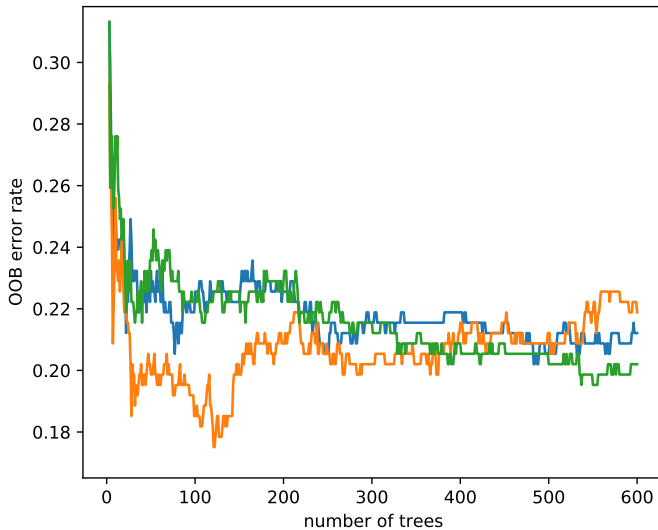


Figure: Heart data. Top: The unpruned tree. Bottom left: CV error, training, and test error, for different sizes of the pruned tree. Bottom right: The pruned tree corresponding to the minimal CV error.

Heart data: bagging



Chapter 16: Random forest

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
- 16. Random forest**
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

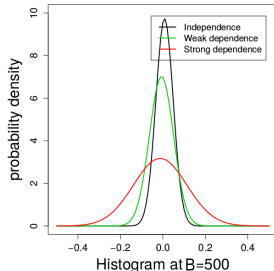
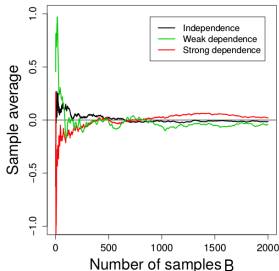
Variance reduction and correlation

- Recall that given a set of B **independent**(!) observations $z_1, \dots, z_B$ each with variance σ^2 , the variance of the mean

$$\bar{z} = \frac{1}{B} \sum_{i=1}^B z_i$$

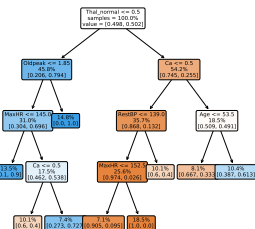
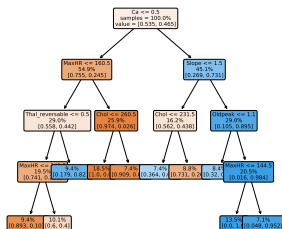
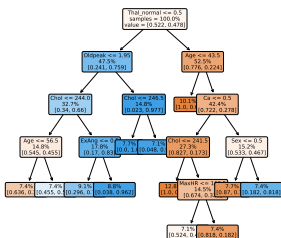
of the observations is given by σ^2/B .

- If the samples $z_1, \dots, z_B$ are not independent but **correlated** then this reduction in variance can change dramatically.
- Consider $z_i \sim N(0, 1)$, and the following the cases
 - (1) Independence: $\text{cor}(z_i, z_j) = 0$ for all i, j ;
 - (2) **Weak dependence**: $\text{cor}(z_i, z_{i+1}) = 0.3$ for all i ;
 - (3) **Strong dependence**: $\text{cor}(z_i, z_{i+1}) = 0.8$ for all i .



Bagging: correlated trees

- ▶ In bagging, the trees grown with each bootstrap data set **share many common** training observations.
- ▶ The greedy algorithm thus chooses often the **same predictors** for **early splits**!
- ▶ This means that the trees are very similar and the predictions $\hat{f}^{*b}(x_0)$ for a new input x_0 are very correlated for all $b = 1, \dots, B$.



Random Forest

- ▶ **Random Forest** extends the concept of bagging.
- ▶ The idea is to **decorrelate** the trees from the bootstrap data sets by introducing constraints in the growing process.

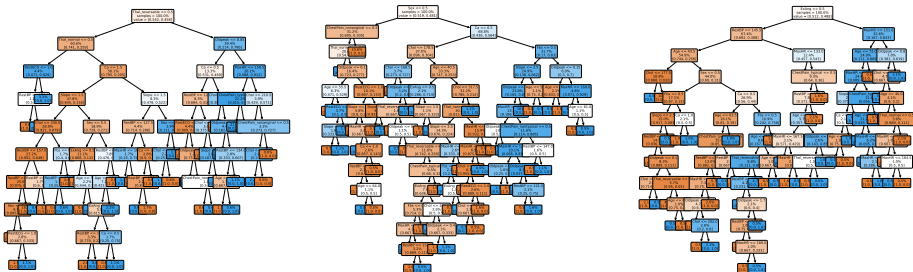
Random Forest algorithm:

- (1) Bootstrap the data B times.
- (2) When growing each tree on the bootstrap samples, for each split **randomly choose m of the p predictors** to be used for the split. This subset of variables changes for each split.
- (3) Grow **full unpruned trees** (pure terminal nodes).
- (4) For prediction (as for bagging):
 - ▶ **regression**: average the predictions from the B trees;
 - ▶ **classification**: majority vote from the B trees.

Random Forest

Intuition: selecting a random subset of m variables for each split **avoids too similar trees**.

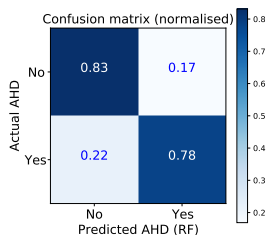
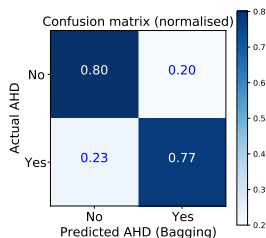
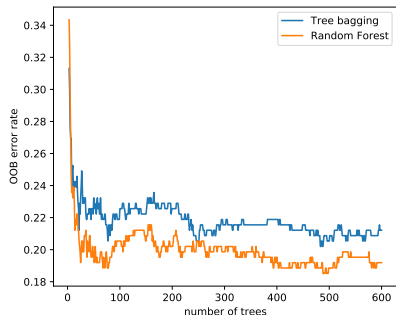
Choice of m : $m \approx \sqrt{p}$ for classification, and $m \approx p/3$ for regression typically works well in practice.



Heart data: bagging and random forest in python

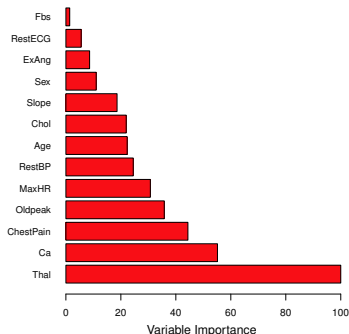
```
from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier(n_estimators=500, max_features="sqrt", oob_score=True,
                           warm_start=True, random_state=1)
bagging = RandomForestClassifier(n_estimators=500, max_features=None, oob_score=True,
                               warm_start=True, random_state=1)

oob_rfc, oob_bag = list(), list()
for i in range(3,601):
    rfc.set_params(n_estimators=i)
    rfc.fit(X, y)
    oob_rfc.append(1 - rfc.oob_score_)
    bagging.set_params(n_estimators=i)
    bagging.fit(X, y)
    oob_bag.append(1 - bagging.oob_score_)
plt.plot(np.arange(3,601), oob_bag, label="Tree bagging")
plt.plot(np.arange(3,601), oob_rfc, label="Random Forest")
```



Variable importance

- ▶ Bagging and random forest have **good predictive power**, but we can no longer plot a single tree for **interpretation**.
- ▶ However, we can define an overall summary of **variable importance**.
- ▶ For regression, we record the total amount that the **RSS is decreased** due to splits over a given predictor X_j , $j = 1, \dots, p$.
- ▶ This can be done by **randomly permuting** this given predictor in the **OOB observations** and then measure the increase in prediction error.
- ▶ Similarly, classification, we record the total amount that the **Gini index/deviance is decreased** due to splits over a given predictor.



Random forests and nearest-neighbors

- ▶ Random forests can be seen as an **adaptive/weighted nearest-neighbors method**, where “close” observations are assigned different **weights**.
- ▶ Recall that the trees in random forests are **fully grown and unpruned**.
- ▶ Hence for each tree the prediction of y_0 for a **new input** x_0 is equal to a response y_i corresponding to one training observation x_i , as in one-NN.
- ▶ The “neighbor” x_i might be obtained from a particular path in the feature space and **might not** correspond to the nearest observation in terms of Euclidean distance.
- ▶ Each **tree in the forest** will predict the value of y_0 as a y_i . The final random **forest prediction** (for regression) will be of the form

$$\hat{y}_0 = \sum_{i=1}^n w_i y_i,$$

where the weights $w_i \geq 0$ are the fractions of trees where x_i was **in the terminal node** of x_0 .

⇒ Random forests can thus be seen as a **smoothing** method with complex notion of neighbors.

Pros and cons of random forests

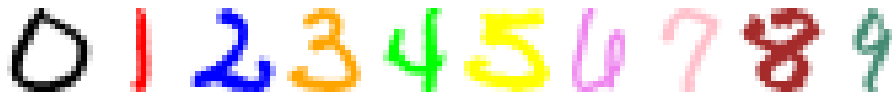
Pros:

- ▶ Great **predictive performance**.
- ▶ Almost no tuning needed.
- ▶ **Out-of-bag error** estimate; no need for CV.
- ▶ **Variable importance** measure available.
- ▶ Easy to fit using existing functions in R and **quite fast**.

Cons:

- ▶ By bagging trees we **lose the interpretability** of a single tree.

Digit classification with random forest

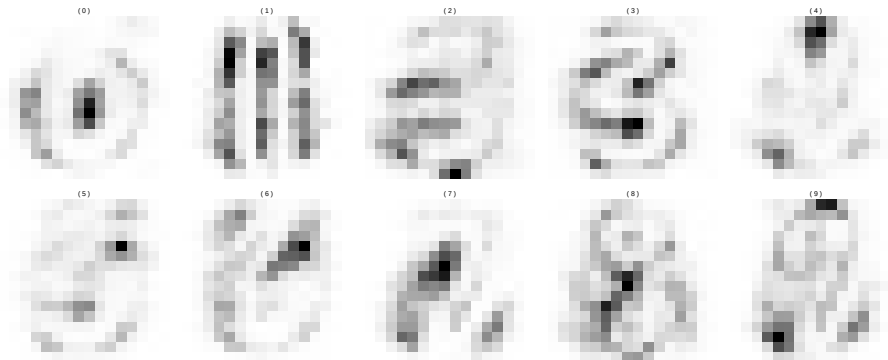


► We can update our results on the **digit classification**.

- linreg: direct linear regression;
- LDA1: LDA based on the values x_i ;
- LDA2: LDA based on x_i and x_i^2 ;
- QDA: different cov. matrix for each class;
- log: logistic regression (multinomial).
- log-ridge: log. reg. with ridge penalty.
- log-lasso: log. reg. with lasso penalty.
- RF: random forest.

	Training error	Test error
linreg	7.6%	13.1%
LDA1	6.2%	11.5%
LDA2	3.9%	10.2%
QDA	1.8%	13.5%
log	0.01%	11.1%
log-ridge	4.1%	9.0%
log-lasso	2.7%	8.8%
RF	0%	5.9%

Digit classification: variable importance



Chapter 17: Boosting

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
- 17. Boosting**
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Boosting for trees

- ▶ The idea of **bagging** is to use many **unbiased** models with **high variance** and reduce the variance by **averaging**.
- ▶ Bagging involved fitting **many large trees** to the bootstrap data sets to create a single predictive model.
- ▶ Like bagging, **boosting** is a general approach that can be applied to many statistical learning methods for **regression or classification**. We only discuss boosting for **regression trees**.
- ▶ Boosting works in a similar way as bagging, but the principle is to use many **weak, biased** models with **low variance**, and combine them to **reduce the bias**.
- ▶ The **main difference** is that in boosting, the trees are **grown sequentially**: each tree is grown using information from previously grown trees.

Boosting algorithm for regression trees

Fix the following constants:

- (a) The number of **repetitions** $B \in \mathbb{N}$.
- (b) The number of **splits** $d \in \mathbb{N}$ in each tree (i.e., $d + 1$ terminal nodes).
- (c) The **shrinkage** parameter $\lambda \in (0, 1)$.

Then run the following algorithm on the training set (x_i, y_i) , $i = 1, \dots, n$:

- (1) Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all $i = 1, \dots, n$.
- (2) For $b = 1, \dots, B$ repeat:
 - (i) Fit a tree $\hat{f}^b$ with d splits to the temporary training data (x_i, r_i) , $i = 1, \dots, n$.
 - (ii) Update the predictive model by adding a **shrunk version** of the new tree

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x).$$

- (iii) Update the residuals

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

- (3) Output the **boosted model**

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x).$$

Boosting for trees

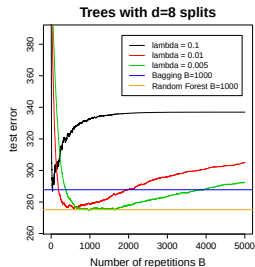
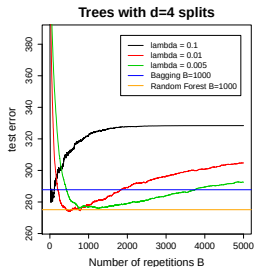
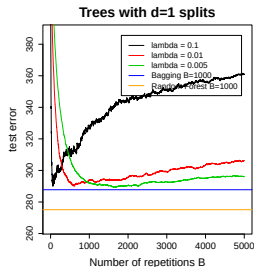
- ▶ Boosting is a form of **slow learning**, since we are fitting small trees that are slowly improving $\hat{f}$ in areas where it does not perform well.
- ▶ The shrinkage parameter λ slows the process down even further, allowing **more and different shaped** trees to attack the residuals.

Boosting has three **tuning parameters**:

- (a) The number of trees B . Unlike bagging and random forests, boosting can **overfit** if B is too large, although this overfitting tends to occur slowly. We use **CV** to select B .
- (b) The shrinkage parameter λ is typically a small positive number such as 0.01 or 0.001. It controls the **learning rate** of boosting. Smaller values may require larger B for good performance.
- (c) The number d of splits in each tree controls the **complexity** of the boosted ensemble. Often $d = 1$ works well, in which case each tree is a **stump**, consisting of a single split.

Boosting for classification is similar but a bit more complex to formulate.

Hitters data: boosting



Bagging vs Boosting

Chapter 18: Support Vector Machines: Maximal Margin Classifier

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
- 18. Support Vector Machines: Maximal Margin Classifier**
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Support vector machines

- ▶ **Support vector machines** (SVM) are a very popular technique for classification in machine learning.
- ▶ Recall that:
 - ▶ LDA models the joint distribution of (Y, X) ;
 - ▶ logistic regression models the conditional distribution of $Y \mid X$.
- ▶ SVM is a **geometric approach** and makes **no distributional assumption**.
- ▶ SVM directly tries to find the **separating hyperplane**

$$x \in \mathbb{R}^p : \quad \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = 0$$

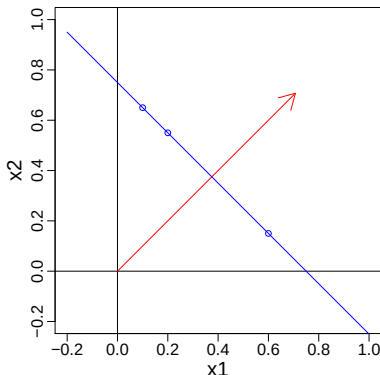
that separates the classes in the feature space.

1. **Maximal Margin Classifier**: perfect linear separation if the classes are **separable**.
 2. **Support Vector Classifier**: soft linear separation if the classes are **not separable**.
 3. **Support Vector Machines**: soft **non-linear separation** (in the original feature space) if the classes are not separable.
- ▶ SVM is a **kernel method** that relies on distances, so we will always assume that our predictors have been **standardized** to have mean 0 and variance 1.

Hyperplanes: construction

Hyperplane: $x \in \mathbb{R}^p : \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = 0$

- ▶ The vector $\beta = (\beta_1, \dots, \beta_p)$ is called **normal vector**, it points in the direction that is **orthogonal** to the hyperplane.
- ▶ Without losing generality, it can be normalized to have norm one: $\|\beta\|^2 = \sum_{j=1}^p \beta_j^2 = 1$.
- ▶ Then: for a point $z \in \mathbb{R}^p$, $|\beta_0 + \beta_1 z_1 + \cdots + \beta_p z_p|$ is the **distance from the hyperplane**.



Separating hyperplanes

- ▶ We first look at two classes, and we will always **identify** them as $\mathcal{G} = \{-1, +1\}$.
- ▶ For data $(x_1, y_1), \dots, (x_n, y_n)$, with $y_i \in \{-1, +1\}$, we say that a **hyperplane is separating** if

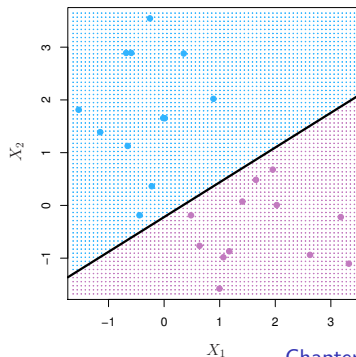
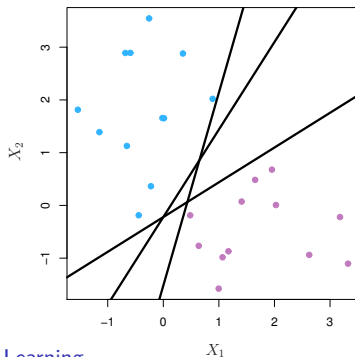
$$\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} > 0, \text{ if } y_i = +1,$$

$$\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} < 0, \text{ if } y_i = -1.$$

or equivalently, for all $i = 1, \dots, n$

$$y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) > 0.$$

- ▶ If a separating hyperplane exists, there are **infinitely many** of them.



Maximal margin classifier

- ▶ If a separating hyperplane exists it is a **natural classifier** for a new test point $x_0 \in \mathbb{R}^p$:

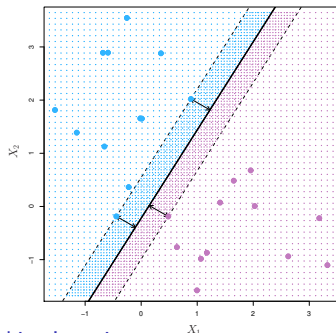
$$f(x_0) = \beta_0 + \beta_1 x_{01} + \cdots + \beta_p x_{0p},$$

and if $f(x_0) > 0$ is positive, we assign the class **+1**, and if $f(x_0) < 0$ we assign the class **-1**.

- ▶ The **magnitude** of $f(x_0)$ tells us how far it is from the hyperplane.
- ▶ The **margin** is the smallest distance of the training data from the hyperplane

$$M = \min_{i=1,\dots,n} y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}).$$

Maximal margin classifier: tries to make the **biggest gap** between the two classes.



Constrained optimization problem:

$$\text{maximize}_{\beta_0, \dots, \beta_p} M$$

$$\text{subject to } \sum_{j=1}^p \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}) \geq M,$$

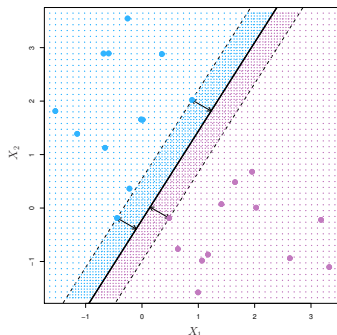
$$\text{for all } i = 1, \dots, n.$$

Comments and limitations

- ▶ This optimization problem is a **convex quadratic program** and can be solved efficiently.
- ▶ The observations (here three) determining the width of the margin are called **support vectors**. Moving the support vectors only slightly, the maximal margin hyperplane will move as well.
- ▶ Moving the other observations slightly, does not affect the hyperplane!

Limitations:

- ▶ The maximal margin hyperplane can be **unstable**, since it depends only on few support vectors.
- ▶ Most often, the data is **not separable**!

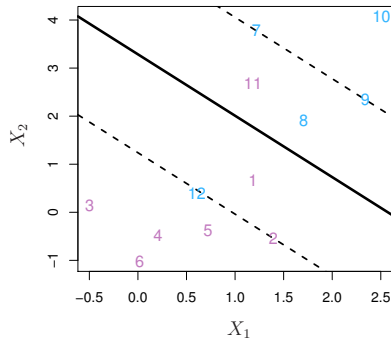
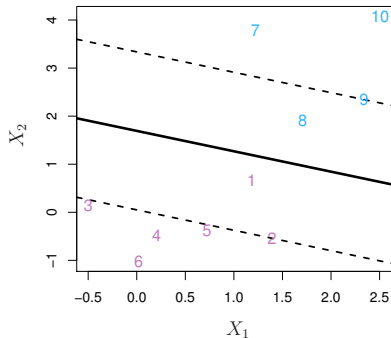


Chapter 19: Support Vector Machines: Support Vector Classifier

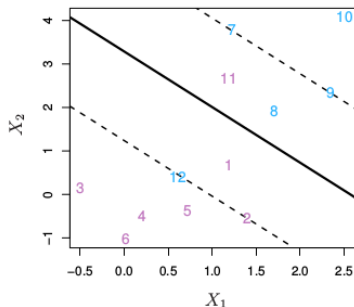
1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
- 19. Support Vector Machines: Support Vector Classifier**
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Support vector classifier

- ▶ The maximal margin classifier tries to separate the classes **perfectly**.
- ▶ The **support vector classifier** is a **soft margin classifier** that does not aim to separate the classes perfectly.
- ▶ It allows a “budget” to have observations on the **wrong side** of the margins, or even on the wrong side of the hyperplane, in order to
 - ▶ have **greater robustness** to individual observations;
 - ▶ do **better classification** on most of the observations;
 - ▶ allow to find a solution in the **non-separable case**.



Support vector classifier



$$\text{maximize}_{\beta_0, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n} M$$

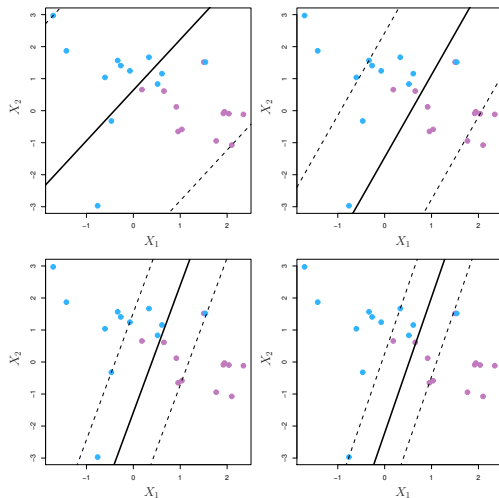
$$\text{subject to } \sum_{j=1}^p \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M(1 - \epsilon_i),$$

$$\epsilon_i \geq 0, \sum_{i=1}^n \epsilon_i \leq b \quad \text{for all } i = 1, \dots, n.$$

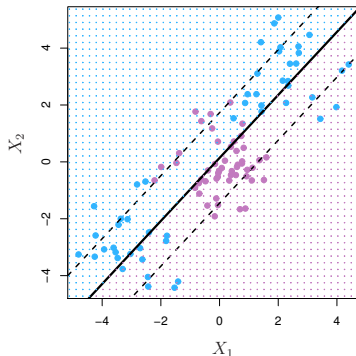
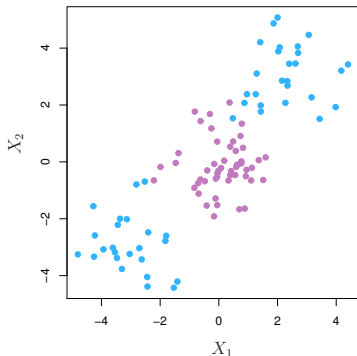
- ▶ The “budget” ϵ_i tells us for each observation x_i by **how much it violates the margin**: it is on the correct side of the margin if $\epsilon_i = 0$; on the wrong side of the margin if $\epsilon_i > 0$; on the wrong side of the hyperplane if $\epsilon_i > 1$.
- ▶ The constant $b > 0$ is a **tuning/regularization** parameter, the total budget for the amount that the margin can be violated by the n observations.
- ▶ Here, the **support vectors** are those who are on the wrong side of the margin for their class ($\epsilon_i > 0$) or lie directly on the margin, and only they affect the classifier!
- ▶ Thus, b controls the number of support vectors, and therefore the **bias-variance trade-off** of the support vector classifier.

Support vector classifier: the tuning parameter b



- ▶ Large b gives **wide margins**, and **high bias/small variance** for the SV classifier
- ▶ Small b gives **tight margins**, and **small bias/high variance** for the SV classifier.
- ▶ The optimal b can be found by **cross-validation**.

Support vector classifier: limitations



- ▶ A **linear decision boundary** can fail if the true boundary between the classes is not linear in the feature space of $(X_1, \dots, X_p)$! Any linear classifier will perform poorly here.
- ▶ One might want to **enlarge the feature space**, say with order-two polynomials $X_1, X_1^2, \dots, X_p, X_p^2$, and then use a SV classifier with the $2p$ features.
- ▶ There are many ways to enlarge the space, and the huge number of features will make the **computations infeasible**.

Chapter 20: Support Vector Machines

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
- 20. Support Vector Machines**
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Support vector machines

- ▶ **Support vector machines** (SVM) enlarge the feature space in an automatic way.
- ▶ A **deep mathematical result** is that the optimal support vector classifier (the solution to the optimization problem) can be written as the linear function

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i y_i \langle x, x_i \rangle,$$

where there is one parameter α_i per training observation, and the **inner product** is defined as $\langle x, z \rangle = \sum_{j=1}^p x_j z_j$.

- ▶ In fact, all α_i are zero except those for the support vectors.
- ▶ You can see an inner product as a **similarity measure**; here it is the **correlation**!
- ▶ The SVM simply replaces the usual inner product by a more general similarity measure, **called a kernel**.

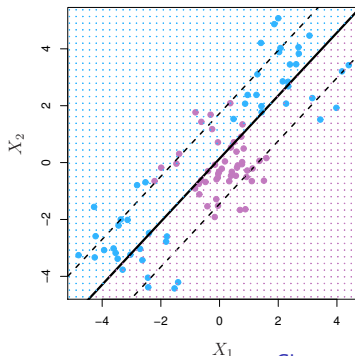
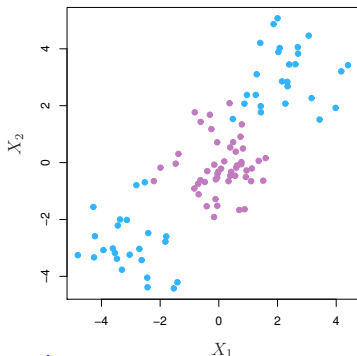
The kernel trick

- ▶ The SVM classifies the classes according to the, in general, **non-linear** function

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i y_i K(x, x_i),$$

where $K(x, z)$ is a (positive definite) **kernel** that measure similarity between $x, z \in \mathbb{R}^p$.

- ▶ By doing this, we **implicitly enlarge the feature space**, without actually doing it explicitly! This is called the **kernel trick**.
- ▶ With the **linear kernel** $K(x, z) = \langle x, z \rangle$ we recover the SV classifier.

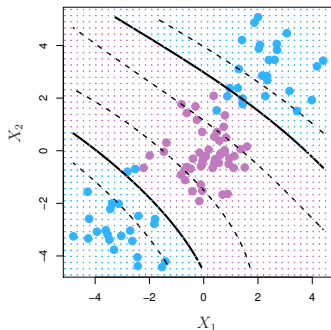


The kernel trick: examples

The polynomial kernel

$$K(x, z) = \left(1 + \gamma \sum_{j=1}^p x_j z_j \right)^d$$

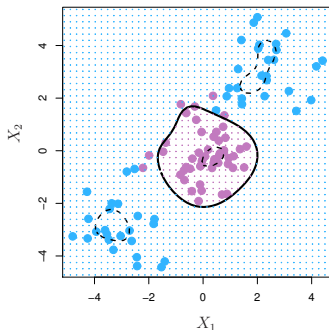
- ▶ Corresponds to fitting a SV classifier in a **higher dimensional space** with all polynomials of degree d !
- ▶ The tuning parameter γ controls the scale.



The radial/Gauss kernel

$$K(x, z) = \exp \left(-\gamma \sum_{j=1}^p (x_j - z_j)^2 \right)$$

- ▶ Corresponds to fitting a SV classifier in an **infinite dimensional space**!
- ▶ The tuning parameter γ controls how “local” the fit is.

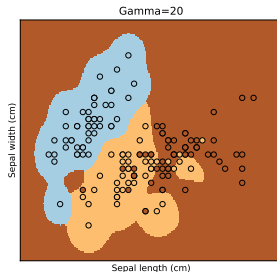
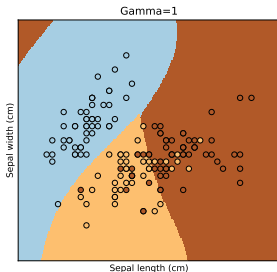
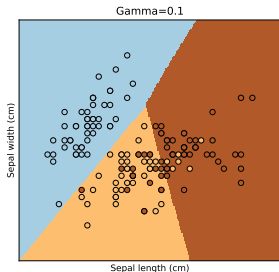


Support vector machines: comments

- ▶ SVMs are a very **powerful and flexible tool** for classification.
- ▶ The mathematics are quite hard, but thanks to the **kernel trick** we do not need to know the details!
- ▶ For fitting, we **only compute** all similarities $K(x_i, x_j)$ for all $\binom{n}{2}$ distinct pairs $i, j = 1, \dots, n$ in the training set.
- ▶ Many different kernels have been developed, but even the “standard” ones **perform usually well**.
- ▶ For **more than two classes**, there are simple extensions, so-called **One-Versus-One** classification, or **One-Versus-All** classification.
- ▶ For more details see Chapter 9 in **[ISL]**.

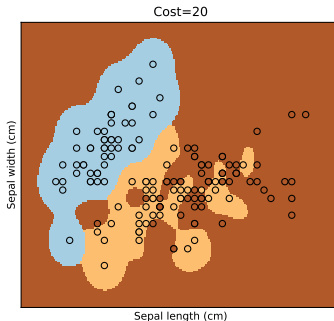
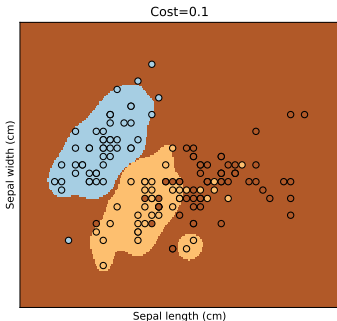
Support vector machines in python

```
from sklearn.svm import SVC
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target # we only keep the first two features: 'Sepal length', 'Sepal width'
fig, axs = plt.subplots(1,3,figsize=(6*3,5))
gammas = [0.1,1,20]
for i, gamma in enumerate(gammas):
    svm = SVC(C=1, kernel='rbf', gamma=gamma)
    svm.fit(X, y)
    x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    h = .02 # step size in the mesh
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    axs[i].pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)
    axs[i].scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired)
plt.show()
```



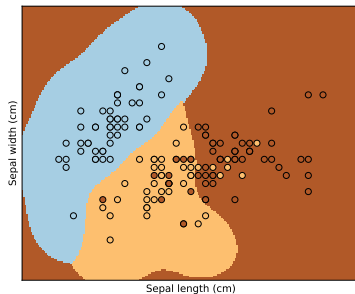
Support vector machines in python

```
from sklearn.svm import SVC
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target # we only keep the first two features: 'Sepal length', 'Sepal width'
fig, axs = plt.subplots(1,3,figsize=(6*3,5))
costs = [0.1,20]
for i, cost in enumerate(costs):
    svm = SVC(C=cost, kernel='rbf', gamma=20)
    svm.fit(X, y)
    x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    h = .02 # step size in the mesh
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    axs[i].pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)
    axs[i].scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired)
plt.show()
```

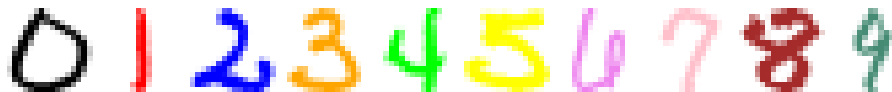


Support vector machines in python

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, KFold
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target # we only keep the first two features: 'Sepal length', 'Sepal width'
hyper_parameters = {"C": [0.5, 0.8, 1, 1.2, 1.5], "gamma": [0.1, 1, 3, 5, 7, 10, 15]}
folds = KFold(n_splits=5, shuffle=True, random_state=1)
svmCV = GridSearchCV(estimator=SVC(kernel='rbf'), param_grid=hyper_parameters, cv=folds)
svmCV.fit(X, y)
print(svmCV.best_params_)# Out: {'C': 1.2, 'gamma': 5}
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
h = .02 # step size in the mesh
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
Z = svmCV.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Paired);
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired);
```



Digit classification with SVMs



- ▶ We can update our results on the **digit classification**.
- ▶ We use SVMs (One-Versus-One) with **linear, polynomial and radial kernels**.
- ▶ The **tuning parameters** the value of C were found by 10-fold-CV.

- ▶ linreg: direct linear regression;
- ▶ LDA1: LDA based on the values x_i ;
- ▶ LDA2: LDA based on x_i and x_i^2 ;
- ▶ QDA: different cov. matrix for each class;
- ▶ log: logistic regression (multinomial).
- ▶ log-ridge: logistic regression with ridge.
- ▶ log-lasso: logistic regression with lasso.
- ▶ RF: random forest.
- ▶ linear SVM: linear support vector classifier.
- ▶ radial SVM: SVM with radial kernel.

	Training error	Test error
linreg	7.6%	13.1%
LDA1	6.2%	11.5%
LDA2	3.9%	10.2%
QDA	1.8%	13.5%
log	0.01%	11.1%
log-ridge	4.1%	9.0%
log-lasso	2.7%	8.8%
RF	0%	5.9%
linear SVM	1.5%	6.5%
radial SVM	0.6%	5.4%

Chapter 21: Deep learning: Single hidden layer neural networks

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
- 21. Deep learning: Single hidden layer neural networks**
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Neural networks

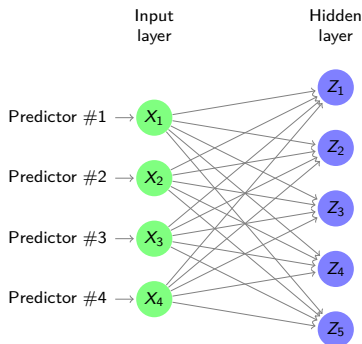
- ▶ In recent years, the term (artificial) neural networks has caused a big hype in machine learning, especially in connection with deep learning.
- ▶ In the media, this method appears to be magical or mysterious and is often believed as being the solution to all prediction problems.
- ▶ We will de-mystify this term and understand a (deep) neural network as what it is: a highly flexible, non-linear prediction method.
- ▶ Neural networks can be used for both regression and classification.

Single hidden layer neural network for regression

- ▶ In its simplest form, a **neural network** for predicting a quantitative Y is constructed as follows.
- ▶ The p inputs $X = (X_1, \dots, X_p)$ are **non-linearly** combined to form m **hidden** variables by

$$Z_j = g(b_j^{(1)} + w_j^{(1)\top} X), \quad j = 1, \dots, m$$

where $w_j^{(1)} = (w_{j1}^{(1)}, \dots, w_{jp}^{(1)}) \in \mathbb{R}^p$ and $b_j^{(1)} \in \mathbb{R}$ are called the **weights** and the **bias**, respectively, and the **activation function** g can introduce the **non-linearity**.



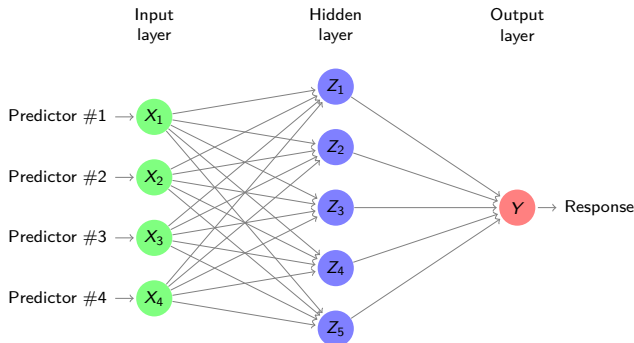
Single hidden layer neural network for regression

- The $Z = (Z_1, \dots, Z_m)$ are then further combined to **model the output** Y as

$$Z_j = g(b_j^{(1)} + w_j^{(1)\top} X), \quad j = 1, \dots, m,$$

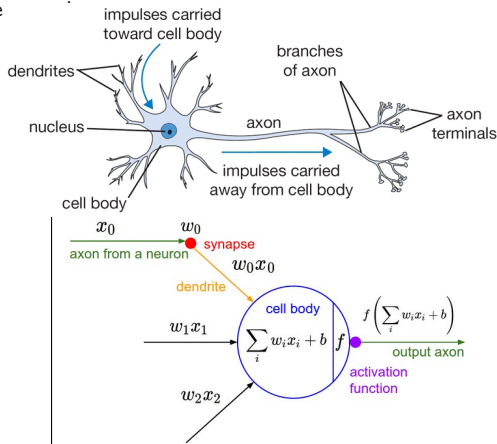
$$Y \approx \hat{f}(X) = h(b^{(2)} + w^{(2)\top} Z).$$

with weights $w^{(2)} = (w_1^{(2)}, \dots, w_m^{(2)}) \in \mathbb{R}^m$ and bias $b^{(2)} \in \mathbb{R}$, and some function h .



Biological motivation and notation

- ▶ **Artificial neural network** are loosely motivated by the functioning of **neurons** in our brain.
- ▶ The **nomenclature** in this area of machine learning is therefore linked to **biological terms**.
- ▶ The nodes in the network are called **neurons**, they can be **activated** and they **fire** if the activation is large



Single hidden layer neural network for regression

- In matrix notation, we can write the **hidden layer vector** as

$$Z = g(b^{(1)} + \mathbf{W}^{(1)}X) \in \mathbb{R}^m, \quad \mathbf{W}^{(1)} = \begin{bmatrix} w_1^{(1)} \\ \vdots \\ w_m^{(1)} \end{bmatrix} \in \mathbb{R}^{m \times p}, \quad b^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_m^{(1)} \end{bmatrix} \in \mathbb{R}^m,$$

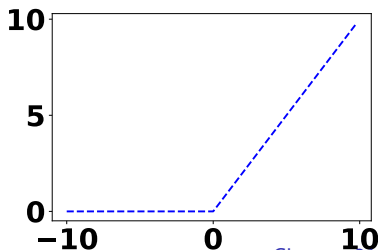
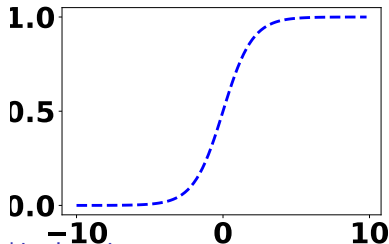
where the function g is applied to each component of the argument vector.

- Traditionally, the **activation function** g was chosen to be the **sigmoid function**

$$\sigma(x) = \exp(x)/(1 + \exp(x)).$$

- In modern neural networks, the default recommendation for g is the **rectified linear unit** or **ReLU** given by

$$\text{ReLU}(x) = \max\{0, x\}.$$



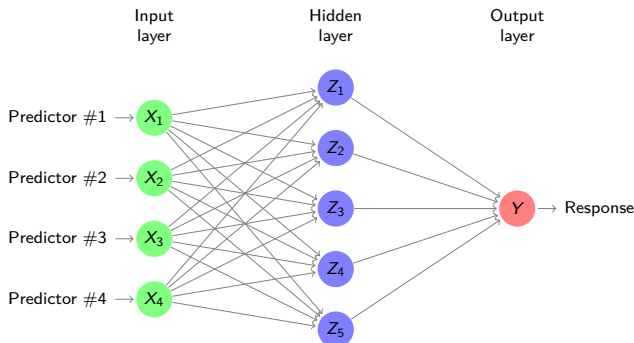
Single hidden layer neural network for regression

- ▶ In regression, the function h is typically the **identity** $h(z) = z$, that is,

$$\hat{f}(X) = b^{(2)} + w^{(2)\top} Z.$$

- ▶ Together with the **ReLU activation** $Z = \max\{0, b^{(1)} + \mathbf{W}^{(1)}X\}$, the **single hidden layer neural network** models the output Y in a non-linear way by

$$\hat{f}(X) = b^{(2)} + w^{(2)\top} \max\{0, b^{(1)} + \mathbf{W}^{(1)}X\}$$

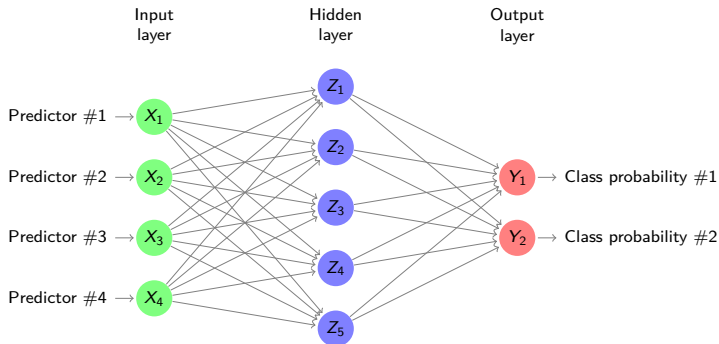


Single hidden layer neural network for classification

- ▶ If the output is **qualitative** with q classes $Y \in \mathcal{G} = \{1, \dots, q\}$ then there are q **output nodes**, each standing for the probability $Y_i = f_i(X) = \Pr(Y = i \mid X)$, $i = 1, \dots, q$.
- ▶ The neural network is then

$$Z = g(b^{(1)} + \mathbf{W}^{(1)}X)$$
$$\hat{f}_i(X) = h_i(b^{(2)} + \mathbf{W}^{(2)}Z), \quad i = 1, \dots, q.$$

with **weight matrix** $\mathbf{W}^{(2)} = (w_{ij}^{(2)}) \in \mathbb{R}^{q \times m}$ and bias vector $b^{(2)} = (b_1^{(2)}, \dots, b_q^{(2)}) \in \mathbb{R}^q$, and some functions h_i .



Single hidden layer neural network for classification

- ▶ As for regression, the default choice for the **activation** is the **ReLU** $g(x) = \max\{0, x\}$.
- ▶ For classification, the functions h_i are the **softmax** functions

$$h_i(x_1, \dots, x_q) = \text{softmax}(x_1, \dots, x_q)_i = \frac{\exp(x_i)}{\sum_{j=1}^q \exp(x_j)}, \quad i = 1, \dots, q.$$

- ▶ Note that this is the same function as used in **multinomial logistic regression**.
- ▶ With these choices the **single hidden layer neural network** for classification models the **posterior probabilities** as

$$\hat{f}_i(X) = \Pr(Y = i \mid X) = \text{softmax}(b^{(2)} + \mathbf{W}^{(2)} \max\{0, b^{(1)} + \mathbf{W}^{(1)}X\})_i.$$

- ▶ For a new input $x_0 \in \mathbb{R}^p$ we then use the **Bayes classifier** to predict the class

$$\hat{G}(x_0) = \hat{y}_0 = \arg \max_{i=1, \dots, q} \hat{f}_i(x_0).$$

Chapter 22: Deep learning: Deep neural networks

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
- 22. Deep learning: Deep neural networks**
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Deep neural networks

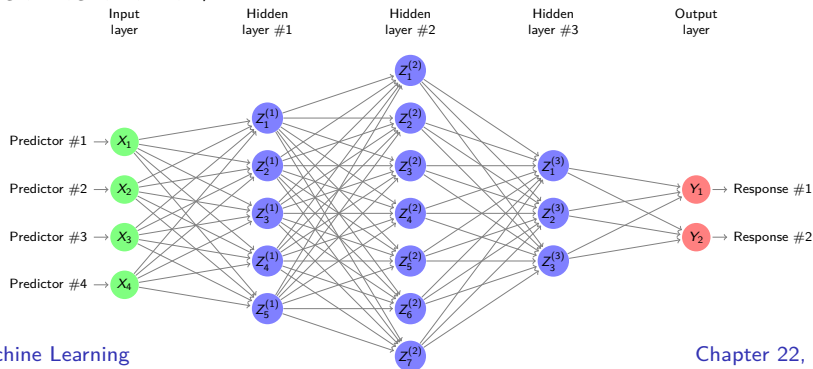
- ▶ A **deep neural network** is a neural network with **multiple hidden layers** between the input and the output layers.
- ▶ The predictive model is then defined **sequentially**

$$Z^{(1)} = g_1(b^{(1)} + \mathbf{W}^{(1)}X),$$

$$Z^{(2)} = g_2(b^{(2)} + \mathbf{W}^{(2)}Z^{(1)}), \dots, Z^{(M)} = g_M(b^{(M)} + \mathbf{W}^{(M)}Z^{(M-1)})$$

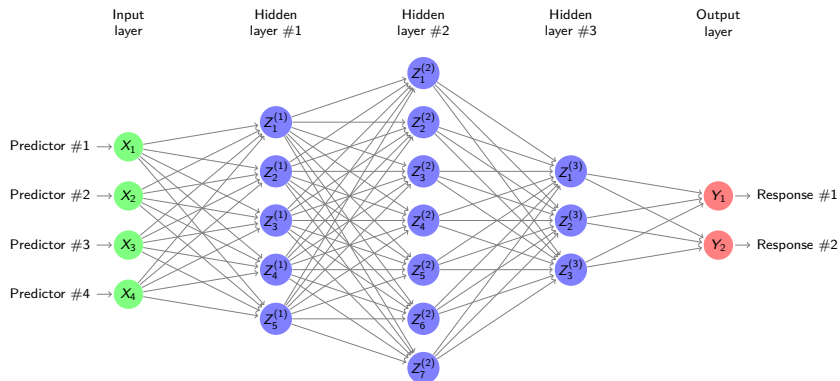
$$\hat{f}_i(X) = h_i(b^{(M+1)} + \mathbf{W}^{(M+1)}Z^{(M)}), \quad i = 1, \dots, q,$$

with weight matrices $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(M+1)}$, bias vectors $b^{(1)}, \dots, b^{(M+1)}$, and functions $g_1, \dots, g_M, h_1, \dots, h_q$.



The architecture of deep neural networks

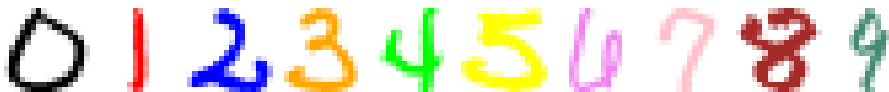
- ▶ The **depth** of the neural network is the total number of layers.
- ▶ For each layer we call the number of neurons the **width** of that layer.
- ▶ The **architecture** of the network is mainly determined by the depth and width.
- ▶ There can be further architectural considerations concerning the connections between neurons or **shared parameters**; see **convolutional neural networks**.
- ▶ A good architecture is based on **experience** and **cross-validation**.



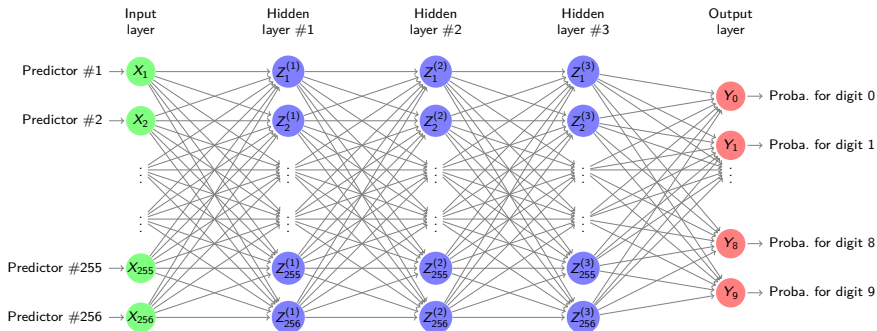
Comments on deep neural networks

- ▶ As before the default choice for all **activation** functions is the **ReLU**
 $g_j(x) = \max\{0, x\}, j = 1, \dots, M.$
- ▶ For regression the number of outputs is $q = 1$ and the function $h = h_1$ is the identity
 $h(z) = z.$
- ▶ For classification, the functions h_i are the **softmax** functions.
- ▶ The **hidden layers** of the network **extract patterns** or features of the data.
- ▶ Especially the first layer that is **directly linked** to the inputs allows for **interpretation** and **visualization** of the extracted features; more on this for image data later.
- ▶ Deep neural networks are **very flexible**. Indeed, if the number of neurons is large enough, they can approximate any continuous function on $\mathbb{R}^p$ **arbitrarily well** (under mild assumptions).
- ▶ Deep neural networks have **many (!) parameters**, meaning that fitting them is computationally **very costly** and **overfitting** must actively be prevented.

A simple example: digit classification



- ▶ For the digits data set, the number of predictors is $p = 256$.
- ▶ Consider a **fully connected** network with $M = 3$ hidden layers, each having 256 neurons.
- ▶ This “simple network” already has $3 \times 257 \times 256 + 257 \times 10 \approx 200k$ parameters!



Chapter 23: Deep learning: Fitting neural networks and back-propagation

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
- 23. Deep learning: Fitting neural networks and back-propagation**
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Loss functions for neural networks

- ▶ Neural networks are non-linear **parametric models**, with parameter vector $\theta = (\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(M+1)}, b^{(1)}, \dots, b^{(M+1)})$ consisting of the **weight matrices** and the **bias vectors**.
- ▶ For regression, we let $\hat{f} = \hat{f}_\theta$ denote the predictive model from the neural network based on the parameter set θ . Similarly, for classification with $\hat{f}_1, \dots, \hat{f}_q$.
- ▶ Suppose now we have **training data** $(x_1, y_1), \dots, (x_n, y_n)$.
- ▶ For regression ($y_i \in \mathbb{R}$) we use the **squared-error loss**

$$J(\theta) = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2.$$

- ▶ For classification ($y_i \in \{1, \dots, q\}$) we use **cross-entropy (deviance) loss**

$$J(\theta) = - \sum_{i=1}^n \sum_{j=1}^q \mathbf{1}_{\{y_i = j\}} \log \hat{f}_j(x_i),$$

which is the **negative log-likelihood** of the **multinomial distribution**.

Fitting of neural networks

- ▶ **Fitting neural networks** is an art (and an area of research) for itself.
- ▶ The problem of **minimizing** $J(\theta)$ does not have a closed form solution and is **non-convex**.
- ▶ Since we have so many parameters, we are typically not interested in the **global/local minimum** of $J(\theta)$ since this would be an **overfit**.
- ▶ Instead we will prevent overfitting by **regularization** or **early stopping** (later more).
- ▶ For now, we first consider how we can improve the initial model using **gradient descent**.

Gradient descent for neural networks

- Start with an arbitrary initial value $\theta^{(0)}$.
- For $i = 1, 2, \dots$:
 - Compute the **gradient/derivative** of J at the position $\theta^{(i-1)}$:

$$\nabla_{\theta} J(\theta^{(i-1)}) = \left. \frac{\partial J(\theta)}{\partial \theta} \right|_{\theta=\theta^{(i-1)}}$$

- Update your estimate with **learning rate** $\alpha > 0$ to

$$\theta^{(i)} = \theta^{(i-1)} - \alpha \nabla_{\theta} J(\theta^{(i-1)}).$$

- Stop if a **stopping criterion** is satisfied.

Thus, all we need are the **gradients** $\nabla_{\theta} J(\theta)$!

Computing the gradient: back-propagation

- ▶ In the sequel we consider only **regression**; classification is similar.
- ▶ The gradient $\nabla J(\theta)$ at some parameter vector θ is the sum of the gradients over all training samples:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (\hat{f}_{\theta}(x_i) - y_i)^2.$$

- ▶ We can thus consider the summands separately, and for now we consider only the **loss contribution** of one training sample (x, y)

$$L(\theta) = (\hat{f}_{\theta}(x) - y)^2.$$

- ▶ **Back-propagation** is a way of efficiently deriving the gradient using the **chain-rule** for differentiation.
- ▶ Recall: for two functions f and g , let $h(x) = f(g(x))$, the chain-rule is

$$\left. \frac{\partial h(x)}{\partial x} \right|_{x=x_0} = \left. \frac{\partial f(z)}{\partial z} \right|_{z=g(x_0)} \cdot \left. \frac{\partial g(x)}{\partial x} \right|_{x=x_0},$$

or in short-hand notation with $z = g(x)$

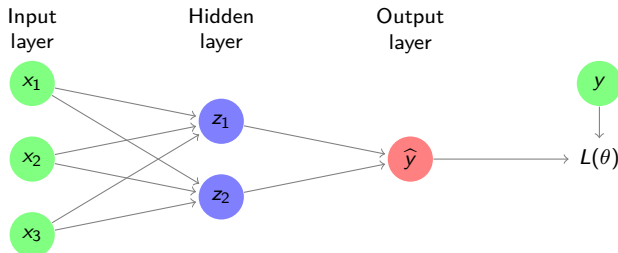
$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial x}$$

Back-propagation: the forward pass

- Fix the training sample $x = (x_1, \dots, x_p)$ and y , and also the parameter vector θ consisting of weights and biases where we want to compute the gradient of $L(\theta)$.
- In order to know all the values of all the hidden neurons, we first have to do a **forward pass**.
- For this, we start with the **activations** $(x_1, \dots, x_p)$ of the inputs $(X_1, \dots, X_p)$ and then successively compute the activations z_i of all neurons of the hidden layers and the predicted value

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_{\theta}(x) = b^{(2)} + w^{(2)\top}z.$$

- This allows us to compute **the error** for this θ on that training sample $L(\theta) = (\hat{y} - y)^2$.



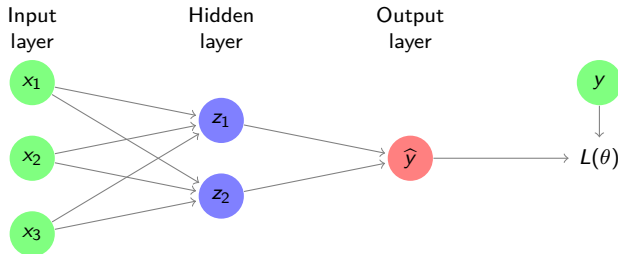
Back-propagation

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_{\theta}(x) = b^{(2)} + w^{(2)\top}z, \quad L(\theta) = (\hat{y} - y)^2.$$

- ▶ To obtain the gradient, we compute the **derivatives** of L with respect to **all parameters**.
- ▶ Let us start with the weight vector $w^{(2)} = (w_1^{(2)}, w_2^{(2)})$:

$$\frac{\partial L}{\partial w_1^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1^{(2)}} = 2(\hat{y} - y)z_1$$

$$\frac{\partial L}{\partial w_2^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2^{(2)}} = 2(\hat{y} - y)z_2$$



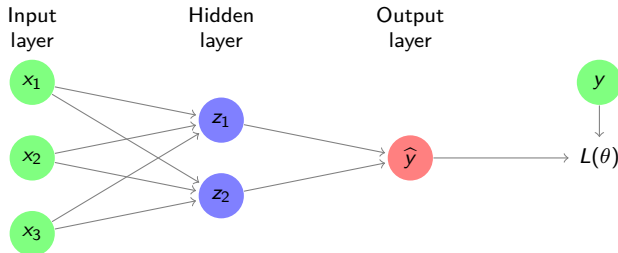
Back-propagation

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_{\theta}(x) = b^{(2)} + w^{(2)\top}z, \quad L(\theta) = (\hat{y} - y)^2.$$

► For the weight $w_{11}^{(1)}$:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}^{(1)}} = 2(\hat{y} - y) \cdot w_1^{(2)} \cdot g'(b_1^{(1)} + w_1^{(1)\top}x)x_1$$

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_i} \cdot \frac{\partial z_i}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top}x)x_j$$



Back-propagation

- Now write

$$\frac{\partial L}{\partial w_i^{(2)}} = 2(\hat{y} - y)z_i = \delta_i^{(2)}z_i,$$

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x)x_j = \delta_i^{(1)}x_j$$

- The quantities $\delta_i^{(2)}$ and $\delta_i^{(1)}$ are called the **errors** from the current model corresponding to θ at the output and hidden layer neurons, respectively.
- They satisfy

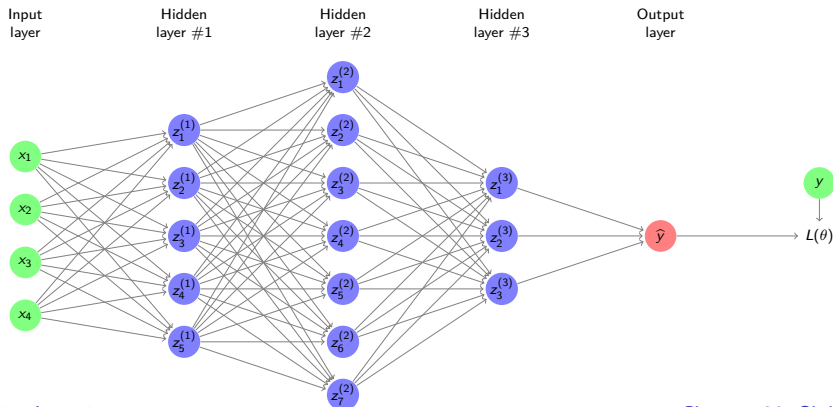
$$\delta_i^{(1)} = \delta_i^{(2)} \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x). \quad (1)$$

Note: for the **ReLU activation** $g(x) = \max\{0, x\}$ we have $g'(x) = \mathbf{1}\{x > 0\}$.

- Back-propagation is implemented with a **two-pass algorithm**:
 - 1) Compute all z_i and $\hat{y}$ in the **forward pass**.
 - 2) In the **backward-pass**, the error $\delta_i^{(2)}$ are computed and then **back-propagated** via (1) to obtain the errors $\delta_i^{(1)}$, and then the partial derivatives required for the gradient.
- The equations for the derivatives with respect to the biases are similar.

Advantages of back-propagation

- ▶ Back-propagation for **deep networks** works similarly, where we have error vectors $\delta^{(1)}, \dots, \delta^{(M+1)}$ at all M hidden layers and the output layer.
- ▶ Back-propagation is very **efficient** since the quantities $\delta^{(m)}$ can be stored and re-used, especially if the network is large.
- ▶ Only the **local derivatives** at each node are required.
- ▶ It can be efficiently **parallelized** on GPUs.



Batch optimization

- ▶ Recall that the gradient required for **gradient descent** is the sum of the gradients over all training samples:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (y_i - \hat{f}_{\theta}(x_i))^2. \quad (1)$$

- ▶ For **each training sample** (x_i, y_i) we can compute the gradient via **back-propagation**.
- ▶ In order to improve our error function $J(\theta)$ we have two possibilities.

1. Batch optimization and gradient descent

- ▶ Use the **batch** of all training samples $(x_1, y_1), \dots, (x_n, y_n)$ to compute the gradient in (1).
- ▶ This amounts to standard gradient descent and is called **batch-optimization**.

Stochastic gradient descent

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (y_i - \hat{f}_{\theta}(x_i))^2. \quad (1)$$

2. Minibatch optimization and stochastic gradient descent

- ▶ Modern problems often have **billions** of training samples $n \approx 10^9$, making a **single gradient step** prohibitively long.
- ▶ Randomly divide the training data in n/m **minibatches** of size m .
- ▶ The minibatch size m is usually **much smaller** than n , ranging from 1 to a few hundred.
- ▶ In each step, **stochastic gradient descent** uses only $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$ from a minibatch to approximate the gradient (1) for updating θ by

$$\nabla_{\theta} J(\theta) \approx \sum_{i=1}^m \nabla_{\theta} (y^{(i)} - \hat{f}_{\theta}(x^{(i)}))^2.$$

- ▶ Minibatch optimization takes **many small steps** that are possibly not optimal but **very fast** to compute.
- ▶ Batch optimization takes **few optimal steps** that are **much slower** and possibly infeasible for large data sets.
- ▶ Often, training samples are **similar** or **redundant** (e.g., similar images), then using minibatches discards less information.
- ▶ A **training epoch** is one iteration over the **entire training set**.

Chapter 24: Deep learning: Regularization for neural networks

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
- 24. Deep learning: Regularization for neural networks**
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

Parameter initialization

- ▶ Fitting of deep neural networks is a **difficult task**, and the underlying theory is not well-understood.
- ▶ Therefore, most rules and recommendations are **heuristic** and it requires a good amount of **engineering** to obtain good solutions.
- ▶ It is usually **advisable to standardize** the inputs to have mean zero and standard deviation one, so that all inputs are **treated equally**, especially in **regularization**.

Initial parameters

- ▶ Since the function $J(\theta)$ is non-convex, the **starting values** $\theta^{(0)}$ can have a strong influence on the final solution.
- ▶ The **weights** and **biases** are usually initialized as **small random values** of a **normal or uniform distribution**.
- ▶ Larger starting weights help to **avoid losing signal** during forward and back-propagation.
- ▶ Too large weights, however, result in **exploding values** during forward propagation.

Regularization

- ▶ Typically, a deep neural network has **too many parameters** and will **overfit** the data at the global minimum of $J(\theta)$.
- ▶ In order to make the network **more robust** and to improve the **generalization performance** there are several possibilities to do **regularization**.

Parameter penalties

- ▶ Similarly to **lasso** and **ridge regression**, we introduce an extra term in the objective function that **penalizes** the size of the parameters:

$$J_{\text{reg}}(\theta) = J(\theta) + \alpha\Omega(\theta),$$

where $\alpha > 0$ is a tuning parameter and $\Omega(\theta)$ is a norm penalty term, e.g.,

$$\Omega(\theta) = \sum_{i,j,m} w_{ij}^{(m)2},$$

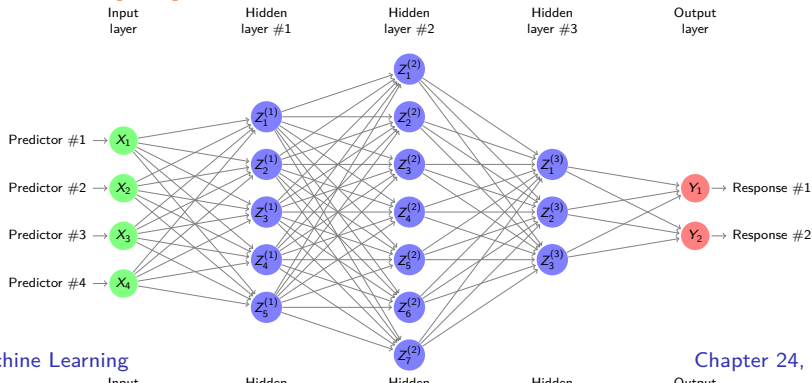
or some other norm such as ℓ_1 .

- ▶ The **interpretation** is similar as for regularized linear models, and the tuning parameter is also chosen by **cross-validation**.

Regularization

Dropout

- ▶ **Dropout** is a regularization method where in each update step in the gradient descent, the **network structure** is changed.
- ▶ In fact, before a forward pass and back-propagation, each **non-output neuron** is **independently** dropped out with some probability $q \in [0, 1]$.
- ▶ For **hidden units** this probability is usually $q = 0.5$, and for **input units** $q = 0.2$.
- ▶ This **robustifies** the fitting, since every unit must learn to work independently from the dropped units.
- ▶ Dropout is a type of **bagging**, where the final model is a **combination** of the fitted weights and the **dropout probabilities**.



Regularization

Early stopping

- ▶ Probably the most commonly used method for regularization is **early stopping** in the gradient descent algorithm.
- ▶ For large networks with enough **capacity to overfit** the training error decreases steadily over time, but the **validation error** begins to raise at some point.
- ▶ Early stopping means to return the parameter value $\theta^{(i^*)}$ at step i^* where a **minimum** of the **validation error** is reached, and then **stop the descent**.
- ▶ This means we do not go all the way to the (local) minimum of $J(\theta)$, but stop at a parameter set that gives the **best validation error**.
- ▶ In some cases, early stopping can be shown to be **equivalent** to ℓ_2 regularization.

Parameter sharing

- ▶ Due to invariances or other domain knowledge, often it makes sense to **force** sets of parameters to have the **same value**.
- ▶ This can drastically **reduce** the number of unique parameters and thus performs regularization.
- ▶ The most popular use of this **parameter sharing** occurs in **convolutional neural networks** applied to image recognition.

Regularization for deep neural networks is an active **domain of research**; the whole Chapter 7 in the book *Deep Learning* by Goodfellow et al. (2016) is devoted to this topic.

Chapter 25: Deep learning: Convolutional neural networks

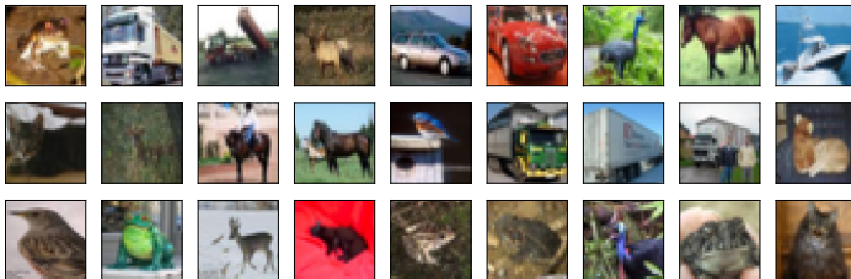
1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
- 25. Deep learning: Convolutional neural networks**
26. Deep learning: Further topics

Image recognition: CIFAR-10

- ▶ A benchmark data set for **image recognition** is **CIFAR-10**:
<https://www.cs.toronto.edu/~kriz/cifar.html>
- ▶ It consists of 60000 32×32 RGB colour images with 10 different classes

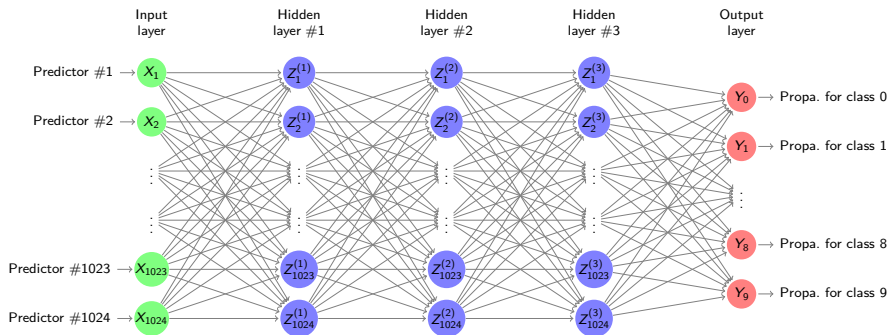
$$\mathcal{G} = \{\text{airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck}\}$$

- ▶ Each image has **three channels** (red, green, blue), each consisting of the intensities of that color in each pixel of that image.
- ▶ The predictor space has thus **dimension** $p = 32 \times 32 \times 3 = 1024$.
- ▶ State-of-the-art methods have prediction accuracy above 95%:
<https://www.kaggle.com/c/cifar-10/leaderboard>



A dense neural network

- ▶ For the **CIFAR-10** data set, the number of predictors is $p = 1024$.
- ▶ A **dense neural network** is one where **all neurons** are connected with **individual weights**.
- ▶ For this data set, **each layer** in a dense network has about **1 million parameters**!
- ▶ Fitting so many parameters is **computationally very expensive**.
- ▶ Moreover, with only 60000 training samples, heavy **overfitting will occur**.



Spatial structures in image data

- ▶ We have already seen different methods of regularization that aim at avoiding overfitting.
- ▶ If **domain knowledge** is available, a very effective way of regularization is to **decrease** the number of parameters in the network **architecture**.
- ▶ This can be done by **parameter sharing**.
- ▶ For image data, there is a natural **spatial structure** underlying the predictors: two **neighboring pixels** are usually more **similar** than pixels far apart.
- ▶ **Convolutional neural networks** incorporate this intuition into the network architecture.



Convolutional neural networks

- ▶ Convolutional neural networks are **deep neural networks** with many hidden layers.
- ▶ The hidden layers can be of **different types**, meaning that different operations or parameter sharings are used.
- ▶ We already know one type of hidden layer, namely the **dense/fully connected layer** whose neurons are **connected to all neurons** of the previous layer (either input or hidden).
- ▶ There are **many other layer types** and we will study some of the most important.
- ▶ A **convolutional layer** tries to **extract local features** of images in three stages:
 - a) the convolution stage;
 - b) the activation stage;
 - c) the pooling stage.

The convolutional stage

- ▶ The **first stage** is the **convolution operation** that linearly combines the inputs of its **receptive field** (e.g., 3×3 pixels) to a single score.
- ▶ The **weights** for this linear combination are called a **filter**, and different filters extract different features (horizontal lines, vertical lines, bright spots, etc.).
- ▶ The receptive field **slides** through the whole image and thus producing a whole grid of neurons as output.
- ▶ Below the input image is called I , the filter is K and the **convoluted image** $I * K$ is the output of this layer.
- ▶ The filter can be fixed, but usually its weights are **learned** during the fitting.
- ▶ The only parameters of this layer are the **weights of the filter** and a bias.

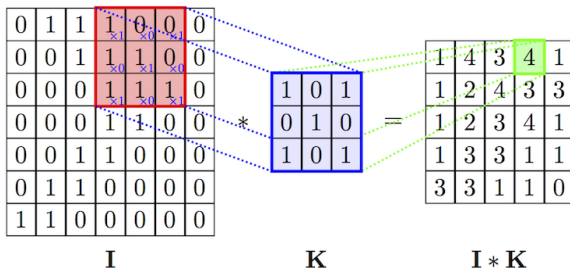


Image from <https://cambridgespark.com/content/tutorials/convolutional-neural-networks-with-keras/index.html>

The convolution stage

- ▶ Convolutional layers have several **hyper parameters**, e.g., the **width** and the **height** of the filter.
- ▶ There is also the **depth** of the layer, that is, the number of **different filters** applied in the same step.
- ▶ The **stride** specifies by how much we **shift the filter** to obtain the next pixel of the output.
- ▶ The output image may have **smaller size** than the input image.
- ▶ **Zero-padding** (also called “same” padding) might be used to **keep same sizes** of input and output; in this case the image is sufficiently padded with **zeros** at the edges.

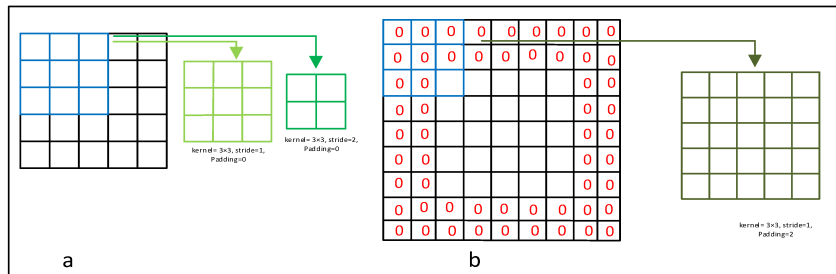


Image from <http://www.mdpi.com/2078-2489/9/1/19>

The detector and pooling stages

- ▶ The **second** stage of the convolutional layer is the **detector stage** where a non-linearity is applied to each neuron, usually the **ReLU**.
- ▶ A neuron is **active** and the feature is **present** in that part of the image only if the linear score of the filter is high enough.
- ▶ This stage does not have any parameters when the non-linearity is fixed.
- ▶ The third stage is **pooling** which is a form of **downsampling**.
- ▶ Typically, **max-pooling** is applied to aggregate chunks of the image into a single value by taking the **maximum activation** of the convolution output in that chunk (often 2×2).

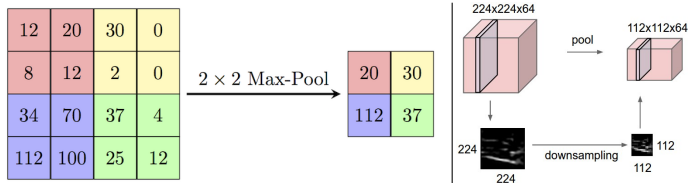


Image from <https://cambridgespark.com/content/tutorials/convolutional-neural-networks-with-keras/index.html>

The convolutional layer

- ▶ The **convolutional layer** are the three stages together

convolution operation $\rightarrow$ ReLU $\rightarrow$ pooling,

(sometimes only the convolutional operation is called a convolutional layer).

- ▶ There may be **several of these layers** in a deep neural network, sometimes with several convolutions plus ReLU operations before pooling.

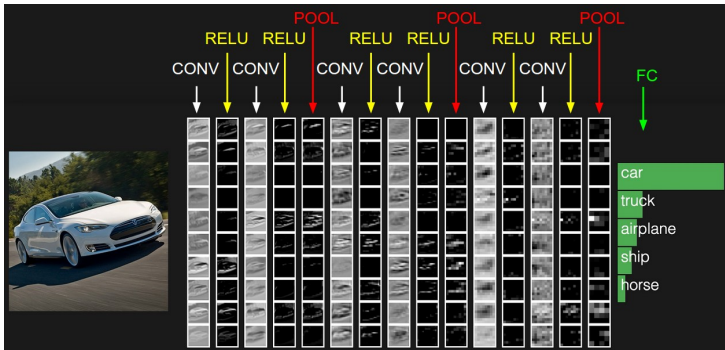


Image from <http://cs231n.github.io/convolutional-networks/>

Convolutional layers



Filter 1

-1	-1	1	1
-1	-1	1	1
-1	-1	1	1
-1	-1	1	1

Filter 2

1	1	-1	-1
1	1	-1	-1
1	1	-1	-1
1	1	-1	-1

Filter 3

-1	-1	-1	-1
-1	-1	-1	-1
1	1	1	1
1	1	1	1

Filter 4

1	1	1	1
1	1	1	1
-1	-1	-1	-1
-1	-1	-1	-1

Activation Map for Filter 1



Activation Map for Filter 2



Activation Map for Filter 3



Activation Map for Filter 4



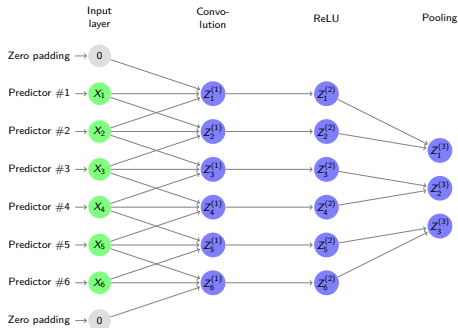
Convolutional layer

- Mathematically a **convolutional layer** is defined as

$$Z_j^{(1)} = b_j^{(1)} + w_j^{(1)\top} X, \quad j = 1, \dots, m_1, \quad Z_j^{(2)} = \max\{0, Z_j^{(1)}\}, \quad j = 1, \dots, m_1$$
$$Z_j^{(3)} = \text{pool}(Z^{(2)})_j, \quad j = 1, \dots, m_3$$

with equal biases $b_1^{(1)} = \dots = b_m^{(1)}$ and weight vectors $w_1^{(1)}, \dots, w_m^{(1)}$ having zeros except for close pixels (defined by the **filter**), and moreover the **weights are shared**, that is, equal in all m vectors, but shifted to match the neighbors.

- The **number of parameters** are therefore only **filter height $\times$ filter width + 1**, in the example below $3 \times 1 + 1 = 4$.



A typical CNN

- ▶ Often more than one filter is used in one **convolutional layer**.
- ▶ The **number of parameters** is $(\text{filter height} \times \text{filter width} + 1) \times \text{filter depth}$, where **filter depth** is the number of filters.
- ▶ In the first convolutional layer below $(5 \times 5 + 1) \times 10 = 260$.
- ▶ Note: A **flattening layer** simply transforms a 2-dimensional layer into a 1-dimensional vector in order to use a **fully connected layer**.

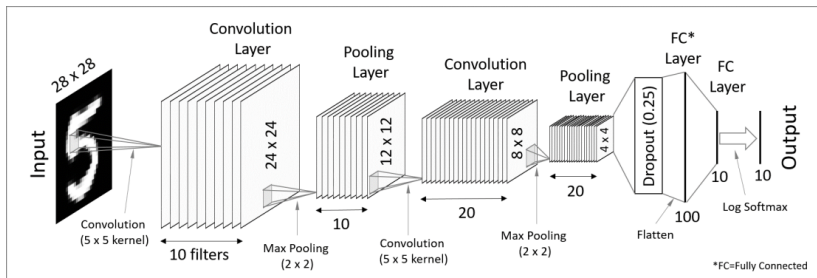
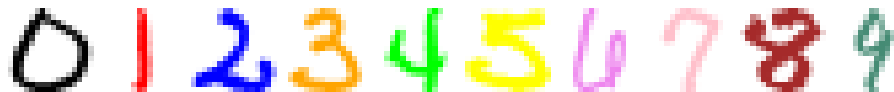


Image from

<https://codetolight.wordpress.com/2017/11/29/getting-started-with-pytorch-for-deep-learning-part-3-neural-network-basics/>

Digit classification with neural networks



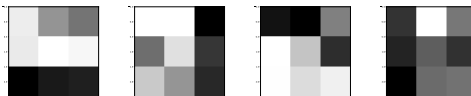
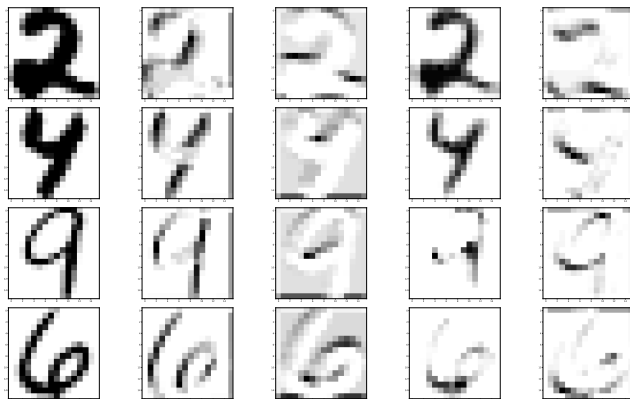
► We fit a **fully connected** and a **convolutional neural network**.

- linreg: direct linear regression;
- LDA1: LDA based on the values x_i ;
- LDA2: LDA based on x_i and x_i^2 ;
- QDA: different cov. matrix for each class;
- log: logistic regression (multinomial).
- log-ridge: logistic regression with ridge.
- log-lasso: logistic regression with lasso.
- RF: random forest.
- linear SVM: linear support vector classifier.
- radial SVM: SVM with radial kernel.
- FC NN: fully connected neural network.
- CNN: convolutional neural network.

	Training error	Test error
linreg	7.6%	13.1%
LDA1	6.2%	11.5%
LDA2	3.9%	10.2%
QDA	1.8%	13.5%
log	0.01%	11.1%
log-ridge	4.1%	9.0%
log-lasso	2.7%	8.8%
RF	0%	5.9%
linear SVM	1.5%	6.5%
radial SVM	0.6%	5.4%
FC NN	~ 0%	5.8%
CNN	~ 0%	3.2%

Activations and filters of first layer

- The first layer extract **generic features** of the image.

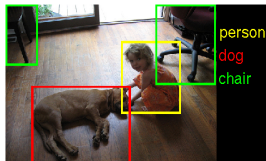
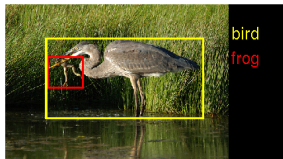


Chapter 26: Deep learning: Further topics

1. Course modalities and motivation
2. What is machine learning?
3. Regression framework
4. Linear Regression
5. Gradient Descent
6. Training versus test error
7. Cross-validation
8. Linear model selection and regularization
9. Classification framework
10. Linear and quadratic discriminant analysis
11. Logistic Regression
12. Image recognition
13. Decision trees: regression
14. Decision trees: classification
15. Bagging
16. Random forest
17. Boosting
18. Support Vector Machines: Maximal Margin Classifier
19. Support Vector Machines: Support Vector Classifier
20. Support Vector Machines
21. Deep learning: Single hidden layer neural networks
22. Deep learning: Deep neural networks
23. Deep learning: Fitting neural networks and back-propagation
24. Deep learning: Regularization for neural networks
25. Deep learning: Convolutional neural networks
26. Deep learning: Further topics

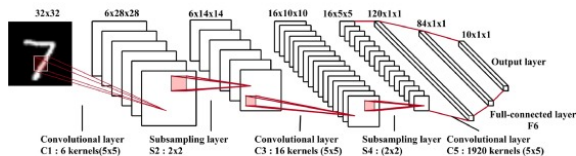
ImageNet

- ▶ A more complex **benchmark** for computer vision is the **ImageNet** data base:
<http://www.image-net.org/challenges/LSVRC/2014/>
- ▶ There are 200 different **categories** and multiple objects can be in one image.
- ▶ Over 450k images as training and 40k images as test.
- ▶ The sizes can be different with an average **image size** of 482×415 pixels.

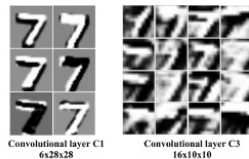


Popular network architectures

- ▶ **Convolutional neural networks** were first developed and used in the 1990's by Yann LeCun and others.
- ▶ The well-known **LeNet** architecture was successfully used to read handwritten zip codes.
- ▶ Since then, uncountable **new architectures** have been proposed, continuously improving the scores on the benchmarks like **ImageNet**.



(a) LeNet-5 network



(b) Learned features

Image from <https://www.sciencedirect.com/science/article/pii/S0031320317304120>

AlexNet

- ▶ AlexNet first popularized convolutional neural networks in computer vision, and that systematically uses ReLU instead of sigmoid activations.
- ▶ It was submitted by Alex Krizhevsky, Ilya Sutskever and Geoff Hinton to the ImageNet ILSVRC challenge in 2012.
- ▶ It improved the then best error of 26% to 16%.
- ▶ The architecture is similar to LeNet, but was deeper, bigger, and had convolutional layers stacked on top of each other (previously, one used only a single conv. layer immediately followed by pooling).

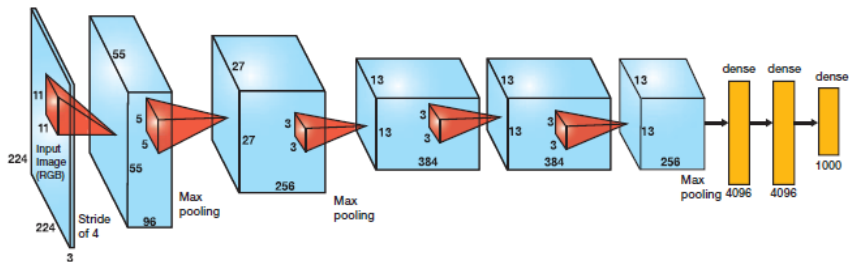


Image from <http://cv-tricks.com/cnn/understand-resnet-alexnet-vgg-inception/>

VGGNet

- ▶ The winner of the ILSVRC 2014 challenge was the **VGGNet** by Karen Simonyan and Andrew Zisserman.
- ▶ It proved that the **depth** of the network is **critical** for good performance.
- ▶ Their final model has a very **homogeneous** architecture with 16 layers that only performs **3×3 convolutions** and **2×2 pooling** from the beginning to the end.
- ▶ A downside of the **VGGNet** is that it uses a lot more **memory** and **parameters (140M)**.

Classical CNN topology - VGGNet (2013)

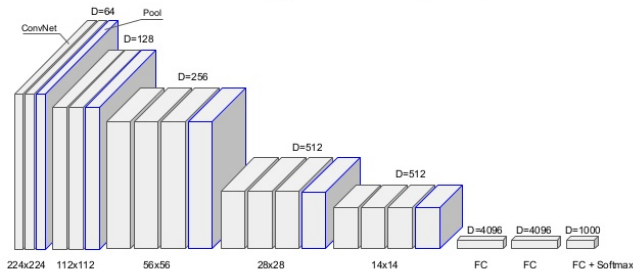
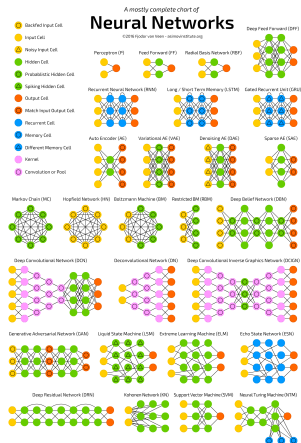


Image from <http://srdas.github.io/DLBook/ConvNets.html>

More networks

- ▶ Besides these, there are many other variations of these networks and different types of architectures (e.g., **ResNet** or Google's **Inception**).
- ▶ The variety is due to different applications, and to **active research** and advances in this area.



- ▶ For an overview see <https://github.com/BVLC/caffe/wiki/Model-Zoo>

Typical filter weights of first layer

- ▶ The filters of the first layer can be **interpreted** since they operate directly on the image.
- ▶ They extract **generic features** and can be seen as **edge or blob detectors**, for instance.

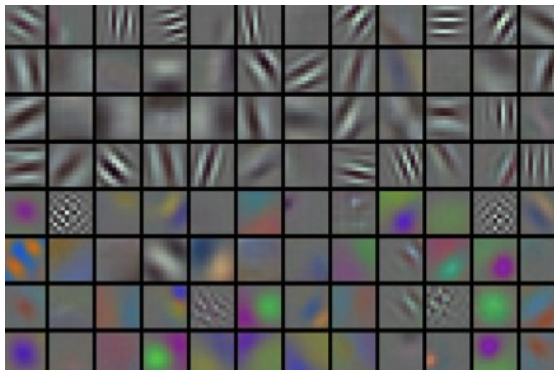


Image from <http://cs231n.github.io/neural-networks-3/>

Unconverged filters

- ▶ Sometimes the filters do not look as nice as on the previous slide, and they will not extract **clear features**.
- ▶ There can be different reasons for this.
- ▶ The network might not be **trained sufficiently**.
- ▶ The **learning rate** might be chosen poorly, or very low **weight regularization** is used.

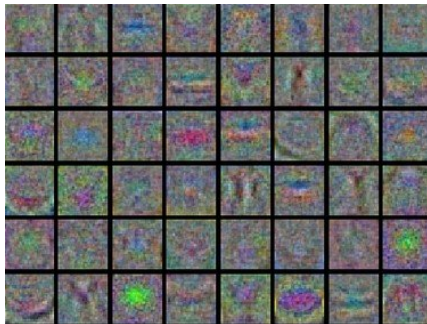


Image from <http://cs231n.github.io/neural-networks-3/>

Activations of deeper layers

- ▶ The filters of later, deeper layers are more **specific to the data set** used during training.
- ▶ The **task of a specific** neuron can be visualized by plotting input images that **maximally activate** that neuron.
- ▶ Below are examples for six neurons in the 5th (last) pooling of **AlexNet**.

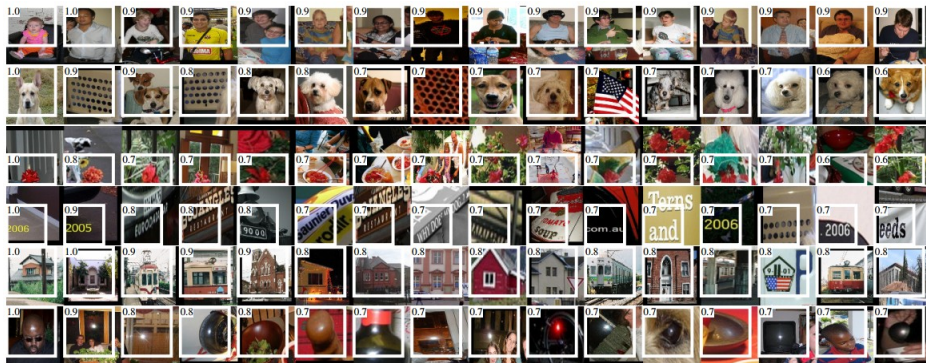


Image from Girshick et al. (2013, <https://arxiv.org/abs/1311.2524>)

Pre-trained networks

- ▶ As an end-user, you might neither have the **data** nor the **computational resources** to fit a modern network yourself **from scratch**.
- ▶ The expensive part is the **fitting**, not the **forward-pass**.
- ▶ As an alternative you can use a state-of-the-art **network** that has been **pre-trained** on large data sets such as **ImageNet**.
- ▶ The **weights** for many different networks are available, e.g., <https://github.com/BVLC/caffe/wiki/Model-Zoo>
- ▶ This will only allow you to perform **exactly the same task** on your data (i.e., same **input size**, same **output classes**) as the network was trained to do.
- ▶ The input image size is not a problem, since images can typically be **scaled** to have the **desired format**.

Transfer learning

- ▶ A main issue with pre-trained networks is that they can only predict their **pre-defined categories**.
- ▶ Usually, your problem will require to predict **other categories**, e.g., flower species, cancer types, car brands, traffic signs, etc.
- ▶ Moreover, you might only have a **small number** of training samples, certainly not enough to train a complicated CNN.
- ▶ **Transfer learning** is the idea to build on information from a pre-trained model, but adapt this model to your problem.
- ▶ You might for instance use a model trained on **ImageNet** to build a model that distinguishes different types of galaxies.

Transfer learning

- ▶ How can this work?
- ▶ You should see a CNN as consisting of two parts: the **generic feature extractor** and the **problem specific classifier**.

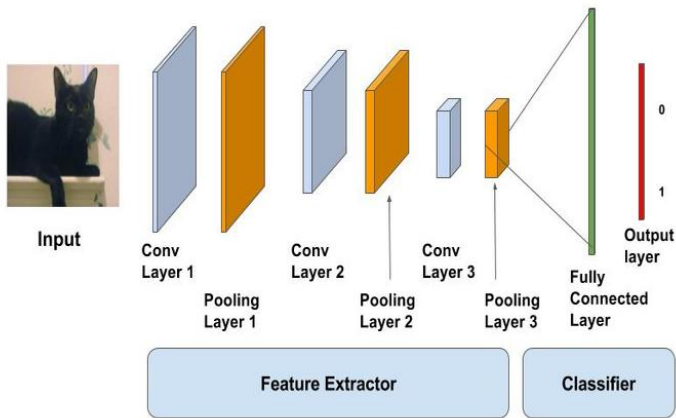


Image from <https://www.learnopencv.com/keras-tutorial-transfer-learning-using-pre-trained-models/>

Transfer learning

- More mathematically, **transfer learning** can be represented as follows.

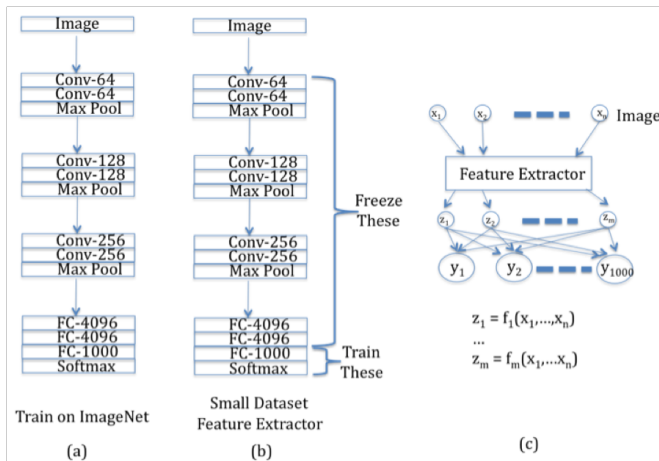


Image from <http://srdas.github.io/DLBook/ConvNets.html#transfer-learning>

Transfer learning

1. Strategy

- ▶ Take a **pre-trained CNN** (typically on **ImageNet**), e.g., **AlexNet**.
- ▶ Remove the last fully connected layer whose output are the 1000 different **class probabilities**.
- ▶ For all your new images, use the rest of the network as a feature extractor with **fixed weights**.
- ▶ For each image you obtain a the activations/features (also called **codes**) of the last but one layer; for **AlexNet** a vector of dimension 4096.
- ▶ Train a **linear classifier** on the codes, e.g., **multinomial logistic regression** or **linear SVM**.

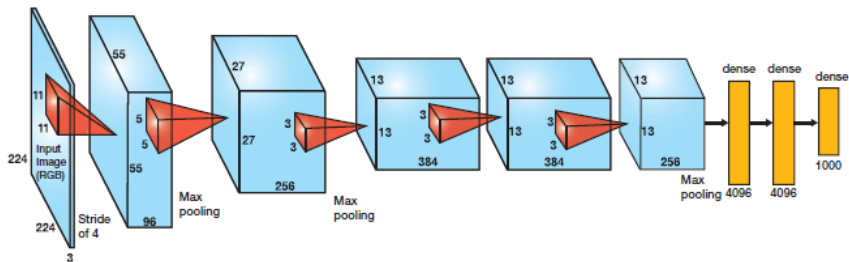


Image from <http://cv-tricks.com/cnn/understand-resnet-alexnet-vgg-inception/>

Transfer learning

2. Strategy

- ▶ Alternatively, you can also **fine-tune** the CNN.
- ▶ Instead of only replacing and retraining the classifier, you can keep only some **early layers fixed** and fine-tune the later layers.
- ▶ This makes sense, since the early features of a CNN are generic such as **edge or blob detectors**, which are useful to many tasks.
- ▶ The later layers are more **specific** to the data set used during training.

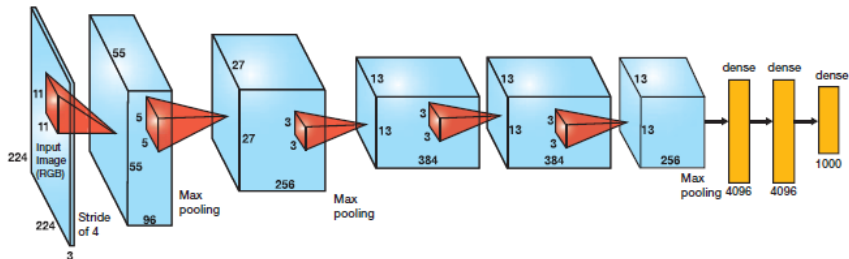


Image from <http://cv-tricks.com/cnn/understand-resnet-alexnet-vgg-inception/>

Transfer learning

- ▶ Which of the strategies you should choose depends on **different factors**: the **size** of your data set, the **similarity** to the original data set of the pre-trained model, etc.
- ▶ If your data set is **small** and **very similar** to the original data set, you should only train the classifier but not fine-tune the CNN due to **overfitting problems**.
- ▶ If your data set is **big** and **not too different** to the original data set, you can additionally fine-tune the CNN with less danger of overfitting.

Further topics

- ▶ Around “classical” deep neural networks there are plenty of new **applications/variants/extensions** that are published each month.
- ▶ For instance, **recurrent neural networks** can be used for times series forecasting.
- ▶ A popular topic at the moment are for instance **Generative Adversarial Networks** (GANs), see Goodfellow et al. (2014).
- ▶ So-called **autoencoders** use deep neural networks to do dimension reduction and data compression.
- ▶ Deep neural network are also used in **Reinforcement Learning**.

Reinforcement Learning

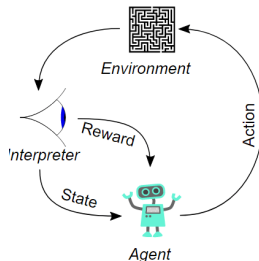


Image from https://en.wikipedia.org/wiki/Reinforcement_learning

- ▶ Reinforcement Learning involves an agent who takes actions in an environment; this action results in a **reward** and a change of the **state**; over many iteration the agent learns an optimal **strategy of action** to maximize the reward.
- ▶ In complex settings, this learning involves **deep neural networks**.
- ▶ “An Outsider’s Tour of Reinforcement Learning”:
<http://www.argmin.net/2018/06/25/outsider-rl/>
- ▶ A nice **Reinforcement Learning course** by David Silver (**AlphaGo**) can be found on youtube.
- ▶ And to learn how computers defeated humans in the game **Go**, watch the documentary film **AlphaGo**.